CHAPTER 3 — PART A: OBJECTS, REFERENCES & MEMORY

Objects • References • Memory Model • Aliasing • null • Pass-by-Value • Equality • Immutability • Garbage Collection

JAVA FUNDAMENTALS

WHY THIS MATTERS

- Understanding how objects and references work is essential for writing correct, efficient Java code.
- Helps you avoid common bugs related to sharing, modification, and memory management.

MASTER THESE

- How objects are created and referenced in memory.
- When two variables refer to the same object.
- The difference between identity and logical equality.
- How mutability, parameter passing and garbage collection work.

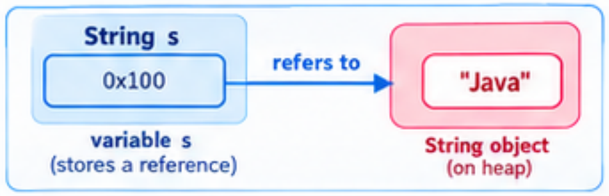
KEY TAKEAWAY

- Variables hold references to objects, not the objects themselves.
- Multiple references can refer to the same object.
- Java is pass-by-value (value of the reference is passed).
- Use equals() for logical equality; == checks identity.
- Objects become eligible for garbage collection when unreachable.

1 Objects vs References

Variables hold references to objects

- An object is an instance of a class, stored in memory (on the heap).
- A variable of a reference type stores a reference (address) to that object.
- Multiple variables can refer to the same object.



JAVA EXAMPLE • Object and reference

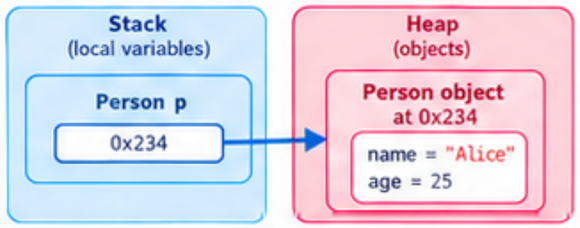
```
String s = "Java";
String t = s;
System.out.println(s);
```

OUTPUT `Java`

2 Object Creation & Memory Model

Stack holds references, heap holds objects

- Objects are created using new and stored on the heap.
- Reference variables (local variables) live on the stack and store the address of the object.
- The stack and heap are separate areas of memory.



JAVA EXAMPLE • Creating an object

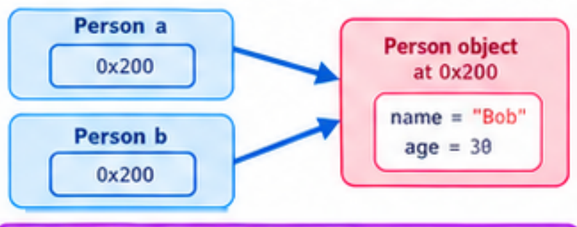
```
Person p = new Person("Alice", 25);
System.out.println(p.name);
```

OUTPUT `Alice`

3 Aliasing

Two references can point to the same object

- Two or more variables can refer to the same object (aliasing).
- Changes made through one reference are visible through all references to that object.



JAVA EXAMPLE • Aliasing example

```
Person a = new Person("Bob", 30);
Person b = a;          // both refer to same object
b.age = 31;
System.out.println(a.age);
```

OUTPUT `31`

4 null and NullPointerException

null means no object is referenced

- null means the reference does not point to any object.
- Calling a method or accessing a field on null throws a NullPointerException.
- Always check for null before using a reference.

JAVA EXAMPLE • null and safe check

```
Person p = null;
// System.out.println(p.name);  // throws NPE

if (p != null) {
  System.out.println(p.name);
} else {
  System.out.println("p is null");
}
```

OUTPUT `p is null`

COMMON RISK
Calling a method or accessing a field on a null reference will throw a NullPointerException.

5 Java Pass-by-Value

A copy of the value is passed to methods

- Java is always pass-by-value.
- For primitives, a copy of the value is passed (changes do not affect the original).
- For objects, a copy by the reference is passed (the method can modify the object, but reassigning the parameter does not change the caller's reference).

JAVA EXAMPLE • primitives vs objects

```
public class Demo {
  static void changeValue(int x) { x = 20; }
  static void changeName(Person p) { p.name = "Ann"; }
  static void reassign(Person p) { p = new Person("New", 0); }

  public static void main(String[] args) {
    int n = 10; changeValue(n);
    System.out.println(n);                    // 10

    Person p = new Person("Sam", 20);
    changeName(p); System.out.println(p.name); // Ann
    reassign(p);   System.out.println(p.name); // Ann
  }
}
```

OUTPUT
```
10
Ann
Ann
```

6 == versus equals ()

Identity vs logical equality

- == checks if two references point to the same object (identity).
- equals() checks if the contents are logically equal (defined by the class).
- For String, equals() compares the characters.

JAVA EXAMPLE • String equality

```
String a = new String("Java");
String b = new String("Java");

System.out.println(a == b);       // different objects
System.out.println(a.equals(b));  // same content

String s = "Java";
String t = s + " Rocks";
System.out.println(s);
System.out.println(t);
```

OUTPUT
```
false
true
Java
Java Rocks
```

7 Mutability, Immutability & Defensive Copying

Some objects can change, some cannot

- Immutable objects (e.g., String) cannot be changed. Operations create new objects.
- Mutable objects (e.g., StringBuilder, custom classes) can be changed.
- Use defensive copying to protect internal state from unintended modification.

JAVA EXAMPLE • immutability vs mutability

```
String s = "Java";
String t = s + " Rocks";
System.out.println(s);   // original unchanged
System.out.println(t);   // new String

StringBuilder sb = new StringBuilder("Hi");
sb.append("!");
System.out.println(sb);  // modified in place
```

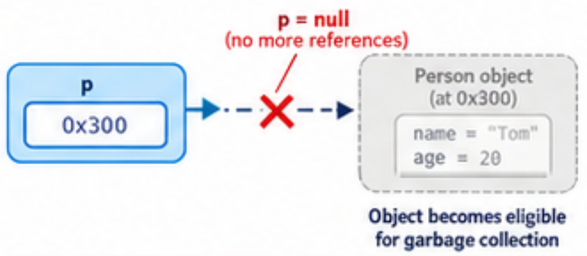
OUTPUT
```
Java
Java Rocks
Hi!
```

DEFENSIVE COPYING
Create a new copy of mutable objects when passing them in or returning them, to prevent external changes.

8 Garbage Collection & Reachability

Unreachable objects can be collected

- Objects that are no longer reachable from any live reference become eligible for garbage collection.
- Garbage collection reclaims memory automatically.
- The timing of collection is not guaranteed.



IMPORTANT
- You cannot force garbage collection to happen.
- System.gc() is only a hint; it does not guarantee immediate collection.
- Garbage collection reclaims memory for unreachable objects automatically.

COMMON MISTAKES TO AVOID

- Using == to compare String content (use equals()).
- Assuming Java is pass-by-reference.
- Forgetting that multiple references can refer to the same object (aliasing).
- Calling methods or accessing fields on a null reference.
- Assuming System.gc() guarantees immediate collection.

QUICK REFERENCE SUMMARY

- Variables hold references to objects.
- Objects live on the heap; references are on the stack.
- Multiple references can point to the same object (aliasing).
- Java is pass-by-value (copy of the value, including references).
- == checks identity; equals() checks logical equality.
- String is immutable; many other objects are mutable.
- Use defensive copies to protect internal state.
- Unreachable objects become eligible for garbage collection (but timing is not guaranteed).

PART A

OBJECTS, REFERENCES MEMORY

YOUR LEARNING OBJECTIVES

- new creates an object; variables hold reference values.
- Assignment copies a reference; it does not duplicate the object.
- Java passes all arguments by value, even when the values are references.
- == tests reference identity; equals() may test logical equality.
- final does not freeze mutable objects; GC is based on reachability.

19 focused concept lessons • one infographic

Mini exercises and reasoned answers included

Understand identity, aliasing, pass-by-value, equality, mutability and reachability.

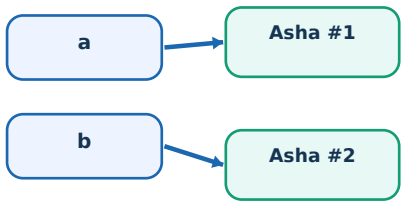
Objects are instances; classes describe them

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A class defines a type. new creates a distinct object of that type, invokes a constructor, and produces a reference that you can store. Identical field values do not imply identical objects. a and b receive separate references because new executed twice. A reference variable is not the object itself. Java checks allowed operations from the declared type.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person("Asha");
Person b = new Person("Asha");
System.out.println(a == b);
System.out.println(a.getName());
```

EXPECTED OUTPUT

```
false
Asha
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The first new runs a constructor and returns a reference to object #1.
2. The second new returns a reference to object #2, even though its name is also Asha.
3. == compares those references; getName() reads a field on object #1.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A variable declaration such as Person p; only reserves a reference slot. Construction is a separate expression. The declared type controls what members the compiler lets you call; the object's identity and its field values are different properties. A failed constructor does not return a successfully constructed object.

COMMON TRAP

Do not equate a variable name with a physical heap address; references are abstract language-level values.

TRY THIS IN INTELLIJ

Replace b with a. Predict the equality check.

REMEMBER Two new expressions normally mean two distinct objects.

APPLY THE RULE / CHANGE ONE LINE

Replace the second new Person("Asha") with Person b = a;

Now a == b is true: the same object has two aliases rather than two identities.

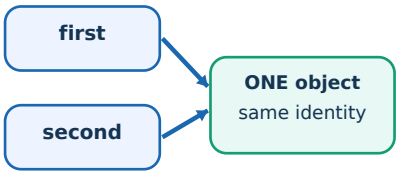
A practical stack-versus-heap mental model

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Think of local variables as slots in the running method and objects as separately allocated entities; Java does not promise a particular physical location for every value. Both local variables hold the same reference value; there is one Person object in this example. The JVM may optimize allocation, so diagrams of “stack references / heap objects” are conceptual teaching models, not guaranteed memory layouts.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person first = new Person("Asha");
Person second = first;
System.out.println(first == second);
```

EXPECTED OUTPUT

```
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. first stores a reference returned by the constructor.
2. second = first copies that same reference value into a second local variable.
3. == is true because both variables identify the same object.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A stack/heap picture is a reasoning aid: method invocations have local variables and objects have independent identity. JVM optimizations can keep values in registers, inline data, or remove allocations. None of those optimizations changes the Java-visible result: two aliases can refer to one object.

COMMON TRAP

A String or an object field can hold references too; not every reference is in a stack frame.

TRY THIS IN INTELLIJ

Draw the two variable arrows pointing to a single box.

REMEMBER Draw reference arrows, not physical memory addresses.

APPLY THE RULE / COMPARE

Replace second = first with second = new Person("Asha");

The name values match, but the references now identify separate objects; == is false.

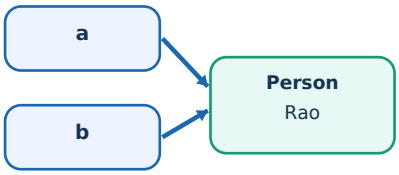
Reference assignment creates an alias

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Assigning one reference variable to another copies the reference value; subsequent mutations through either alias affect the same object. There are two variables but one Person. The method call changes the state of the object that both variables identify. Reassigning b later does not change a: aliases are copied values, not linked variables.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person("Asha");
Person b = a;
b.setName("Rao");
System.out.println(a.getName());
System.out.println(a == b);
```

EXPECTED OUTPUT

```
Rao
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new creates one mutable Person, initially named Asha.
2. b = a copies the reference; it does not create another Person.
3. b.setName updates the shared object, so a.getName() reads Rao.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Aliasing matters any time a method holds a reference received from elsewhere. If either alias mutates shared state, all observers see that state when they read it. By contrast, b = new Person(...) changes only b's reference slot. Copying a reference is cheap, but sharing mutability needs deliberate ownership.

COMMON TRAP

Confusing assignment of a reference with cloning an object is a frequent bug.

TRY THIS IN INTELLIJ

Now set b = new Person("New"); and print a.getName().

REMEMBER Reference assignment copies an arrow, not an object.

APPLY THE RULE / MUTATION VS REBINDING

After the setter, add b = new Person("New");

a still reads Rao. Rebinding b changed no field on the original Person.

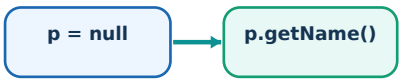
null: a reference with no object

Understand the rule • trace the code • read the diagram • test it

CORE RULE

null is a special value that can be assigned to reference types. It does not point to an object, so dereferencing it is invalid. The equality test is safe; invoking an instance method on null throws NullPointerException. A field may default to null; local reference variables must be definitely assigned before use.

VISUAL MODEL



NO TARGET
NPE

CODE EXAMPLE / ALLMAN BRACES

```
Person p = null;
System.out.println(p == null);
try
{
    System.out.println(p.getName());
}
catch (NullPointerException ex)
{
    System.out.println("Cannot dereference null");
}
```

EXPECTED OUTPUT

```
true
Cannot dereference null
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. p holds null, not a reference to an existing Person.
2. p == null safely evaluates to true without dereferencing anything.
3. p.getName() attempts an instance call through null and throws NullPointerException, which the catch handles.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Distinguish a null reference from an uninitialized local variable. Instance fields default to null, but local references require definite assignment. Prefer clear non-null APIs and validation at boundaries; Objects.requireNonNull can fail early with a helpful message. Catching NPE is usually not a substitute for correct design.

COMMON TRAP

A null check just before use is not a universal concurrency guarantee; prefer clear ownership and non-null invariants.

TRY THIS IN INTELLIJ

Use if (p != null) to guard a call and predict what prints.

REMEMBER Checking null is safe; dereferencing null is not.

ONE MORE QUESTION / SAFER ALTERNATIVE

Guard the call with if (p != null) before using p.getName().

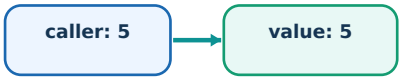
Pass-by-value with primitive parameters

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Java always passes argument values to parameters. For primitives, the parameter gets a copy of the numeric or boolean value. The parameter named value is local to the called method. Incrementing it does not write back to count. A return value can communicate the computed result. Calling count = addOne(count) would require addOne to return int.

VISUAL MODEL



COPY
caller stays 5

CODE EXAMPLE / ALLMAN BRACES

```
static void addOne(int value)
{
    value++;
}
int count = 5;
addOne(count);
System.out.println(count);
```

EXPECTED OUTPUT

```
5
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. count contains the primitive int value 5 in the caller.
2. The call copies that value into the method's separate parameter slot named value.
3. value++ changes only the parameter; the caller's count remains 5.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Java has no ordinary pass-by-reference parameters. A method can compute a new value and return it, but the caller must assign the result. The callee receives the evaluated argument expression, not access to the caller's local variable slot. That remains true if you call addOne(count + 10).

COMMON TRAP

Java has no pass-by-reference parameter syntax for ordinary methods.

TRY THIS IN INTELLIJ

Change addOne to return value + 1 and assign the result.

REMEMBER Primitive parameters receive copied values.

APPLY THE RULE / RETURN A RESULT

Change the method to return value + 1 and write count = addOne(count);

The caller explicitly stores the returned result; count then becomes 6.

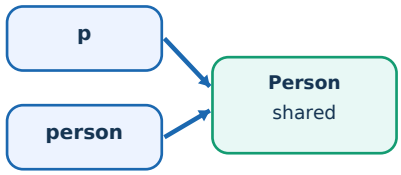
Pass-by-value with object references

Understand the rule • trace the code • read the diagram • test it

CORE RULE

For objects, Java passes a copy of the reference value. The called method can use it to reach and mutate the same object. The reference was copied, but both copies identify the same Person instance during the call. This is not a special pass-by-reference rule; the copied value simply happens to be a reference.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static void rename(Person person)
{
    person.setName("New name");
}
Person p = new Person("Old name");
rename(p);
System.out.println(p.getName());
```

EXPECTED OUTPUT

```
New name
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. p receives a reference to the Person named Old name.
2. rename(p) copies that reference into the method parameter person.
3. person.setName updates the one shared Person, so the caller observes New name.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The copies are references to the same object, not copies of the object itself. Think in two layers: parameter assignments affect local slots, while setter calls affect object state. This distinction works the same with List, Map, and interface-typed parameters. Passing null succeeds as an argument but dereferencing it inside rename would fail.

COMMON TRAP

“Objects are passed by reference” is a misleading shortcut in Java discussions.

TRY THIS IN INTELLIJ

Would rename(null) fail at call time or at the dereference?

REMEMBER Pass-by-value can still produce visible object mutations.

APPLY THE RULE / FAILURE LOCATION

Call rename(null) with the original method body.

Passing null is permitted. The exception occurs when person.setName dereferences it.

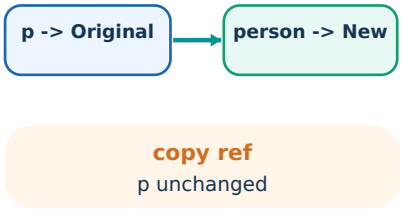
Rebinding a parameter does not rebind the caller

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Reassigning an object parameter replaces only the local parameter reference. It does not replace the reference stored in the calling method. Java first copies p into person. Assigning a new reference to person cannot alter p. Contrast this with person.setName(...): that operation changes the shared object rather than the parameter slot.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static void replace(Person person)
{
    person = new Person("Replacement");
}
Person p = new Person("Original");
replace(p);
System.out.println(p.getName());
```

EXPECTED OUTPUT

```
Original
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The caller's p and the parameter person initially identify the Original object.
2. person = new Person(...) creates object #2 and changes only the parameter slot.
3. After the method returns, p still identifies object #1, so Original prints.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

You can mutate an object through a parameter and later rebind the parameter; the earlier mutation persists. What never happens is Java silently writing a new reference back into the caller's variable. To replace the caller's object, return a new reference and explicitly assign it: p = replace(p).

COMMON TRAP

Two methods that “receive a Person” may behave very differently: one mutates, the other merely rebinds.

TRY THIS IN INTELLIJ

Combine replace with a mutation before reassignment; what changes persist?

REMEMBER Rebinding a parameter cannot redirect a caller's variable.

APPLY THE RULE / MUTATE THEN REBIND

Call person.setName("Changed") before person = new Person(...).

p still refers to the original object, but its name becomes Changed.

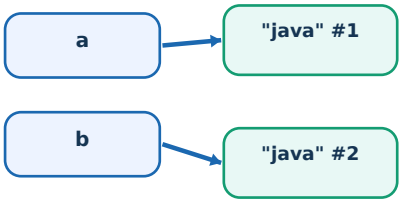
Identity comparison: == for references

Understand the rule • trace the code • read the diagram • test it

CORE RULE

For two reference expressions, == asks whether they refer to the same object (or are both null), not whether their contents match. This deliberate use of new constructs distinct String instances, while equals compares their character sequences. The behavior of == for primitive expressions differs: it compares primitive values after applicable numeric promotion.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
String a = new String("java");
String b = new String("java");
System.out.println(a == b);
System.out.println(a.equals(b));
```

EXPECTED OUTPUT

```
false
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new String creates a fresh String object for a; the second new creates another for b.
2. a == b is false because the two references identify different objects.
3. a.equals(b) is true because both strings contain the characters java.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

String literals may refer to pooled objects, so comparing two identical literals with == may sometimes look correct. It is still the wrong question when you mean textual equality. Use a.equals(b) when a is non-null or Objects.equals(a, b) when either side can be null. Identity tests are valid only when sameness is intentional.

COMMON TRAP

String interning may make == appear to work for some literals. Never use that as a text comparison strategy.

TRY THIS IN INTELLIJ

Test String a = "java"; String b = "java"; then explain why that case is different.

REMEMBER For text content, use equals(), not ==.

APPLY THE RULE / WHY LITERALS MISLEAD

Try String a = "java", b = "java"; with ==.

Interned literals can compare true by identity; that does not make == a text comparison.

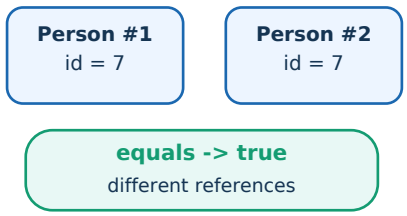
equals(): logical equality and contracts

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Classes may define logical equality by overriding equals(Object). The contract includes reflexive, symmetric, transitive, consistent, and null-safe behavior. This assumes Person.equals is based on stable id, as in the companion project. Different instances can then be logically equal. An equality definition should use fields that match the domain meaning of identity and should remain stable when objects are used as keys.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person(7, "Asha");
Person b = new Person(7, "Rao");
System.out.println(a == b);
System.out.println(a.equals(b));
```

EXPECTED OUTPUT

```
false
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The two Person constructors return distinct references, making == false.
2. The project Person.equals implementation compares a stable id instead of the name.
3. Both objects have id 7, so equals returns true even though their names differ.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

An equals override must accept Object and respect reflexivity, symmetry, transitivity, consistency, and false for null. Decide whether your type represents entity identity or a set of values. Fields used in equality should ideally be stable while the object is in collections. An equals(Person) overload does not override Object.equals.

COMMON TRAP

Overloading equals(Person) instead of overriding equals(Object) is not the standard equality override.

TRY THIS IN INTELLIJ

Check a.equals(null), b.equals(a), and a.equals(a).

REMEMBER Logical equality is a class-defined contract, not reference identity.

APPLY THE RULE / OVERRIDE VS OVERLOAD

Write equals(Person p) but leave equals(Object) unchanged.

Collections and Object-typed calls still use the inherited Object.equals implementation.

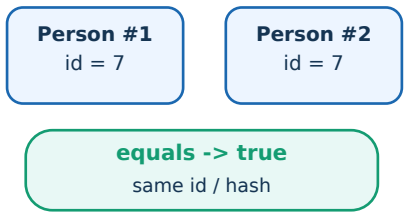
hashCode must agree with equals

Understand the rule • trace the code • read the diagram • test it

CORE RULE

If two objects are equal under equals, they must have the same hashCode. The reverse is not required: collisions are allowed. Collections that use hashing rely on this contract. Violation can make an equal object impossible to find in a hashed structure. Java Object supplies default identity-based implementations; override both when defining value equality.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person(7, "Asha");
Person b = new Person(7, "Rao");
System.out.println(a.equals(b));
System.out.println(a.hashCode() == b.hashCode());
```

EXPECTED OUTPUT

```
true
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The objects use the same equality-defining id, so equals returns true.
2. Their hashCode implementations use that same id, so hash codes match.
3. A hash-based collection can use the consistent hash to locate a candidate bucket before checking equals.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The one-way rule is: equal objects MUST have equal hash codes. Unequal objects can collide. A hash code is not a unique identifier or an address. If a key changes its hash-relevant fields after insertion into HashMap or HashSet, lookups may fail even though the object still exists.

COMMON TRAP

Never use mutable fields in equality/hashCode for objects that remain in hashed collections unless you control mutation carefully.

TRY THIS IN INTELLIJ

What could happen if a key's hash-relevant field changes after insertion?

REMEMBER When overriding equals(), also override hashCode().

APPLY THE RULE / HASH COLLECTION WARNING

Insert an object into a HashSet, then change a field used by hashCode.

The set may no longer locate the object using the changed hash value.

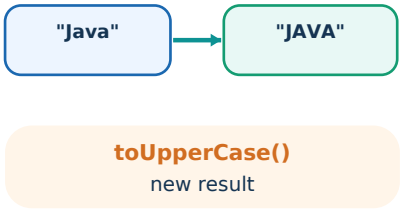
String immutability is different from variable immutability

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A String cannot change its character sequence. A String variable can still be rebound to a different String unless the variable is final. toUpperCase returns a String value; ignoring it leaves text unchanged. The second assignment updates the variable to a new result. Immutable objects are easier to share safely because callers cannot change their internal state.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
String text = "Java";
text.toUpperCase();
System.out.println(text);
text = text.toUpperCase();
System.out.println(text);
```

EXPECTED OUTPUT

```
Java
JAVA
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. text initially refers to the immutable String Java.
2. toUpperCase returns a result, but the first return value is ignored; text still prints Java.
3. The assignment text = text.toUpperCase() rebinds the variable; the next print is JAVA.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Immutability describes an object's state, not whether a variable can change its reference. A String method such as substring, strip, replace, or concat returns a value rather than changing the receiver. For lots of character-by-character edits, StringBuilder provides a mutable buffer, but it is a different type.

COMMON TRAP

Do not assume every method whose name sounds like an edit mutates its receiver.

TRY THIS IN INTELLIJ

Predict text.concat(" 17") without assigning the result.

REMEMBER An immutable object can be referenced by a reassignable variable.

APPLY THE RULE / IGNORED RETURN VALUE

Run text.concat(" 17"); without assigning its result.

The original String is unchanged. Print text to verify before and after.

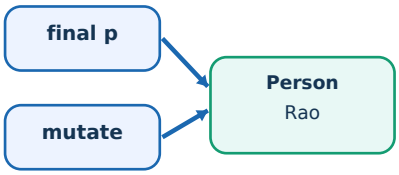
final reference versus mutable referent

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A final local reference cannot be reassigned after initialization, but that restriction does not freeze the referenced object. final applies to the variable slot. The mutable Person instance still permits valid changes through its methods. For deeper immutability, design the object with private final fields and do not expose mutable internal objects.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
final Person person = new Person("Asha");
person.setName("Rao");
System.out.println(person.getName());
// person = new Person("Other"); // Error
```

EXPECTED OUTPUT

Rao

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. final binds the local variable person once to a Person reference.
2. person.setName mutates the Person, so the new name Rao is visible.
3. Reassigning person is a compile-time error, even though mutating its fields was allowed.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

final applies at the variable-reference layer. Immutable object design goes further: private final fields, no mutators, and no leaking mutable members. A final List variable can still call add() on a mutable list. Keep the distinction between 'cannot rebind' and 'cannot change observable state'.

COMMON TRAP

Declaring List or Map references final does not make their contents immutable.

TRY THIS IN INTELLIJ

Design an immutable Name object with one final field and no setter.

REMEMBER final reference does not mean immutable object.

APPLY THE RULE / FINAL IS SHALLOW

Make a final reference to a mutable ArrayList, then call add().

The list contents change normally even though the reference cannot be rebound.

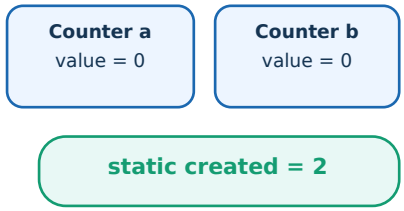
Fields, local variables, and static state

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Instance fields belong to each object. Static fields belong to the class. Local variables belong to a method invocation and require definite assignment. Each new Counter has its own value field, initially zero; created is shared across all Counter instances. Field default initialization occurs before constructor logic. A local int x; does not receive a usable default.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Counter
{
    static int created;
    int value;
    Counter()
    {
        created++;
    }
}
Counter a = new Counter();
Counter b = new Counter();
System.out.println(Counter.created);
```

EXPECTED OUTPUT

```
2
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Constructing a initializes its own instance value field and increments Counter.created.
2. Constructing b gets a different instance field but increments the same static created field.
3. Counter.created prints 2 because the static counter is associated with the class.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Fields receive default values before constructor code executes: numeric zero, false, or null. A local variable has no usable default; Java's definite-assignment analysis prevents reading it uninitialized. static changes ownership, not visibility: static fields still follow their access modifiers, and shared mutable static data can be a concurrency risk.

COMMON TRAP

Shared mutable static state can create surprising interactions and thread-safety concerns.

TRY THIS IN INTELLIJ

Set a.value=9. Predict b.value.

REMEMBER Instance fields are per object; static fields are shared per class.

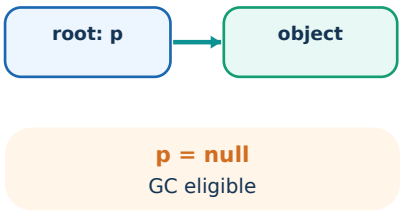
Garbage collection and object reachability

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Objects become eligible for garbage collection when they are no longer reachable through live references. Collection timing is not specified. Clearing the only live reference may make an object eligible for collection, but JVM optimization and other references affect reachability. The JVM handles memory reclamation; GC does not replace deterministic closing of files, sockets, or database connections.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person p = new Person("Short lived");
p = null;
System.out.println("Reference cleared");
```

EXPECTED OUTPUT

```
Reference cleared
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new makes a Person object reachable through the local p.
2. p = null removes that local path to the object.
3. The print statement only confirms that the reference was cleared; it does not prove GC ran.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Reachability is what matters to garbage collection, not the execution of a particular null assignment. The runtime may delay collection and optimize object lifetime. Do not build behavior around finalizers or System.gc(). File handles, sockets and database connections have separate lifecycles; use try-with-resources for reliable closing.

COMMON TRAP

Do not predict exactly when finalize or garbage collection will run, and do not use System.gc() as a correctness mechanism.

TRY THIS IN INTELLIJ

If another variable holds the same reference, is the object eligible?

REMEMBER Eligible for GC is not the same as collected now.

APPLY THE RULE / GC IS NOT A CALLBACK

Call System.gc() and then print a status message.

Only the message is deterministic; collection timing is not guaranteed.

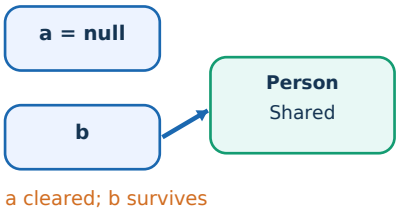
Reachability versus alias count

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An object can remain reachable through instance fields, static fields, locals, or other objects even if one variable is cleared. b still reaches the object, so clearing a does not make that shared object unreachable. Real object graphs can have cycles; Java garbage collectors use reachability rather than simple per-object reference counting.

VISUAL MODEL



a cleared; b survives

CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person("Shared");
Person b = a;
a = null;
System.out.println(b.getName());
```

EXPECTED OUTPUT

```
Shared
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. a receives a reference to the new Person Shared.
2. b = a makes b a second path to the same Person.
3. a = null removes one path, but b still reaches the object and prints Shared.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

GC traces paths from live roots, through object fields and collections, to reachable objects. A cycle can still be reclaimed if no roots reach it. A static cache or long-lived collection can unintentionally hold objects alive. Ask 'is there any path from a live root?' rather than counting how many pointers were assigned.

COMMON TRAP

A cycle alone does not keep objects alive if nothing reachable points into that cycle.

TRY THIS IN INTELLIJ

Draw the arrows before and after a=null.

REMEMBER One surviving path is enough to keep an object reachable.

APPLY THE RULE / TRACE THE LAST PATH

After a = null, also set b = null (assuming no other roots).

The object may become unreachable; that still does not say when reclamation occurs.

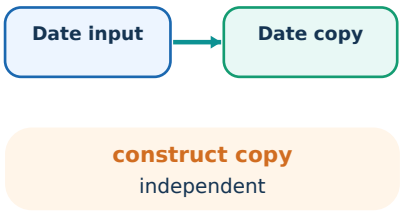
Defensive copying prevents ownership leaks

Understand the rule • trace the code • read the diagram • test it

CORE RULE

If a class accepts or returns a mutable object, it may need a copy to preserve its invariants and avoid external modifications. Date is mutable, so copying the timestamp separates ownership. java.time immutable classes are usually preferable for new code. Create copies both on input and output when a class must shield mutable fields from callers.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Date input = new Date(0);
Date copy = new Date(input.getTime());
input.setTime(99_000);
System.out.println(copy.getTime());
```

EXPECTED OUTPUT

```
0
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. input is a mutable Date representing 0 milliseconds since the epoch.
2. new Date(input.getTime()) constructs a separate Date with the same timestamp.
3. Changing input to 99,000 does not change copy, so copy prints 0.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

For a class that stores mutable caller-owned data, copy on input so the caller cannot change your internals later. Copy on output so readers cannot mutate your internals through a getter. A shallow copy of a container may still share nested mutable elements. Prefer java.time immutable date/time values for new APIs.

COMMON TRAP

Copying just the reference does not create a defensive copy. A shallow copy may still share nested mutable objects.

TRY THIS IN INTELLIJ

What happens if copy=input instead of new Date(...)?

REMEMBER Defensive copying separates ownership only as deeply as you copy.

APPLY THE RULE / NO COPY AT ALL

Replace the constructor call with Date copy = input; and mutate input.

Now copy and input alias one Date, so copy.getTime() becomes 99000.

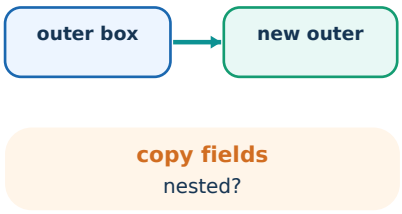
Copy constructors, shallow and deep copies

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A copy constructor can create a new outer object. Whether nested data is copied determines if the result is shallow or deep. For a primitive field, copying its value is straightforward. For a mutable nested reference, you must decide whether to create an independent nested object. Cloneable and Object.clone exist, but explicit constructors or factories are often clearer about what gets copied.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Box
{
    int value;
    Box(int value)
    {
        this.value = value;
    }
    Box(Box other)
    {
        this.value = other.value;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. A copy constructor receives an existing Box as input.
2. It constructs a new outer Box and copies the primitive value into its own field.
3. If Box later owns a mutable Date or List, merely copying that reference would still share nested state.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

'Deep copy' depends on the shape of the object graph and your ownership contract. Immutable nested values can safely be shared. Mutable nested objects may require new independent instances. Cycles and duplicated references complicate naive recursive copying; explicit constructors/factories often make copying intent clearer than clone().

COMMON TRAP

A “copy” method is not necessarily deep: inspect every mutable field.

TRY THIS IN INTELLIJ

Add a mutable Date field. How would you copy it defensively?

REMEMBER New outer object does not automatically mean deep copy.

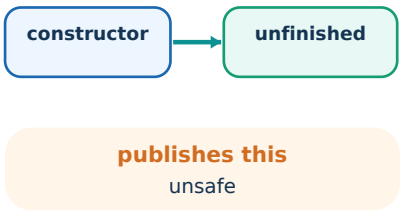
Object escape during construction

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Publishing this before a constructor finishes can expose a partially initialized object to other code or threads. Normally keep this private until initialization finishes. Constructor injection and factories can help control publication. Overridable method calls inside constructors are also risky: they may dispatch to a subclass before its fields initialize.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Unsafe
{
    static Unsafe published;
    String name;
    Unsafe()
    {
        published = this; // avoid early escape
        name = "ready";
    }
}
```

EXPECTED OUTPUT

```
No console output (illustrates design risk).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The Unsafe constructor begins before the name field is assigned ready.
2. published = this exposes a reference to the in-progress object.
3. Only afterward does the constructor set name; other observers may have seen incomplete state.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Constructor escape is particularly dangerous when a listener, callback, static registry, or other thread can inspect the object before setup is complete. Calling overridable methods from a superclass constructor can invoke subclass code before subclass initialization. Finish validation and initialization, then publish the fully built object through a factory or normal return path.

COMMON TRAP

The danger may be subtle, especially with concurrent execution or inheritance.

TRY THIS IN INTELLIJ

Why is calling an overridable method from a base constructor hazardous?

REMEMBER Construct completely first; publish the reference afterward.

ONE MORE QUESTION / SAFE CONSTRUCTION RULE

Move publication to a factory after the constructor returns.

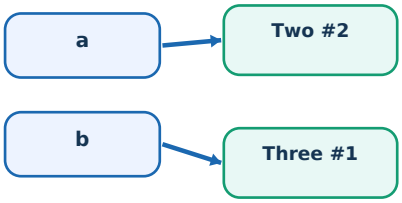
Reference tracing: one object or two?

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Read each assignment as moving reference values between variable slots. Mutations change objects; reassignment changes only the selected slot. The first new creates object #1. b aliases it. The second new creates object #2, and only a is rebound. b continues to refer to object #1. When debugging state changes, draw arrows to objects rather than mentally treating variables as containers of entire objects.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person("One");
Person b = a;
a = new Person("Two");
b.setName("Three");
System.out.println(a.getName());
System.out.println(b.getName());
System.out.println(a == b);
```

EXPECTED OUTPUT

```
Two
Three
false
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The first new makes object #1 named One, and b aliases a.
2. The second new makes object #2 named Two; only a is rebound to object #2.
3. b.setName changes object #1 to Three, so prints are Two, Three, false.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A reliable trace uses one box per new expression, one arrow per reference variable, and state updates only for setters. At each assignment, erase/redraw just the variable's arrow. This works for larger object graphs too, including services with shared dependencies and collections that store object references.

COMMON TRAP

Do not confuse two variable names with two objects, or one variable name with only one lifetime object.

TRY THIS IN INTELLIJ

Change the order of the last three statements and predict the output.

REMEMBER To trace state, count objects and track arrows separately.

APPLY THE RULE / ARROWS FIRST

After the second new, ask which object b points to before changing its name.

b still points to object #1; a points to object #2. The setter affects only #1.

CHAPTER 3 · PART A · VISUAL MASTER MAP

Core rules • mental models • common pitfalls • executable examples



01 OBJECT CREATION

- new constructs a distinct object.
- A variable stores a reference, not a whole object.
- The declared type controls callable members.

02 ALIASING

- b = a copies a reference value.
- Both variables may identify one object.
- Mutate object vs rebind variable.

03 PASS-BY-VALUE

- Every parameter gets a copy.
- Reference copies can mutate shared objects.
- Rebinding a parameter cannot rebind caller.

04 IDENTITY & EQUALITY

- == checks same reference for objects.
- equals can define logical equality.
- Equal objects require equal hash codes.

05 MUTABILITY

- String is immutable.
- final reference prevents rebinding.
- Defensive copies protect mutable state.

06 MEMORY & GC

- Stack/heap diagrams are conceptual.
- GC eligibility follows reachability.
- Use explicit close for external resources.

EXAMPLE — SOURCE AND OUTPUT

CODE / RULE

```
Person a = new Person("Asha");
Person b = a;
b.setName("Rao");
System.out.println(a.getName());
```

EXPECTED RESULT

Rao

PART A / PRACTICE

Predict before running

9 thinking questions. Answer from the Java rule, not from memorized output.

A1 Two variables alias one Person. Which operation changes both views: b.setName(...) or b = new Person(...)?

A2 Explain why a primitive parameter increment cannot update the caller local.

A3 Can a callee replace a caller’s Person variable by reassigning its parameter?

A4 Two new Person objects have the same ID. Predict == and equals under ID equality.

A5 If equals returns true for a and b, what must hashCode guarantee?

A6 Does final Person p mean p cannot be mutated?

A7 After a = null but b still references the object, can it be collected?

A8 Why is returning an internal java.util.Date risky?

A9 Why is calling an overridable method from a constructor risky?

Reasoning and answers

Explain each rule in your own words before moving on.

A1 Mutation of the shared object through b changes what a sees; rebinding b does not change a.

A2 The parameter holds a copy of the primitive value. Return the new value and assign it in the caller.

A3 No. A parameter gets a copied reference value; local reassignment cannot rebind the caller slot.

A4 == is false for two distinct objects; ID-based equals is true.

A5 a.hashCode() must equal b.hashCode(); equal hash codes alone do not imply equality.

A6 No. final prevents reassignment of p; allowed methods may still mutate the Person.

A7 Not while b remains a live reachability path to the object.

A8 Date is mutable; clients could modify internal state through the returned reference.

A9 Runtime dispatch may reach subclass code before subclass initialization is complete.

CHAPTER 3 — PART B: CLASSES, PACKAGES & ENCAPSULATION

Classes • Objects • Fields • Methods • Constructors • this • static • final • Packages • Access Modifiers • Encapsulation • Composition

JAVA FUNDAMENTALS

WHY THIS MATTERS

- Classes help you model real-world objects and behaviour.
- Encapsulation and access control create safe, maintainable code.
- Packages organise code for larger applications.

MASTER THESE

- How to define classes with fields, methods and constructors.
- The difference between instance and static members.
- Using packages, imports and access modifiers correctly.
- Applying encapsulation and composition in your own classes.

KEY TAKEAWAY

- A class is a blueprint; objects are instances.
- Use access modifiers and encapsulation to protect state.
- Packages organise related classes and avoid name conflicts.
- Use composition (has-a) for code reuse.
- Understand static, final and initialization order.

1 Classes, Fields & Methods

A class is a blueprint for creating objects

- A class defines fields (state) and methods (behaviour).
- Objects are instances of a class.
- Fields store data about the object.
- Methods define what the object can do.

Class (Person) Blueprint → new Person() creates an object → Object (Instance) Person p

JAVA EXAMPLE • A simple Person class

```
public class Person
{
    String name;
    int age;

    void introduce()
    {
        System.out.println("Hi, I'm " + name
                           + " and I'm " + age);
    }
}
```

OUTPUT

```
Hi, I'm Alice and I'm 25
```

2 Constructors & this

Initialize objects and refer to the current object

- A constructor initializes fields when an object is created.
- Constructors can be overloaded.
- Constructorr `chaining` calls another constructor using `this(...)`.
- `this` refers to the current object and helps disambiguate fields from parameters.

JAVA EXAMPLE • Constructors, this and chaining

```
public class Person
{
    String name;
    int age;

    public Person(String name, int age)
    {
        this.name = name;
        this.age = age;
    }

    public Person(String name)
    {
        this(name, 0);    // constructor chaining
    }
}
```

OUTPUT

```
Alice (age 25)
Bob (age 0)
```

3 Instance vs static

Per-object vs class-wide members

- Instance fields/methods belong to each object (separate copy for each object).
- Static fields/methods are shared by all objects (belong to the class).
- A static method cannot directly access instance fields without an object.

JAVA EXAMPLE • A simple counter using static

```
public class Counter
{
    static int count = 0;     // class-level (shared)
    int id;                   // instance field

    public Counter()
    {
        count++;              // shared counter
        id = count;
    }

    public static int getCount()
    {
        return count;
    }
}
```

OUTPUT

```
c1 id = 1
c2 id = 2
Total count = 2
```

4 final and constants

Prevent changes and define constants

- `final` variables cannot be reassigned.
- `final` fields can only be set once (usually in a constructor).
- `static final` is used for constants (class-wide).
- A `final` reference means the reference cannot change, but the object it refers to may still be mutable.

JAVA EXAMPLE • final variables and constants

```
public class Example
{
    static final double PI = 3.14159;  // constant
    final String name;                 // final field

    public Example(String name)
    {
        this.name = name;              // set once
    }
}
```

OUTPUT

```
PI = 3.14159
Name = Math
```

NOTE
A `final` reference cannot be changed, but the object it refers to can still be modified (if mutable).

5 Packages & imports

Organise classes and reuse code

- Use the `package` statement to declare a package.
- Folder structure must match the package name.
- Use `import` to use classes from other packages.
- You can also use a `fully qualified name` (no import).
- `Static import` lets you import static members.
- Subpackages are separate packages.

```
com
└ example
  └ model
    Person.java
```

Package declaration

```
package com.example.model;
```

Using the class (in another file)

```
import com.example.model.Person;
// or fully qualified name
com.example.model.Person p;
```

JAVA EXAMPLE • Package and import

```
// File: com/example/model/Person.java
package com.example.model;
public class Person { }

// In another file
import com.example.model.Person;
class TestPackage
{
    public static void main(String[] args)
    {
        Person p = new Person();
        System.out.println("Package works!");
    }
}
```

OUTPUT

```
Package works!
```

6 Access modifiers

Control visibility of classes and members

- Access modifiers control which classes can use a class, field or method.
- Top-level classes can be `public` or `package-private` (no modifier).
- `protected` members are accessible in subclasses even in different packages.

Modifier	Same Class	Same Package	Subclass (other package)	Other Package
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
(package-private)	✓	✓	✗	✗
private	✓	✗	✗	✗

KEY POINTS

- Top-level classes can be `public` or package-private.
- `protected` is mainly useful through inheritance (across packages).
- Use the most restrictive access level that works.

7 Encapsulation & getters/setters

Hide internal state and control access

- Encapsulation keeps fields private and provides public methods to access and modify them.
- Use `getters` to read fields and `setters` to update fields with validation.
- Helps protect invariants and keeps objects in a valid state.

JAVA EXAMPLE • Encapsulation with validation

```
public class Account
{
    private String owner;
    private double balance;

    public Account(String owner)
    {
        this.owner = owner;
    }
    public double getBalance()
    {
        return balance;
    }
    public void deposit(double amount)
    {
        if (amount <= 0)
        {
            throw new IllegalArgumentException(
                "Amount must be positive");
        }
        balance += amount;
    }
```

OUTPUT

```
Balance: 100.0
```

8 Composition & initialization order

Has-a relationships and object creation order

- Composition models a `has-a` relationship (one object contains another).
- Prefer composition over inheritance for code reuse.
- Initialization order when creating an object:
  1. Static fields and static blocks (once, when class loads)
  2. Instance fields and instance blocks (in declaration order)
  3. Constructor (last)

Composition (has-a)

Car (has an Engine) → Engine (part of Car)

JAVA EXAMPLE • Composition and initialization order

```
class Engine
{
    Engine()
    {
        System.out.println("Engine created");
    }
}
class Car
{
    private Engine engine;  // has-a relationship
    Car()
    {
        engine = new Engine();
        System.out.println("Car created");
    }
}
```

OUTPUT

```
Engine created
Car created
```

COMMON MISTAKES TO AVOID

- Exposing mutable fields directly (breaks encapsulation).
- Confusing static and instance members.
- Forgetting package-private access (no modifier).
- Assuming subpackages have the same access (they don't).
- Misunderstanding final references vs immutable objects.
- Overusing public fields instead of getters/setters.

QUICK REFERENCE SUMMARY

- A class is a blueprint; objects are instances.
- Use constructors (and this) to initialize fields.
- Instance members belong to each object; static are shared.
- Use final for constants and to prevent reassignment.
- Organise code with packages and imports.
- Use access modifiers to control visibility.
- Keep fields private and use getters/setters.
- Prefer composition (has-a) over inheritance.
- Initialization order: static → instance → constructor.

PART B

CLASSES, PACKAGES ENCAPSULATION

YOUR LEARNING OBJECTIVES

- Classes expose behavior while protecting internal state.
- package and import resolve names; neither bypasses visibility.
- public, protected, package-private and private have distinct boundaries.
- Constructors, this, static and final control initialization and ownership.
- Composition makes dependencies explicit and keeps abstractions focused.

22 focused concept lessons • one infographic

Mini exercises and reasoned answers included

Organize Java source, enforce invariants and control cross-package access.

JAVA 17+ • Allman braces • Code font enlarged

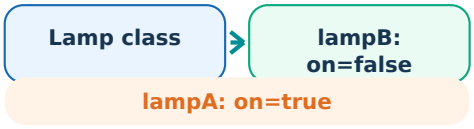
Anatomy of a class and its members

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A class groups instance state and operations. Fields describe what an object remembers; methods describe operations available through its type.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Lamp
{
    private boolean on;
    public void turnOn()
    {
        on = true;
    }
    public boolean isOn()
    {
        return on;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Class declaration creates the type Lamp; no Lamp instance exists yet.
2. When an instance is constructed, its on field starts false.
3. turnOn() changes that instance field; isOn() reads the same state.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Do not make fields public merely because the class is used widely. Methods let you preserve invariants while changing the representation later. For example, a balance operation should usually be deposit(amount) rather than setBalance(anything). Source readers should first identify the visible public API, then inspect private fields to understand how that API is implemented. A class declaration is a blueprint, not an object. Every new Lamp owns a distinct on field. Marking that field private means callers cannot set arbitrary values directly; they must use the supported methods. A method can read or change its receiver’s state even if other code cannot access that state.

COMMON TRAP

A class with only fields exposed publicly has very little encapsulation.

TRY THIS IN INTELLIJ

Create Lamp, call turnOn(), then print isOn().

REMEMBER turnOn() changes that instance field; isOn() reads the same state.

APPLY THE RULE / CHANGE ONE THING

Create two Lamps, turn on only the first, then call isOn() on both.

Answer: The first is true; the second remains false.

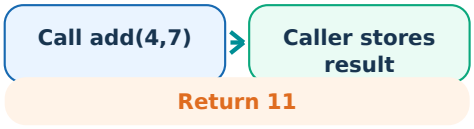
Methods, parameters, and return values

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A method declares a return type, name, parameters, and body. Calling it evaluates arguments, copies their values to parameters, and executes its statements.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static int add(int a, int b)
{
    return a + b;
}
int result = add(4, 7);
System.out.println(result);
```

EXPECTED OUTPUT

```
11
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The caller evaluates 4 and 7 before the call.
2. The parameters a and b receive copies of those int values.
3. return a + b evaluates to 11; the caller assigns it to result.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The parameter list is part of the method’s contract, and each argument expression is evaluated before the call. Returning does not implicitly alter caller locals; the caller must store the result if needed. A method may have side effects as well, which should be obvious from its name or documentation. Keep long methods focused on one coherent responsibility. The signature is a contract: return type, name, and ordered parameter types. The returned value goes to the call expression; it is not automatically displayed or retained. A void method produces no return value for assignment, whereas a method returning int may be used inside a larger expression.

COMMON TRAP

The method signature used for overloading excludes the return type; changing only the return type is not a valid overload.

TRY THIS IN INTELLIJ

Change return a+b to return a*b and predict the result.

REMEMBER return a + b evaluates to 11; the caller assigns it to result.

APPLY THE RULE / CHANGE ONE THING

Call add(4, 7) without storing or printing its result.

Answer: The method still returns 11, but nothing is printed.

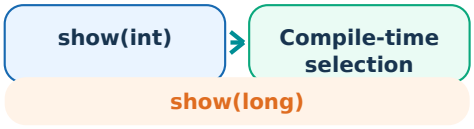
Overloading methods at compile time

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Methods can share a name if their parameter lists differ. Java chooses an applicable overload using compile-time types and conversion rules.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static String show(int n)
{
    return "int";
}
static String show(long n)
{
    return "long";
}
System.out.println(show(4));
System.out.println(show(4L));
```

EXPECTED OUTPUT

```
int
long
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The compiler sees the literal 4 has type int.
2. The int overload is the closest applicable method; 4L has type long.
3. The selected methods return the strings int and long.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Java overload resolution considers method applicability and specificity, not just textual order. A literal 4 has type int, while 4L is long. Overloads with reference types can become ambiguous when passed null, and combinations of widening, boxing and varargs can surprise you. Avoid APIs that require readers to memorize a complicated overload hierarchy. Overloading is not about the return type alone. The compiler considers arity, parameter types, and allowed conversions. If two candidates are equally specific, it can reject a call as ambiguous. A change to the variable's declared type can change which overload is selected, even when the runtime numeric value stays the same.

COMMON TRAP

Overloading does not mean runtime polymorphism; ordinary overload selection is compile-time behavior.

TRY THIS IN INTELLIJ

Add show(double n). Which call matches 4.0?

REMEMBER The selected methods return the strings int and long.

APPLY THE RULE / CHANGE ONE THING

Add show(double) and call show(4.0).

Answer: The double overload is selected; integer literals still select show(int).

The package declaration identifies a namespace

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A package groups related types and controls certain access boundaries. Its declaration normally appears first, before imports and type declarations.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
package com.example.model;
public final class Person
{
    private final String name;
    public Person(String name)
    {
        this.name = name;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The package statement establishes the name com.example.model.
2. The public class Person belongs to that namespace regardless of who imports it.
3. The source path convention mirrors the dotted package path.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The package name appears in the fully qualified type name, such as com.example.model.Person. Files in different packages may declare the same simple class name. A package usually maps to a directory structure under the source root, but the Java language's package membership comes from declarations. Modern projects may add Java modules, which impose additional export rules. A package name is not an object or a folder that Java traverses at runtime. It is part of a type's fully qualified name and also defines package-private access. Build tools conventionally map the package to source directories so the compiler, IDE, and readers can locate code consistently.

COMMON TRAP

A folder called model by itself does not declare package model. The package statement is authoritative.

TRY THIS IN INTELLIJ

Move the file to a different directory but keep the package name. How might IntelliJ react?

REMEMBER The source path convention mirrors the dotted package path.

APPLY THE RULE / CHANGE ONE THING

Move Person.java to another folder without changing its package line.

Answer: The declared package remains com.example.model; an ordinary Maven layout becomes inconsistent.

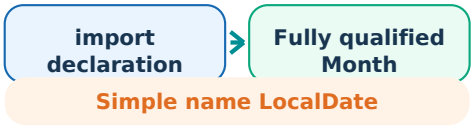
Imports and fully qualified names

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An import lets unqualified simple class names refer to types in other packages. It does not copy or load the class and does not grant access. java.lang types are automatically available by simple name.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
import java.time.LocalDate;
LocalDate date = LocalDate.of(2026, 10, 10);
java.time.Month month = date.getMonth();
System.out.println(month);
```

EXPECTED OUTPUT

```
OCTOBER
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The import gives LocalDate a usable simple name.
2. The of method creates the chosen date value.
3. Month is referenced by its full name and prints OCTOBER.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Import is a source-level convenience. Importing a class has no effect on what methods are visible or whether a constructor is public. You can use a fully qualified name directly and remove the import. Static imports bring accessible static members into simple-name scope; they are useful for limited cases such as assertions or mathematical constants. Import statements affect name resolution during compilation. They do not instantiate classes, grant access to private types, or import dependencies into a build. A fully qualified name always works when that type is accessible. java.lang and classes in your current package do not require ordinary imports.

COMMON TRAP

Imports do not bypass public/package-private restrictions; an inaccessible class remains inaccessible.

TRY THIS IN INTELLIJ

Replace LocalDate with java.time.LocalDate everywhere.

REMEMBER Month is referenced by its full name and prints OCTOBER.

APPLY THE RULE / CHANGE ONE THING

Remove the LocalDate import and write java.time.LocalDate everywhere.

Answer: The result is unchanged; only source-name resolution differs.

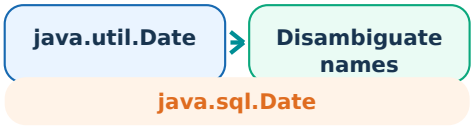
Wildcard imports and name collisions

Understand the rule • trace the code • read the diagram • test it

CORE RULE

import java.util.* makes eligible types in one package available by simple name; it does not recursively import subpackages. Importing both Date classes by their simple names would be ambiguous.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
import java.util.Date;
Date oldDate = new Date(0);
java.sql.Date sqlDate = new java.sql.Date(0);
System.out.println(oldDate.getTime());
System.out.println(sqlDate.getTime());
```

EXPECTED OUTPUT

```
0
0
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Import java.util.Date selects one Date name.
2. The SQL Date is explicitly qualified to avoid conflict.
3. Both objects represent the epoch start and return 0 milliseconds.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A wildcard import never means “all packages underneath this name.” When both java.sql.Date and java.util.Date are in scope, use a fully qualified name to make intent explicit. Ambiguous simple names are a compilation problem, not a runtime class-loading decision. Organized imports in IntelliJ do not change package boundaries. A wildcard import applies to accessible types in one named package only. It does not recursively include subpackages such as java.util.concurrent. Two on-demand imports that expose the same simple type name can force a fully qualified reference. Explicit naming is useful when legacy APIs define classes named Date, List, or Timer.

COMMON TRAP

java.util.* never imports java.util.concurrent.*; package hierarchies are naming conventions, not inheritance.

TRY THIS IN INTELLIJ

Try importing both java.util.Date and java.sql.Date and declaring Date x.

REMEMBER Both objects represent the epoch start and return 0 milliseconds.

APPLY THE RULE / CHANGE ONE THING

Replace the explicit Date import with both java.util.* and java.sql.*.

Answer: Unqualified Date becomes ambiguous; qualify or explicitly import the intended type.

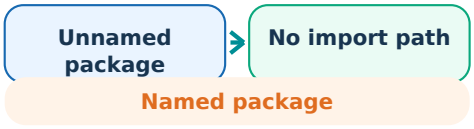
Why the default package causes trouble

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A class without a package declaration lives in the unnamed package. Named-package code cannot import types from that unnamed package.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
// Not recommended for a real project:
public class Scratch
{
    public static void main(String[] args)
    {
        System.out.println("scratch");
    }
}
```

EXPECTED OUTPUT

```
scratch
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Scratch has no package statement and enters the unnamed package.
2. Its main method prints scratch.
3. A class in a named package cannot import Scratch by simple name.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The unnamed package is convenient for a one-file toy program but interacts poorly with named packages and tooling. As soon as code contains multiple collaborating classes, choose a named package and align your source tree. Do not confuse a generated package folder in IntelliJ with the Java package declaration at the top of the file. Unnamed-package examples are acceptable for tiny scratch files, but they do not scale. Professional repositories use stable package names so tests, services, and libraries can reference types reliably. Simply moving a file into a folder does not create a package declaration automatically.

COMMON TRAP

Moving Scratch under a folder does not automatically make it a named-package class.

TRY THIS IN INTELLIJ

Give Scratch a package named com.example.app and align its folder.

REMEMBER A class in a named package cannot import Scratch by simple name.

APPLY THE RULE / CHANGE ONE THING

Put Scratch in the default package, then attempt import Scratch; from a named package.

Answer: The import is not legal; place Scratch in a named package.

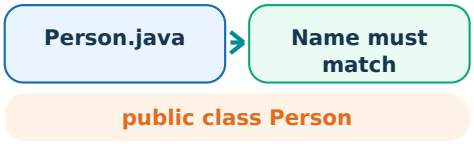
Public top-level classes and file names

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A public top-level Java class must normally be declared in a .java source file with the same simple name. A compilation unit may contain additional package-private top-level types, but only one public top-level type under the normal source-file conventions.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
// Person.java
package com.example.model;
public class Person
{
}
// Another separate file can declare
// package-private Helper in this package.
```

EXPECTED OUTPUT

```
No console output (compile-time rule).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Person is public and top-level.
2. The conventional source file is Person.java.
3. Other package-private top-level types may share the file, but only one public top-level type may be declared.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Multiple top-level package-private classes can appear in the same compilation unit, but one class per file is normally easier to maintain. The public class name must match the source filename used by ordinary Java compilation. A nested public class follows different naming rules because its enclosing top-level class owns the source file. The public type name and source file name must match case exactly in normal source compilation. A Java source file may contain multiple top-level declarations, but burying several major types together makes IDE navigation and maintenance harder. Public nested types follow a different rule because they are members of an enclosing type.

COMMON TRAP

Class and public filename matching is case-sensitive on Java source rules.

TRY THIS IN INTELLIJ

Name a file Human.java while it declares public class Person; observe the compiler diagnostic.

REMEMBER Other package-private top-level types may share the file, but only one public top-level type may be declared.

APPLY THE RULE / CHANGE ONE THING

Rename Person.java to People.java without renaming public class Person.

Answer: The compiler reports a public type/file-name mismatch.

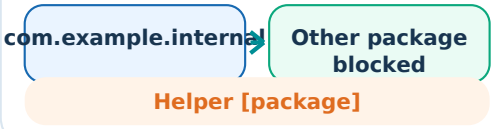
Package-private classes and helpers

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A top-level class without public is package-private. Other classes in the same package can use it, but different packages cannot.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
package com.example.internal;
class Helper
{
    String work()
    {
        return "internal";
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Helper lacks the public modifier.
2. Same-package classes can use Helper and its eligible members.
3. Code in unrelated packages cannot import this class as an accessible public API.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Package-private collaborators are useful for keeping an implementation behind a small public API. They can still be tested by tests located in the same package. However, package-private does not make a type secure from reflection or all possible runtime mechanisms. It is an access rule enforced by the language, not an encryption or sandbox boundary. Package-private is an intentional design boundary. It lets implementation classes collaborate without becoming part of a package's public contract. The restriction is checked by the compiler and language rules; it is not a security sandbox. Remember that com.example and com.example.internal are different packages.

COMMON TRAP

Importing com.example.internal.Helper from another package is not permitted just because its file exists.

TRY THIS IN INTELLIJ

Create a public InternalFacade that delegates to Helper.

REMEMBER Code in unrelated packages cannot import this class as an accessible public API.

APPLY THE RULE / CHANGE ONE THING

Create com.example.app.Main and try new Helper() from it.

Answer: Access is rejected; expose a public facade or change API design.

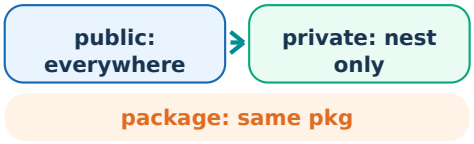
The four member access levels

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Access control limits where code can use classes and members. Public is broadly visible; private is within the enclosing top-level nest; package-private is same package. protected adds an inheritance-based path for code in other packages, with receiver restrictions.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
public class Vault
{
    public int id;
    protected int level;
    int internalCode; // package-private
    private String password;
}
```

EXPECTED OUTPUT

```
No console output (declaration).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Each member uses a different modifier.
2. Visibility depends on caller package and inheritance context.
3. private is restricted to enclosing nest; public is broadly visible.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The access matrix is easy to memorize incorrectly. public does not bypass module exports or all runtime access controls. private applies inside the enclosing nest, including related nested types under modern Java nest-based access. package-private access depends on the actual runtime package identity, including loader/module subtleties beyond this foundation. There are only two kinds of access for top-level classes: public and package-private. All four modifiers apply to ordinary class members. protected is the subtle one: it includes same-package access plus a restricted path for subclasses outside the package. An import never changes this matrix.

COMMON TRAP

The default access level is not protected. An omitted modifier means package-private.

TRY THIS IN INTELLIJ

Draw the same-class/same-package/subclass/unrelated access matrix.

REMEMBER private is restricted to enclosing nest; public is broadly visible.

APPLY THE RULE / CHANGE ONE THING

Try to read password and internalCode from a class in a different package.

Answer: Both are inaccessible (unless the code is in a permitted enclosing nest for private).

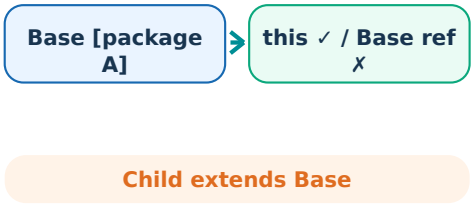
protected across package boundaries

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A subclass in a different package can use an inherited protected member through an appropriate subclass reference (such as this), not any base instance. Cross-package protected access is deliberately narrower than “subclasses can access protected fields on any object.” The companion project contains AccessBase in access.base and AccessChild in access.child so you can test the compiler boundary.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
// In package child:
class Child extends Base
{
    String show()
    {
        return this.protectedValue;
    }
    // new Base().protectedValue is not allowed.
}
```

EXPECTED OUTPUT

```
No console output (access example).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. A child class extends the base from another package.
2. Within the subclass, this.protectedValue is inherited and permitted.
3. new Base().protectedValue is not permitted in that cross-package context.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Across packages, protected access requires a subclass context and an appropriate receiver of that subclass type (or its subtype). Inside the same package, the inheritance receiver limitation does not apply in the same way. Verify protected behavior by compiling the two-package sample, rather than relying on a one-line yes/no table. In a different package, protected access is not unrestricted access to all superclass instances. From instance code of a subclass S, an appropriate qualifying expression must have type S or a subtype of S. This prevents a subclass from reaching protected members on arbitrary base-class objects created elsewhere.

COMMON TRAP

Do not infer protected access from subclass status alone; inspect the receiver expression type.

TRY THIS IN INTELLIJ

Try accessing protectedValue through a new AccessBase() inside AccessChild.

REMEMBER new Base().protectedValue is not permitted in that cross-package context.

APPLY THE RULE / CHANGE ONE THING

In Child, declare Base other = new Base(); and access other.protectedValue.

Answer: The access fails; use an appropriate Child receiver or accessible base API.

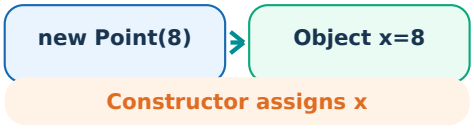
Constructors establish initial object state

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A constructor initializes a new object. Its name matches the class, and it has no return type, not even void.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Point
{
    private final int x;
    Point(int x)
    {
        this.x = x;
    }
    int getX()
    {
        return x;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new Point(8) allocates a Point instance.
2. The constructor receives x=8 and assigns this.x once.
3. The getter would later return that object’s field value, 8.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A constructor cannot declare a return type. When a class declares any constructor, the compiler does not synthesize another no-argument constructor. Constructors can delegate to each other, validate inputs and establish final fields. Avoid performing overridable calls, starting threads, or leaking references before initialization is complete. Constructors establish invariants as objects become usable. Before their bodies execute, fields have default values; field initializers and superclass initialization also participate. A constructor has no declared return type. If you write void Point(), you have created an ordinary method, not a constructor.

COMMON TRAP

A method named Point with a void return type is not a constructor.

TRY THIS IN INTELLIJ

Try new Point() and explain the compile error.

REMEMBER The getter would later return that object’s field value, 8.

APPLY THE RULE / CHANGE ONE THING

Add an argument check that rejects x below zero.

Answer: The constructor can throw before a reference to a valid Point is returned.

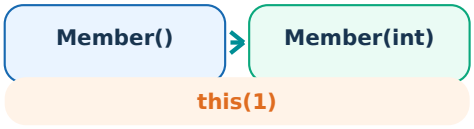
Constructor overloading and chaining with this

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Multiple constructors can offer alternative ways to create a valid object. this(...) calls another constructor of the same class. Explicit constructor invocation with this(...) must be first in a constructor body in Java 17, and chaining cannot be cyclic.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Member
{
    private final int level;
    Member()
    {
        this(1);
    }
    Member(int level)
    {
        this.level = level;
    }
}
```

EXPECTED OUTPUT

```
No console output (constructor definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Member() is selected for the no-argument call.
2. Its first action delegates to Member(1) via this(1).
3. The delegated constructor assigns level exactly once.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Constructor chaining is most useful when one constructor is the single validation path. In Java 17, this(...) or super(...) must be the first constructor statement, and you cannot form cycles. If a child superclass has no accessible no-argument constructor, the child must explicitly select an appropriate super(...). This is a frequent error when changing a base class API. Constructor chaining centralizes validation and assignments, reducing inconsistencies between overloads. For Java 17 source, an explicit this(...) or super(...) constructor invocation must be the first statement; it is not an ordinary method call and cannot recur in a cycle. A constructor eventually invokes a superclass constructor.

COMMON TRAP

Do not write both this(...) and super(...) in one constructor; one explicit constructor invocation occupies that first position.

TRY THIS IN INTELLIJ

Add Member(int level, String name) and chain to Member(int).

REMEMBER The delegated constructor assigns level exactly once.

APPLY THE RULE / CHANGE ONE THING

Change Member() to call this(3) instead.

Answer: New Member() then stores level=3.

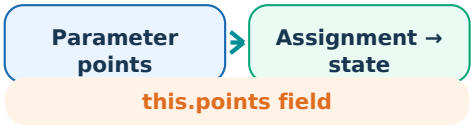
this: current receiver and shadowed names

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Inside an instance method or constructor, this refers to the current receiver object. It distinguishes fields from same-named parameters.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Score
{
    private int points;
    void setPoints(int points)
    {
        this.points = points;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. setPoints receives the parameter named points.
2. this.points refers to the field on the current Score object.
3. The parameter value is copied into the field.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

this can select fields, invoke another constructor with this(...), or be used as the receiver for an instance method. A parameter with the same name as a field shadows it; this.field restores clarity. There is no this inside a static context because no instance receiver exists there. Keep this escape out of constructors when practical. The receiver is the object whose instance method is being executed. this is not available in a static method because static methods do not have an implicit receiver. Writing this.field is especially useful when fields and parameters have the same name; it can also improve readability when several objects of the same type interact.

COMMON TRAP

this is unavailable in a static method because static methods have no receiver object.

TRY THIS IN INTELLIJ

Remove this. Predict whether the object's points field changes.

REMEMBER The parameter value is copied into the field.

APPLY THE RULE / CHANGE ONE THING

Remove this. and leave points = points;

Answer: Only the parameter assigns to itself; the field does not change.

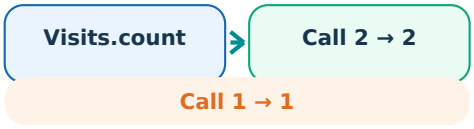
static state belongs to the class

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A static field has one shared value for a class, not a separate field for every instance. Static methods can be called without an instance.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Visits
{
    private static int count;
    static int next()
    {
        return ++count;
    }
}
System.out.println(Visits.next());
System.out.println(Visits.next());
```

EXPECTED OUTPUT

```
1
2
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Visits.count belongs to the class, not individual objects.
2. The first next() increments the shared count from 0 to 1.
3. The second call increments that same field to 2.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A static field is shared class-wide and may survive across many object creations. This is useful for constants, metadata and carefully designed registries. Mutable static state complicates deterministic tests and concurrent execution. Prefer dependencies passed explicitly rather than a globally changeable service reachable through a static field. Static members are associated with the class and are usually accessed through the class name. Shared mutable static state needs special care when multiple threads are present: ++count is not an atomic multi-threaded increment. Constants and stateless utility methods are safer typical uses of static.

COMMON TRAP

A mutable static counter is shared state and requires synchronization if multiple threads update it.

TRY THIS IN INTELLIJ

How do two separate Visits objects affect the one shared count?

REMEMBER The second call increments that same field to 2.

APPLY THE RULE / CHANGE ONE THING

Create two Visits instances and call next() through either one.

Answer: The single static count keeps increasing; instance creation does not reset it.

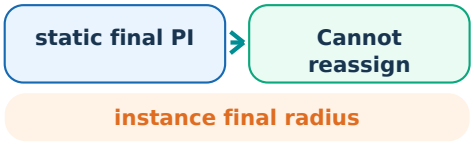
final fields, constants, and immutability

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A blank final instance field must be definitely assigned during object initialization. A static final constant belongs to the class. final references cannot be rebound, but a final field containing a mutable object does not automatically make that object immutable.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Circle
{
    static final double PI = 3.14159;
    private final double radius;
    Circle(double radius)
    {
        this.radius = radius;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The class-level PI constant is initialized once.
2. Each Circle instance receives its own final radius.
3. Each constructor must assign that blank final field before completion.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A final field must be definitely assigned once along all constructor paths. A blank final field may be assigned in an initializer or constructor as permitted. Static final compile-time constants may be inlined into clients, so changing a public constant without rebuilding dependent code can surprise you. Final fields support, but do not guarantee, a wholly immutable object design. final prevents reassignment of a field after definite initialization. It does not mean an object graph is deeply immutable; a final List field can still refer to a mutable list. Truly immutable classes also avoid leaking mutable references and are designed so their valid state is fixed after construction.

COMMON TRAP

Avoid public static final mutable objects as globally editable pseudo-constants.

TRY THIS IN INTELLIJ

What changes if radius is initialized where it is declared rather than in the constructor?

REMEMBER Each constructor must assign that blank final field before completion.

APPLY THE RULE / CHANGE ONE THING

Change radius to final List<String> names; then add to the list.

Answer: The reference cannot be reassigned, but the list itself can still be mutated.

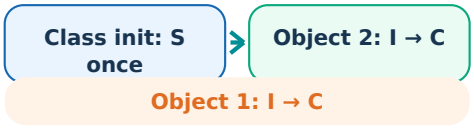
Initialization order: static, instance, constructor

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Static initialization normally occurs once when the class is initialized. Creating each object executes instance initialization before the constructor body.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Order
{
    static
    {
        System.out.print("S");
    }
    {
        System.out.print("I");
    }
    Order()
    {
        System.out.print("C");
    }
}
new Order();
new Order();
```

EXPECTED OUTPUT

```
SICIC
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The static initializer prints S once when Order initializes.
2. Each new Order executes the instance initializer, printing I.
3. Each constructor then prints C; two instances yield SICIC.

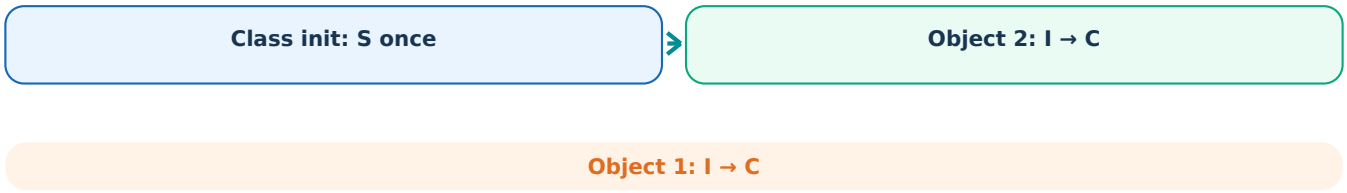
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Initialization order: static, instance, constructor

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

The static initializer prints S once when Order initializes. Each new Order executes the instance initializer, printing I. Each constructor then prints C; two instances yield SICIC.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Initialization order is easier to reason about in phases: class initialization, object allocation/defaults, superclass initialization, then subclass field initializers and constructor logic. Inheritance adds another layer: subclass overridable methods can see default values before their own initialization. When behavior depends on order, prefer simple constructors and explicit dependency creation. Initialization includes default field values, explicit initializers, superclass construction, and constructor bodies. The exact sequence gets more complex under inheritance; first master the once-per-class versus once-per-object distinction. A static initializer normally runs at first active use, not necessarily when a class file is first loaded.

COMMON TRAP

Do not confuse class loading, linking, and initialization; the simple mental model is not the whole JVM specification.

TRY THIS IN INTELLIJ

Predict output if only one Order object is constructed.

REMEMBER Each constructor then prints C; two instances yield SICIC.

APPLY THE RULE / CHANGE ONE THING

Create a third Order.

Answer: The third instance adds IC, producing SICICIC.

Encapsulation means preserving invariants

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Encapsulation is not just adding getters and setters. The class must prevent invalid states and expose operations with meaningful guarantees.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
public void setAge(int age)
{
    if (age < 0 || age > 130)
    {
        throw new IllegalArgumentException("age");
    }
    this.age = age;
}
```

EXPECTED OUTPUT

```
No console output for valid setter calls.
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The setter receives an incoming age.
2. Invalid values cause an exception before state changes.
3. Only acceptable ages reach the assignment to this.age.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

An invariant is a property that should be true whenever a constructed object is available for normal use. For Person, age cannot be negative and name cannot be blank. Validate both the constructor path and any state-changing methods; otherwise a setter may undo constructor validation. For some concepts, no public setter is the clearest design. Encapsulation protects meaningful invariants, not simply field visibility. A public setter for every private field may still permit inconsistent state changes across multiple fields. Prefer operations expressing domain intent (deposit, approve, close) when a single unconstrained setter would permit invalid transitions.

COMMON TRAP

A setter that accepts every value can defeat the very invariants encapsulation should protect.

TRY THIS IN INTELLIJ

Decide whether id should have a public setter.

REMEMBER Only acceptable ages reach the assignment to this.age.

APPLY THE RULE / CHANGE ONE THING

Call setAge(-1) after an object already has age 30.

Answer: An exception is thrown and the earlier valid age remains unchanged.

Defensive copying when returning mutable state

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A getter that returns a direct reference to mutable internal state may allow callers to change that state without validation. A Date copy prevents the caller from mutating the stored Date through the getter.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
private Date created = new Date();
public Date getCreated()
{
    return new Date(created.getTime());
}
```

EXPECTED OUTPUT

```
No console output (getter definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The getter reads the internal Date timestamp.
2. It constructs and returns a distinct Date object.
3. The caller can mutate the returned Date without changing the original field.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A getter returning a mutable Date reference exposes an internal escape route. The caller may change it without calling a validated setter. Defensive copying is useful for Date, arrays and mutable collections, while immutable LocalDate and immutable Address can often be returned directly. The right choice depends on whether the field type itself can change after creation. Defensive copying is usually needed in both directions: copy mutable constructor inputs, and copy mutable outputs. A shallow copy can still share nested mutable components. New Java code should often prefer immutable java.time types, reducing defensive-copy complexity.

COMMON TRAP

A getter need not expose a field directly at all; exposing mutable implementation types couples clients to internals.

TRY THIS IN INTELLIJ

Change return to return created; explain the aliasing leak.

REMEMBER The caller can mutate the returned Date without changing the original field.

APPLY THE RULE / CHANGE ONE THING

Replace return new Date(created.getTime()) with return created.

Answer: A caller could then mutate internal state through the returned reference.

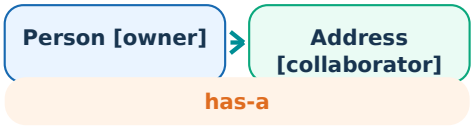
Composition: has-a relationships

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Composition means one object holds references to collaborators or value objects and delegates work to them. It is distinct from inheritance.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Person
{
    private final Address address;
    Person(Address address)
    {
        this.address = address;
    }
    String city()
    {
        return address.getCity();
    }
}
```

EXPECTED OUTPUT

```
No console output (relationship definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The Person constructor receives an Address reference.
2. The Person stores that collaborator as an instance field.
3. city() delegates to address.getCity() rather than inheriting Address.

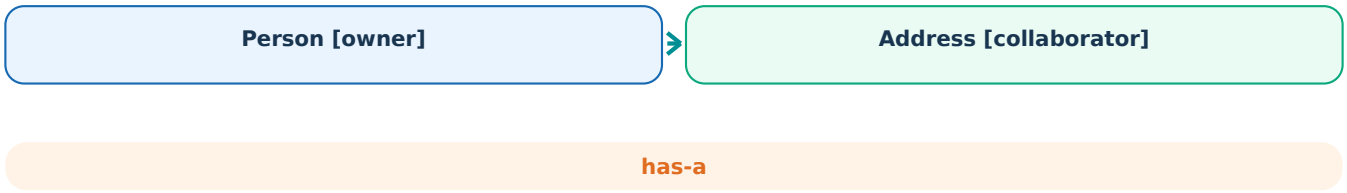
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Composition: has-a relationships

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

The Person constructor receives an Address reference. The Person stores that collaborator as an instance field. city() delegates to address.getCity() rather than inheriting Address.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

“Has-a” by itself does not define a lifetime relationship. Composition, aggregation and dependency are design-level distinctions that matter when deciding who creates, owns and replaces an object. Constructor injection makes required collaborators explicit; a method parameter may be better for a one-time helper. Use inheritance only when substitutability, not convenience, justifies the subtype. Composition expresses has-a relationships and enables swapping collaborations at construction time. A final Address field does not necessarily make the Address itself immutable; ownership still matters. Composition commonly creates smaller classes with clearer responsibilities and is the default choice when no genuine substitutable is-a relationship exists.

COMMON TRAP

Not every field relationship implies strict lifecycle ownership; “has-a” is a broad design shorthand.

TRY THIS IN INTELLIJ

Replace Address with a service interface and explain how it supports swapping implementations.

REMEMBER city() delegates to address.getCity() rather than inheriting Address.

APPLY THE RULE / CHANGE ONE THING

Supply a different Address implementation or object at construction.

Answer: The Person can reuse the same city() behavior without inheritance.

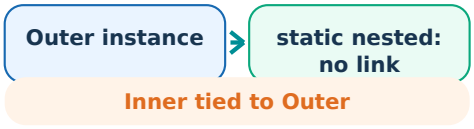
Nested, static nested, and inner classes

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Java supports types declared inside other types. A static nested class has no implicit outer instance; a non-static inner class is tied to an outer instance.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Outer
{
    private int number = 5;
    class Inner
    {
        int read()
        {
            return number;
        }
    }
}
```

EXPECTED OUTPUT

```
No console output (nested class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Outer has a per-instance number field.
2. Its non-static Inner type can read Outer.this.number implicitly.
3. A static nested class would not have that implicit outer object.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A static nested class is often a good fit for a helper that logically belongs to an outer class but needs no outer instance. A non-static inner class stores a relationship to an enclosing instance and can access its members. Nested types have their own access rules; this becomes important when reading builder classes and internal implementation helpers. A non-static inner instance carries an association to an enclosing instance, which can keep that outer object reachable. A static nested class is often preferable when the nested type does not need instance data. A local class is declared inside a method; an anonymous class is an expression creating a one-off subtype/implementation.

COMMON TRAP

Static nested classes are not “top-level static classes”; they remain members of their enclosing type.

TRY THIS IN INTELLIJ

Convert Inner to static and observe why direct access to number no longer works.

REMEMBER A static nested class would not have that implicit outer object.

APPLY THE RULE / CHANGE ONE THING

Make Inner static without changing read().

Answer: The reference to Outer’s instance field then fails until an Outer is provided.

Read the companion project’s package graph

Understand the rule • trace the code • read the diagram • test it

CORE RULE

The IntelliJ project demonstrates a conventional app/model/service/internal/access hierarchy. Package names express namespaces; dependencies express actual relationships.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
import com.refresher.ch3.model.Person;
import com.refresher.ch3.service.PersonService;
PersonService service = new PersonService(
    new FriendlyGreetingService());
System.out.println(service.welcome(person));
```

EXPECTED OUTPUT

```
Hello, Asha Rao! (with project demo values)
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The app package contains the entry point and assembly code.
2. The model package contains state-holding types.
3. The service and implementation packages define contracts and collaborators.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Read a multi-package project from the app entry point outward: identify imported public interfaces and classes, follow constructor arguments, then trace calls to implementations. Use Ctrl+B in IntelliJ to jump to declarations and implementations. Remember that folder names, imports, package declarations and visibility are four different concepts, even when they appear together in the source tree. Package graphs help reveal dependency direction. When you open an unfamiliar repository, navigate from the entry point into services and then their dependencies, rather than reading every class sequentially. A package tree is an organization convention, not an automatic dependency or security policy.

COMMON TRAP

Parent and child package names do not imply greater visibility or inheritance between packages.

TRY THIS IN INTELLIJ

Open PersonService.java, navigate to GreetingService, then follow the implementation.

REMEMBER The service and implementation packages define contracts and collaborators.

APPLY THE RULE / CHANGE ONE THING

Trace MainApp → PersonService → GreetingService implementation.

Answer: You see constructor injection of an interface-based collaborator.

CHAPTER 3 · PART B · VISUAL MASTER MAP

Core rules • mental models • common pitfalls • executable examples

01 CLASS STRUCTURE

- Fields hold state; methods define behavior.
- Constructors establish valid objects.
- Keep data private, expose small APIs.

02 PACKAGES & IMPORTS

- package identifies a type namespace.
- Imports shorten type names only.
- Subpackages are separate packages.

03 ACCESS CONTROL

- public: any accessible caller.
- private: enclosing nest.
- package-private: same package only.



04 PROTECTED RULE

- Same package may access it.
- Other-package subclasses have receiver restrictions.
- Avoid exposing fields unnecessarily.

05 INITIALIZATION

- this distinguishes current receiver.
- static is class-wide state.
- final fields must be assigned once.

06 COMPOSITION

- Has-a collaborators via fields.
- Validate invariants in constructors/methods.
- Prefer injection of abstractions.

EXAMPLE — SOURCE AND OUTPUT

CODE / RULE

```
package com.example.model;
public final class Person
{
    private final int id;
}
```

EXPECTED RESULT

```
No console output: type definition
```

PART B / PRACTICE

Predict before running

9 thinking questions. Answer from the Java rule, not from memorized output.

B1 Does import allow access to a package-private class in another package?

B2 Does package com.a.child count as the same package as com.a?

B3 What is the difference between protected and package-private?

B4 Can an unrelated class in a different package read protected?

B5 Why does this.field distinguish a parameter from a field?

B6 What happens to an implicit no-arg constructor after declaring one constructor?

B7 What restrictions apply to this(...) inside a Java 17 constructor?

B8 What does static say about a field?

B9 What makes a getter unsafe for mutable state?

PART B / ANSWER KEY

Reasoning and answers

Explain each rule in your own words before moving on.

B1 No. Import resolution does not change type accessibility.

B2 No. Package names do not confer hierarchical access.

B3 Package-private allows only same-package access; protected also offers restricted subclass access across packages.

B4 No, not through ordinary access merely because the member exists.

B5 this identifies the current receiver object and selects its field.

B6 The compiler stops providing a default no-arg constructor automatically.

B7 It must appear first and chaining may not be cyclic.

B8 There is one class-level field rather than one field per instance.

B9 Returning a direct mutable internal reference lets callers bypass validation or invariants.

CHAPTER 3 — PART C: INHERITANCE, INTERFACES & POLYMORPHISM

Inheritance • Interfaces • Abstract Classes • Overriding • Overloading • Upcasting • Downcasting • Dynamic Dispatch • instanceof • Composition

JAVA FUNDAMENTALS

WHY THIS MATTERS

- Helps you read and understand real-world frameworks like Spring and other enterprise code.
- Interfaces and polymorphism are widely used in modern, maintainable application design.
- Essential for writing flexible, extensible and testable code.

MASTER THESE

- How inheritance creates hierarchies and reuses behaviour.
- How to implement and work with interfaces and abstract classes.
- How runtime dispatch enables polymorphism.
- How to perform safe type casting using instanceof.
- When to prefer composition over inheritance.

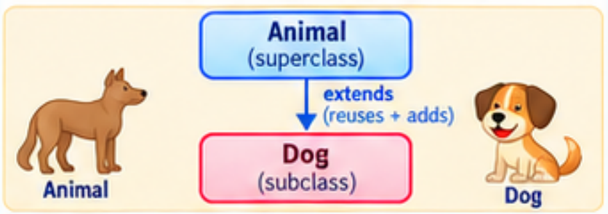
KEY TAKEAWAY

- The declared type of a reference controls which members are accessible at compile time.
- The actual object type controls which overridden method is executed at runtime.
- Prefer interfaces and composition when suitable for flexibility.
- Use inheritance only for a clear "is-a" relationship.

1 Inheritance Basics

A subclass reuses and extends a superclass

- Inheritance lets a class acquire fields and methods from another class using `extends`.
- The subclass can add new fields and behavior.
- Promotes code reuse and a natural hierarchy.



JAVA EXAMPLE — Basic inheritance

```
class Animal
{
    void speak()
    {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog barks");
    }
}
```

OUTPUT
```
Animal makes a sound
Dog barks
```

2 Method Overriding

Subclass provides a new implementation

- A subclass can provide a new implementation of a method with the same signature.
- Use `@Override` to indicate overriding.
- The method to execute is chosen at runtime based on the actual object type (dynamic dispatch).
- Static, private and final methods are not overridden in the same way.

JAVA EXAMPLE — Overriding and runtime dispatch

```
class Animal
{
    void speak()
    {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal
{
    @Override
    void speak()
    {
        System.out.println("Dog says Woof!");
    }
}
public class Test
{
    public static void main(String[] args)
    {
        Animal a = new Dog();    // upcasting
        a.speak();               // calls overridden method
    }
}
```

OUTPUT
```
Dog says Woof!
```

3 Overloading vs Overriding

Different signatures vs same signature

- **Method overloading:** same name, different parameters. Chosen at compile time.
- **Method overriding:** same signature in subclass. Chosen at runtime.

Feature	Overloading	Overriding
Parameter list	Different	Same
Where it happens	Same class	Subclass
When chosen	Compile time	Runtime (dynamic dispatch)
Purpose	Convenience	Provide new behavior
Return type	Can be different	Must be same (or covariant)

JAVA EXAMPLE — Overloading and overriding

```
class Printer
{
    void print(int n)
    {
        System.out.println("Number: " + n);
    }
    void print(String s)
    {
        System.out.println("Text: " + s);
    }
}
class SpecialPrinter extends Printer
{
    @Override
    void print(String s)
    {
        System.out.println("Special: " + s);
    }
}
```

OUTPUT
```
Number: 10
Text: Hello
```

4 Abstract Classes & Interfaces

Abstract class can have state; interface defines a contract

- Abstract class can have fields, constructors and method implementations.
- **Interface** defines a contract of methods (implicitly public and abstract).
- Both can be used to achieve polymorphism.

Abstract Class	Interface
Can have state (fields)	Defines a contract
Can have constructors	Methods are abstract (by default)
Can have concrete methods	No instance state
Can have abstract methods	Implemented using `implements`

JAVA EXAMPLE — Interface implementation

```
interface PaymentService
{
    void pay(double amount);
}
class CardPayment implements PaymentService
{
    public void pay(double amount)
    {
        System.out.println("Paid by card: " + amount);
    }
}
class UpiPayment implements PaymentService
{
    public void pay(double amount)
    {
        System.out.println("Paid by UPI: " + amount);
    }
}
```

OUTPUT
```
Paid by card: 500.0
```

5 Upcasting, Downcasting & instanceof

Convert between types safely

- Upcasting (subclass to superclass) is automatic.
- Downcasting (superclass to subclass) needs an explicit cast.
- Use `instanceof` to check the actual type before downcasting.
- An invalid downcast throws `ClassCastException`.



JAVA EXAMPLE — Casting and instanceof

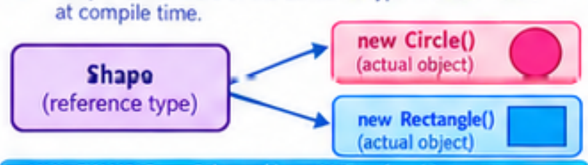
```
class Animal { }
class Dog extends Animal
{
    void run() { System.out.println("Dog runs"); }
public class Test
{
    public static void main(String[] args)
    {
        Animal pet = new Dog();          // upcasting
        if (pet instanceof Dog)          // safe check
        {
            Dog d = (Dog) pet;           // downcasting
            d.run();
        }
```

OUTPUT
```
Dog runs
```

6 Dynamic Dispatch & Polymorphism

Actual object type decides method execution

- A reference of a superclass or interface type can point to any subclass implementation.
- The overridden method of the actual object type is invoked at runtime.
- Only the members of the declared type are accessible at compile time.



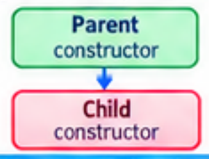
JAVA EXAMPLE — Polymorphism with interface/abstract type

```
abstract class Shape
{
    abstract void draw();
}
class Circle extends Shape
{
    void draw()
    {
        System.out.println("Drawing circle");
    }
}
public class Test
{
    public static void main(String[] args)
    {
        Shape s = new Circle();    // reference type is Shape
        s.draw();                  // runtime dispatch
```

7 super, Constructors & Initialization Chain

Parent is initialized before child

- A subclass constructor calls `super(...)` to initialize the superclass part.
- The superclass constructor runs before the subclass constructor.
- Ensures the object is properly initialized from top to bottom.



JAVA EXAMPLE — Constructor chain using super

```
class Parent
{
    Parent()
    {
        System.out.println("Parent constructor");
    }
}
class Child extends Parent
{
    Child()
    {
        super();
        System.out.println("Child constructor");
    }
}
```

OUTPUT
```
Parent constructor
Child constructor
```

8 Composition vs Inheritance

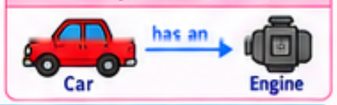
Use has-a for reuse and flexibility

- Inheritance (is-a) models a type relationship.
- Composition (has-a) builds behaviour by using other objects.
- Prefer composition for flexibility and loose coupling.
- Both approaches can be used together when appropriate.

Inheritance (is-a)



Composition (has-a)



JAVA EXAMPLE — Composition (has-a)

```
class Engine
{
    void start()
    {
        System.out.println
            "Engine started");
    }
}
public class Test
{
    public static void main(String[] args)
    {
        Car car = new Car();    car.start();
```

```
class Car
{   private Engine engine;  // has-a relationship
    Car()
    {
        this.engine = new Engine();
    }
    void start()
    {
        engine.start();     // delegate to engine
    }
```

OUTPUT
```
Engine started
```

COMMON MISTAKES TO AVOID

- Forgetting to use `@Override` when overriding methods.
- Performing unsafe downcasts without checking `instanceof`.
- Confusing method overloading with method overriding.
- Overusing inheritance when composition is a better fit.
- Expecting a parent reference to access subclass-only methods.
- Assuming constructors run in the opposite order.

QUICK REFERENCE SUMMARY

- **Inheritance:** subclass reuses and extends a superclass using `extends`.
- **Overriding:** same signature, chosen at runtime (dynamic dispatch).
- **Overloading:** same name, different parameters, chosen at compile time.
- **Interfaces:** define a contract; use `implements`.
- **Casting:** upcasting is automatic; downcasting needs a cast and may fail.
- **instanceof:** checks the actual object type before casting.
- **Polymorphism:** same method call, different behavior at runtime.
- **Constructor chain:** superclass constructor runs before subclass.
- **Composition:** use has-a for flexible and reusable design.

PART C

INHERITANCE, INTERFACES POLYMORPHISM

YOUR LEARNING OBJECTIVES

- Inheritance models a substitutable is-a relationship.
- Overload resolution uses compile-time types; overrides dispatch at runtime.
- Interfaces separate contracts from concrete implementations.
- Casting changes the view of a reference, never the object.
- Composition and constructor injection prepare you for Spring Boot.

22 focused concept lessons • one infographic

Mini exercises and reasoned answers included

Understand which calls compile and which implementation actually runs.

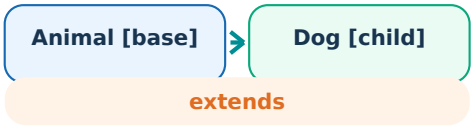
Inheritance models an is-a relationship

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A subclass extends one class and inherits accessible members while adding or refining behavior. The relationship should be substitutable, not merely code reuse.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Animal
{
    String speak()
    {
        return "sound";
    }
}
class Dog extends Animal
{
    @Override
    String speak()
    {
        return "woof";
    }
}
```

EXPECTED OUTPUT

```
No console output (class declarations).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Child extends Base and becomes a subtype of Base.
2. Child inherits accessible Base behavior and may define more behavior.
3. Code expecting Base may accept Child when the is-a relationship is valid.

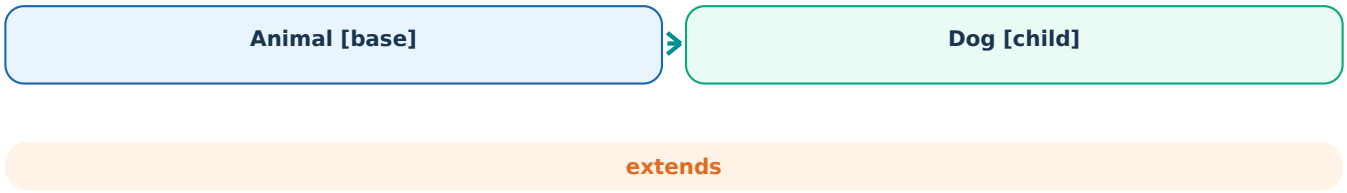
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Inheritance models an is-a relationship

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

Child extends Base and becomes a subtype of Base. Child inherits accessible Base behavior and may define more behavior. Code expecting Base may accept Child when the is-a relationship is valid.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Inheritance transmits an accessible interface and behavior, but it creates coupling: a change in the base class can affect all children. The Liskov substitution idea is more important than sharing code: any promised behavior of Animal must still be meaningful for Dog. Prefer a small stable base abstraction over a deep hierarchy with many special-case overrides. Inheritance is strongest when clients can substitute the child wherever the parent is expected without surprising behavior. A subclass does not inherit private member access as callable methods/fields, though the superclass portion of the object still exists. Java permits one direct class superclass, but a class can implement multiple interfaces.

COMMON TRAP

If the subtype cannot preserve the promised behavior of the parent, inheritance may be the wrong design.

TRY THIS IN INTELLIJ

Would Dog extends String compile? Explain final classes.

REMEMBER Code expecting Base may accept Child when the is-a relationship is valid.

APPLY THE RULE / CHANGE ONE THING

Ask whether a Dog is an Animal and whether a Dog is a Collar.

Answer: Dog is an Animal; Dog has a Collar. The second relationship favors composition.

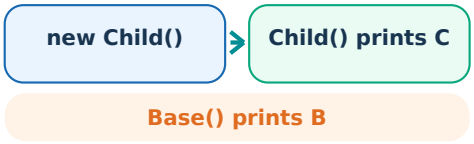
super(...) and constructor execution

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A subclass constructor must initialize its superclass portion before executing its own body. super(...) selects a superclass constructor. Java inserts a no-argument super() when the constructor has no explicit this(...) or super(...) and the superclass supports it.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Base
{
    Base()
    {
        System.out.print("B");
    }
}
class Child extends Base
{
    Child()
    {
        super();
        System.out.print("C");
    }
}
new Child();
```

EXPECTED OUTPUT

```
BC
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new Child() allocates one Child object.
2. The Child constructor invokes Base() first.
3. Base prints B, then Child prints C, giving BC.

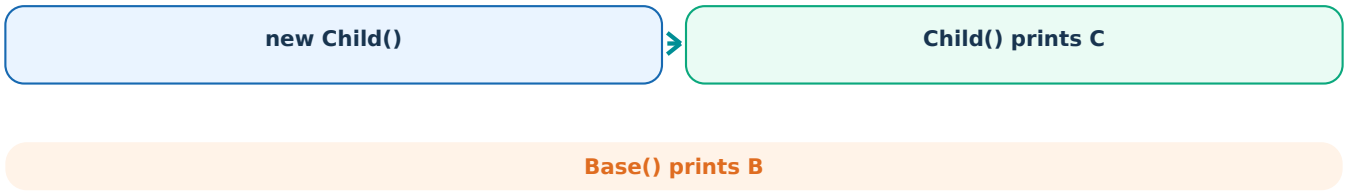
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

super(...) and constructor execution

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

new Child() allocates one Child object. The Child constructor invokes Base() first. Base prints B, then Child prints C, giving BC.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The order starts in Object and works outward through superclasses. The compiler inserts a super() only when appropriate, so a base constructor change can break subclasses even if they never directly use the changed field. Fields are initialized in their own phase, not all at once. Keep constructor logic simple and avoid calling overridable methods from a base constructor. Superclass constructors are invoked in a chain up to Object before subclass constructor bodies run. If no explicit super(...) or this(...) appears, the compiler inserts an implicit no-argument super() where permitted. If the parent has no accessible no-argument constructor, the subclass must select another parent constructor explicitly.

COMMON TRAP

If Base has only Base(int), Child must explicitly invoke an applicable constructor.

TRY THIS IN INTELLIJ

Add a print in an instance initializer and predict the order.

REMEMBER Base prints B, then Child prints C, giving BC.

APPLY THE RULE / CHANGE ONE THING

Remove super() when Base has only Base(int).

Answer: Compilation fails until Child calls an accessible Base(int) constructor.

Overriding gives a new implementation

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An overriding instance method provides subtype-specific behavior for a compatible inherited method signature. @Override makes the compiler verify your intention. Runtime dispatch chooses the implementation associated with the actual receiver object.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal a = new Dog();
System.out.println(a.speak());
// Dog overrides Animal.speak().
```

EXPECTED OUTPUT

```
woof
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The declared Animal reference can call speak().
2. The actual object is Dog and overrides speak().
3. The overriding Dog implementation produces woof.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The annotation @Override catches accidental name and parameter mismatches at compile time. Inherited concrete instance methods are dynamically dispatchable unless restricted by final or other rules. Runtime dispatch uses the actual receiver type, but only after the compiler has found an applicable method in the declared reference type. Only compatible instance methods are dynamically overridden. Using @Override catches signature mistakes early. A subclass cannot override final methods, and private methods are not overridden because they are not inherited as overridable members. Static methods and fields follow different hiding rules.

COMMON TRAP

Different parameter types generally create an overload rather than an override.

TRY THIS IN INTELLIJ

Spell speak as Speak in Dog; what happens to @Override?

REMEMBER The overriding Dog implementation produces woof.

APPLY THE RULE / CHANGE ONE THING

Remove @Override and misspell speak as speek.

Answer: The typo becomes a new method; @Override would have caught it at compile time.

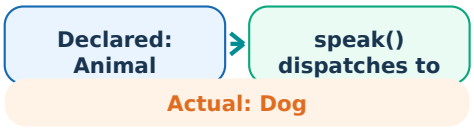
Dynamic dispatch: declared type versus actual object

Understand the rule • trace the code • read the diagram • test it

CORE RULE

The compiler checks accessible members using the declared reference type; the JVM chooses an overridden instance method implementation using the runtime object type. Animal provides the callable contract.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal animal = new Dog("Rex");
System.out.println(animal.speak());
System.out.println(animal.getName());
// animal.fetch(); // not on Animal type
```

EXPECTED OUTPUT

```
woof
Rex
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The compiler checks animal as an Animal reference.
2. It permits speak() and getName() because Animal declares them.
3. At runtime, Dog.speak() executes, while the name comes from the same object.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

There are two separate types to track on paper: declared Animal, actual Dog. Declared type answers whether speak() exists on the expression; actual Dog answers which implementation of speak() executes. This distinction also matters when a superclass method itself calls another overridable method: the nested call can dispatch back into the child. Always annotate a reference with two types when tracing: declared type and runtime object type. The declared type controls which methods you may call and can influence overload resolution; the runtime receiver controls the implementation of overridden instance methods. Switching the object to a Cat changes dispatch without changing the client variable type.

COMMON TRAP

The fact that the runtime object is Dog does not allow you to call Dog-only methods through an Animal reference.

TRY THIS IN INTELLIJ

Assign new Cat("Milo") to the same reference and call speak again.

REMEMBER At runtime, Dog.speak() executes, while the name comes from the same object.

APPLY THE RULE / CHANGE ONE THING

Set animal = new Cat("Milo") and call speak().

Answer: The Cat implementation runs while the declared type remains Animal.

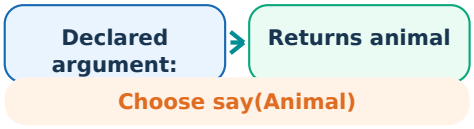
Overloading is selected before dynamic dispatch

Understand the rule • trace the code • read the diagram • test it

CORE RULE

For overloaded methods, Java resolves which signature applies using compile-time types. Only after that does dynamic dispatch select an override for that signature.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static String say(Animal a)
{
    return "animal";
}
static String say(Dog d)
{
    return "dog";
}
Animal ref = new Dog("Rex");
System.out.println(say(ref));
```

EXPECTED OUTPUT

```
animal
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The compiler sees ref has declared type Animal.
2. say(Animal) is selected among overloads at compile time.
3. It returns animal even though ref points at a Dog.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

At this call site, ref has the declared type Animal, so the compiler selects say(Animal). The actual Dog instance does not cause a second overload search. Both say methods in this example are static, so the selected say(Animal) returns "animal" directly. For instance methods, a separate later step could dynamically dispatch an override of the selected signature. Overload choice and override dispatch are separate stages. First determine the signature from compile-time types and conversion rules; only then, if that selected method is overridable, determine the implementation from the receiver’s runtime type. Treating every method choice as “runtime polymorphism” gives wrong answers.

COMMON TRAP

Do not summarize Java dispatch as “the most specific method is always chosen at runtime.”

TRY THIS IN INTELLIJ

Cast ref to Dog in the call and predict the chosen overload.

REMEMBER It returns animal even though ref points at a Dog.

APPLY THE RULE / CHANGE ONE THING

Change only the variable declaration to Dog ref = new Dog(...).

Answer: The call now selects say(Dog) and prints dog.

Static methods are hidden, not overridden

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Static method calls are resolved from compile-time type information. A same-signature static declaration in a subclass hides the inherited static method.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Parent
{
    static String tag()
    {
        return "P";
    }
}
class Child extends Parent
{
    static String tag()
    {
        return "C";
    }
}
Parent p = new Child();
System.out.println(p.tag());
```

EXPECTED OUTPUT

```
P
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Parent p references a Child instance.
2. Static tag() is resolved using Parent at compile time.
3. Parent.tag() returns P, not Child.tag().

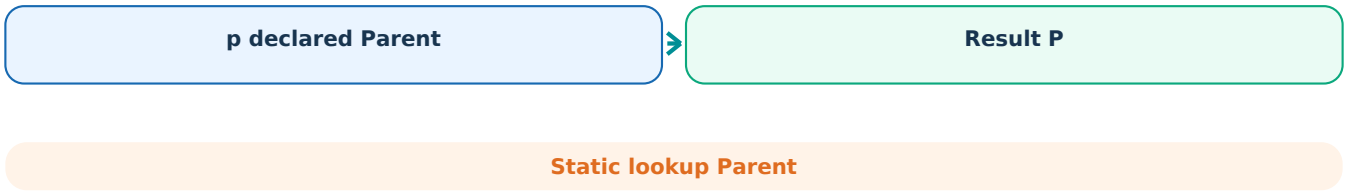
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Static methods are hidden, not overridden

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

Parent p references a Child instance. Static tag() is resolved using Parent at compile time. Parent.tag() returns P, not Child.tag().

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Calling p.tag() is legal syntax but misleading for a static method. The compile-time type Parent chooses Parent.tag; there is no virtual dispatch for static functions. Prefer Parent.tag() and Child.tag() in production code. Likewise, a subclass cannot override a static parent declaration with an instance method of the same signature. A same-signature static method declared in a subclass hides a superclass method, rather than overriding it. The recommended call form is Parent.tag() or Child.tag(), not p.tag(), because the latter disguises static selection as an instance operation. The object's runtime subtype does not choose the method.

COMMON TRAP

Do not put @Override on a static method hiding declaration.

TRY THIS IN INTELLIJ

Change p.tag() to Child.tag() and predict the result.

REMEMBER Parent.tag() returns P, not Child.tag().

APPLY THE RULE / CHANGE ONE THING

Replace p.tag() with Child.tag().

Answer: The result changes to C because the class name now selects Child.

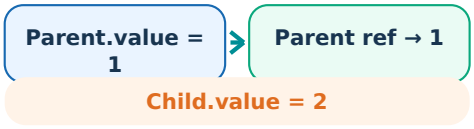
Fields are hidden, not polymorphically overridden

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A field with the same name in a subclass hides the inherited field. The selected field depends on the compile-time reference type.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Parent
{
    int value = 1;
}
class Child extends Parent
{
    int value = 2;
}
Parent p = new Child();
System.out.println(p.value);
```

EXPECTED OUTPUT

```
1
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. p is declared Parent and holds a Child object.
2. p.value is a field access, bound to Parent.value.
3. It prints 1 rather than the Child field value 2.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Two fields with the same simple name can coexist in a child object, one from Parent and another declared in Child. Which one you access depends on the reference type. Encapsulation discourages this pattern: private fields with behavior methods let a subtype supply implementation without exposing confusing duplicate data slots. Fields do not participate in virtual dispatch. A child field with the same name is a separate member, not a replacement of the superclass field. This is usually confusing enough to avoid: keep fields private and expose a method if dynamic subtype-specific behavior is needed.

COMMON TRAP

Hiding fields creates confusion; prefer distinct names and accessors rather than field shadowing.

TRY THIS IN INTELLIJ

Cast p to Child before .value, then predict the result.

REMEMBER It prints 1 rather than the Child field value 2.

APPLY THE RULE / CHANGE ONE THING

Add an overridable getValue() and call p.getValue().

Answer: With a Child override, the method can return 2 via dynamic dispatch.

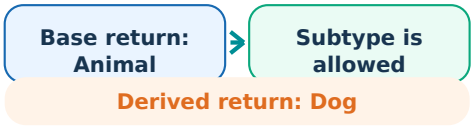
Covariant return types

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An overriding method may return a subtype of the original reference return type (covariant return). Primitive return types cannot change this way.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class AnimalFactory
{
    Animal make()
    {
        return new Dog("Rex");
    }
}
class DogFactory extends AnimalFactory
{
    @Override
    Dog make()
    {
        return new Dog("Bolt");
    }
}
```

EXPECTED OUTPUT

```
No console output (override example).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. AnimalFactory.make declares Animal as its result.
2. DogFactory overrides make and narrows the result to Dog.
3. The override stays compatible because Dog is a subtype of Animal.

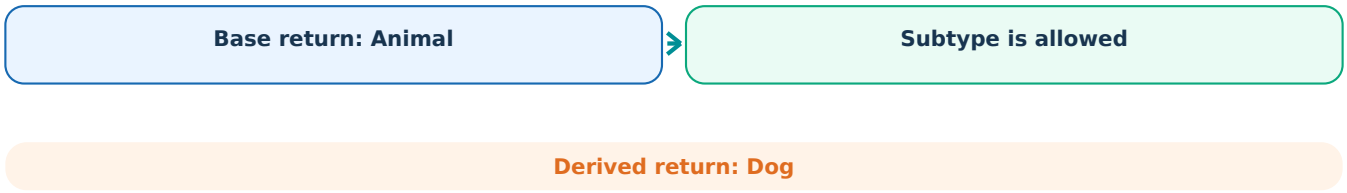
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Covariant return types

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

AnimalFactory.make declares Animal as its result. DogFactory overrides make and narrows the result to Dog. The override stays compatible because Dog is a subtype of Animal.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Covariant return types preserve substitutability while allowing a more specific object result. If AnimalFactory.make() promises Animal, DogFactory.make() may promise Dog. The subtype is still an Animal for callers typed as the base. A changed primitive return type or an unrelated reference return type would break the override contract. Covariant returns help callers of a subtype-specific factory receive more precise return types without casts. They apply to reference returns, not primitive widening. The parameter signature still has to match the overridden method; changing parameters creates an overload instead.

COMMON TRAP

Changing an int-returning method override to long is not a covariant return.

TRY THIS IN INTELLIJ

Change DogFactory.make() to return String and predict compilation.

REMEMBER The override stays compatible because Dog is a subtype of Animal.

APPLY THE RULE / CHANGE ONE THING

Try replacing Dog make() with int make().

Answer: The override is invalid: primitive return changes are not covariant.

Override restrictions: access and exceptions

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An overriding method cannot narrow visibility and cannot declare broader checked exceptions than the overridden contract permits. FileNotFoundException is narrower than IOException; public is more accessible than protected, so this override is legal.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Base
{
    protected void act() throws IOException
    {
    }
}
class Derived extends Base
{
    @Override
    public void act() throws FileNotFoundException
    {
    }
}
```

EXPECTED OUTPUT

```
No console output (legal override).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Base.act is protected and declares IOException.
2. Derived.act is public, increasing visibility.
3. FileNotFoundException is narrower, so the override is legal.

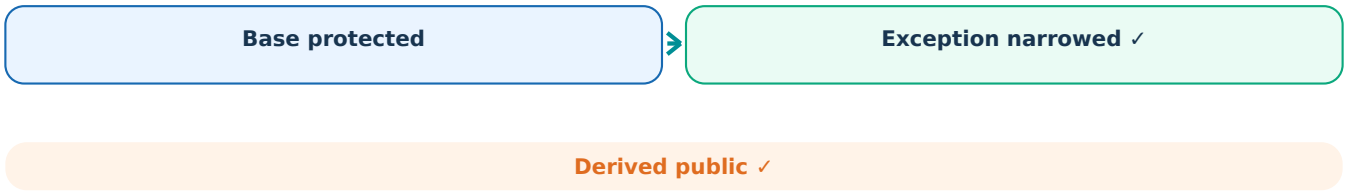
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Override restrictions: access and exceptions

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

Base.act is protected and declares IOException. Derived.act is public, increasing visibility. FileNotFoundException is narrower, so the override is legal.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The Java compiler checks both accessibility and checked-exception compatibility. Making an override more public is allowed; narrowing it is not. Throwing FileNotFoundException where IOException was promised is legal, because callers already know how to handle the broader category. Runtime exceptions are handled differently and do not need declaration in throws. An overriding method must preserve the contract callers rely on. It cannot restrict visibility or introduce a broader checked exception; unchecked exceptions follow different rules. Java also checks return compatibility and static/instance consistency. Reading @Override failures is a useful way to learn these contracts.

COMMON TRAP

Trying to make Derived.act private, or to throw Exception in this example, fails compilation.

TRY THIS IN INTELLIJ

Replace FileNotFoundException with Exception. Why is it rejected?

REMEMBER FileNotFoundException is narrower, so the override is legal.

APPLY THE RULE / CHANGE ONE THING

Change Derived.act to private or throws Exception.

Answer: The override fails because visibility is narrower or checked exception is broader.

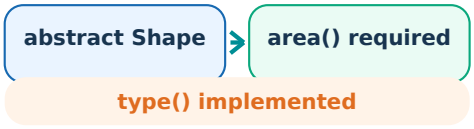
Abstract classes describe partial implementations

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An abstract class cannot be instantiated directly. It may hold constructors, state, implemented methods, and abstract methods requiring subtype implementation.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
abstract class Shape
{
    abstract double area();
    String type()
    {
        return "shape";
    }
}
// new Shape(); // compile error
```

EXPECTED OUTPUT

```
No console output (abstract class).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Shape is abstract and cannot be instantiated directly.
2. It can provide type() behavior while requiring subclasses to implement area().
3. Concrete subclasses become instantiable when all abstract operations are implemented.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

An abstract class may carry constructors, state and concrete helper methods, while an interface emphasizes a callable contract. Neither choice is automatically superior. In Java, even an abstract superclass constructor participates in subclass creation. Use abstract classes when shared implementation/state is intrinsic to the subtype relationship, not merely because some methods look similar. An abstract class can have fields, constructors, and implemented methods; it cannot be instantiated directly. It is useful for genuinely shared state and implementation. An interface is usually a better first choice for a capability contract when implementations need unrelated class hierarchies.

COMMON TRAP

Abstract methods have no body and cannot be private, final, or static in a way that would prevent implementation.

TRY THIS IN INTELLIJ

Create Circle extends Shape with area() returning Math.PI*r*r.

REMEMBER Concrete subclasses become instantiable when all abstract operations are implemented.

APPLY THE RULE / CHANGE ONE THING

Make a concrete Rectangle extends Shape without implementing area().

Answer: Compilation fails unless Rectangle itself is declared abstract.

Interfaces define contracts between components

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An interface names operations independent of a concrete implementation. An implementation class promises to provide required instance methods.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
interface PaymentMethod
{
    String pay(int cents);
}
class Card implements PaymentMethod
{
    public String pay(int cents)
    {
        return "CARD:" + cents;
    }
}
```

EXPECTED OUTPUT

```
No console output (contract definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. PaymentMethod declares the operation pay(int).
2. Card implements that contract with a public method.
3. A PaymentMethod reference can later call pay on any implementation.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Interface-typed dependencies make code resistant to concrete implementation changes. A PaymentService can call payment.pay(cents) without knowing whether the actual object is CardPayment or BankTransfer. The implementation method must be public because interface methods declared this way are public. A class can implement many such contracts even though it extends only one class. Interfaces isolate consumers from implementation classes. An implementing method must be public because it fulfills a public interface contract. A class may implement multiple interfaces, and interfaces can extend multiple interfaces; this supports multiple capabilities without multiple class inheritance.

COMMON TRAP

An interface is not a class constructor; new PaymentMethod() is invalid without an anonymous implementation.

TRY THIS IN INTELLIJ

Add another implementation called Bank and run the same client through each.

REMEMBER A PaymentMethod reference can later call pay on any implementation.

APPLY THE RULE / CHANGE ONE THING

Create Cash implements PaymentMethod; pass either Card or Cash to a client.

Answer: The client works through the same PaymentMethod contract.

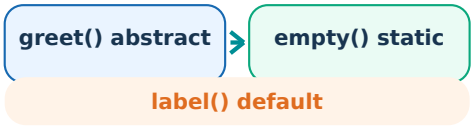
Interface default, static, and private methods

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Modern interfaces can supply default methods, static helper methods, and private helpers used within the interface. Default methods support evolving an interface with behavior; they do not turn an interface into a class with instance fields.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
interface Greeting
{
    String greet();
    default String label()
    {
        return "Greeting";
    }
    static Greeting empty()
    {
        return () -> "";
    }
}
```

EXPECTED OUTPUT

```
No console output (interface definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Greeting declares greet() abstract.
2. label() is a default method available to implementations.
3. Greeting.empty() is an interface static factory returning a lambda.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A default interface method helps add a general behavior while keeping old implementations source-compatible in many cases. Interface static methods are invoked using the interface name. Private interface methods can factor shared logic between defaults or statics. A default method must not be confused with a virtual instance field: interfaces cannot hold mutable per-object state. Default methods offer reusable behavior while allowing old implementations to remain source-compatible when an interface evolves. They are still instance methods. Interface static methods belong to the interface name; private interface methods help share internal default/static implementation logic and are not inherited as public API.

COMMON TRAP

A default method conflict from unrelated interfaces may require an explicit implementation in the class.

TRY THIS IN INTELLIJ

Implement Greeting with an anonymous class, then print label().

REMEMBER Greeting.empty() is an interface static factory returning a lambda.

APPLY THE RULE / CHANGE ONE THING

Attempt to call implementation.label() and Greeting.empty().label().

Answer: Both default calls work when the receiver implements Greeting.

Resolving conflicting default methods

Understand the rule • trace the code • read the diagram • test it

CORE RULE

If two unrelated interfaces provide the same default method, a class implementing both must normally override and resolve the ambiguity. A concrete class can implement tag() itself and may invoke a specific inherited default with Left.super.tag() where permitted.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
interface Left
{
    default String tag()
    {
        return "L";
    }
}
interface Right
{
    default String tag()
    {
        return "R";
    }
}
```

EXPECTED OUTPUT

```
No console output (partial conflict setup).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Left provides default tag() returning L.
2. Right provides default tag() returning R.
3. A class implementing both must explicitly resolve the ambiguous default.

NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Resolving conflicting default methods

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

Left provides default tag() returning L. Right provides default tag() returning R. A class implementing both must explicitly resolve the ambiguous default.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

When both Left and Right provide tag(), Java demands an explicit decision rather than arbitrarily picking one. The implementing class can provide its own tag() and delegate to Left.super.tag() if appropriate. Class methods have priority over interface defaults under Java's resolution rules, which matters in mixed class/interface hierarchies. The class can override tag() with its own body, or choose a specific interface default using Left.super.tag() when the syntax applies. Java also has priority rules: a concrete class method beats an interface default; a more specific subinterface can win over a superinterface default.

COMMON TRAP

Java avoids guessing which unrelated default method the programmer intended.

TRY THIS IN INTELLIJ

Write class Both implements Left, Right and return Left.super.tag() from its override.

REMEMBER A class implementing both must explicitly resolve the ambiguous default.

APPLY THE RULE / CHANGE ONE THING

Implement Left and Right without writing tag().

Answer: Compilation fails due to conflicting inherited defaults.

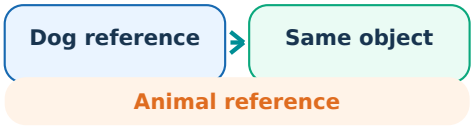
Upcasting to a parent type or interface

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An upcast stores a subtype reference in a variable of a supertype. It is normally implicit and does not change the object.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Dog dog = new Dog("Rex");
Animal animal = dog;
System.out.println(dog == animal);
System.out.println(animal.speak());
```

EXPECTED OUTPUT

```
true
woof
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Dog dog refers to a Dog object.
2. Animal animal = dog copies the reference into a supertype variable.
3. Both references point at the same object; identity comparison prints true.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

An upcast does not construct a new object or remove Dog-only fields. It merely changes the reference's declared viewpoint. That means equals, overridden methods, and object identity still observe the same Dog instance. APIs use this to accept a broad Animal or PaymentMethod contract rather than a concrete Dog or CardPayment class. Upcasting changes what the compiler permits through a reference; it does not create or convert an object. This lets clients depend on a stable abstraction instead of concrete implementation details. The object still supports Dog-specific behavior, but reaching it through an Animal reference requires an appropriate narrowing cast.

COMMON TRAP

Upcasting does not slice objects in Java; the whole original object remains in memory.

TRY THIS IN INTELLIJ

Assign a Dog object to an Object variable and call toString().

REMEMBER Both references point at the same object; identity comparison prints true.

APPLY THE RULE / CHANGE ONE THING

Mutate the Dog through dog after assigning animal = dog.

Answer: The Animal reference sees the same object's changed state.

Downcasting: a runtime assertion about type

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A downcast requests a subtype reference. It can compile even when it later fails because the object is not actually an instance of that subtype.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal animal = new Cat("Milo");
try
{
    Dog dog = (Dog) animal;
}
catch (ClassCastException e)
{
    System.out.println("Wrong subtype");
}
```

EXPECTED OUTPUT

```
Wrong subtype
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. animal refers to a Cat object.
2. The cast to Dog compiles because an Animal could be a Dog.
3. At runtime the Cat is incompatible, so ClassCastException is caught.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A downcast is valid only when the runtime object really belongs to the target subtype. The language may reject casts that can never be valid at compile time; otherwise an incompatible runtime object leads to ClassCastException. A design that repeatedly downcasts from interfaces may be missing a necessary operation on the interface itself. A cast asserts a subtype relationship about the existing object. It does not manufacture a different subtype or change the object's runtime class. Prefer instanceof checks or better interface design when casts would otherwise be common. If a cast is impossible by type rules, Java may reject it at compile time.

COMMON TRAP

Casting an unrelated final type may be rejected at compile time rather than reaching ClassCastException.

TRY THIS IN INTELLIJ

Replace new Cat with new Dog and call a Dog-only method.

REMEMBER At runtime the Cat is incompatible, so ClassCastException is caught.

APPLY THE RULE / CHANGE ONE THING

Replace new Cat with new Dog in the setup.

Answer: The Dog cast succeeds and can be used for Dog-only methods.

instanceof and pattern matching (Java 17)

Understand the rule • trace the code • read the diagram • test it

CORE RULE

instanceof tests whether a reference is compatible with a type. Java 17 pattern matching can introduce a typed binding in a successful test.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal animal = new Dog("Rex");
if (animal instanceof Dog dog)
{
    System.out.println(dog.fetch());
}
```

EXPECTED OUTPUT

```
fetching
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. animal points at a Dog.
2. instanceof Dog succeeds and binds dog as a typed pattern variable.
3. dog.fetch() is callable inside the guarded branch and prints fetching.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Pattern matching reduces redundant test-then-cast boilerplate but retains the same underlying type safety. The variable dog is in scope where the compiler knows the match succeeded, including flow-sensitive combinations such as conditions involving &&. Do not assume the variable is available after any arbitrary if; scope depends on the code path analysis. Pattern matching avoids a redundant explicit cast and introduces a scoped binding that exists where the compiler can prove the test succeeded. If the object is null, instanceof returns false. Flow-scoping rules matter when tests are combined with negation and logical operators.

COMMON TRAP

instanceof null is false. It does not need a separate null check to return false.

TRY THIS IN INTELLIJ

Try Animal animal = null and predict whether the body runs.

REMEMBER dog.fetch() is callable inside the guarded branch and prints fetching.

APPLY THE RULE / CHANGE ONE THING

Change animal to null.

Answer: The condition is false and the block prints nothing.

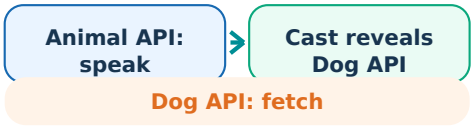
What you can call is a compile-time question

Understand the rule • trace the code • read the diagram • test it

CORE RULE

The declared type determines method availability at the call site, even if the actual object contains additional public methods. Dynamic dispatch answers which implementation of an accessible overridden method runs.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal a = new Dog("Rex");
System.out.println(a.speak());
// a.fetch();     // compile error
System.out.println(((Dog) a).fetch());
```

EXPECTED OUTPUT

```
woof
fetching
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The declared type Animal exposes speak but not fetch.
2. The first call dynamically invokes Dog.speak and prints woof.
3. A checked cast exposes fetch and the second call prints fetching.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The declared reference type is a compile-time promise. If an interface does not expose fetch(), callers cannot use it through that interface even if the actual object happens to be a Dog. Casting can expose the extra method, but that couples the caller to an implementation detail. Interfaces should contain the operations clients genuinely need. Compile-time accessibility and runtime dispatch answer different questions. A downcast makes a narrower method set available to the compiler but must still be valid for the runtime object. If a client continually needs Dog-only operations, revisit whether the Animal abstraction has the right responsibilities.

COMMON TRAP

An interface variable cannot call methods missing from its declared interface simply because you know the implementation class.

TRY THIS IN INTELLIJ

Create PaymentMethod method = new CardPayment() and list which methods are callable.

REMEMBER A checked cast exposes fetch and the second call prints fetching.

APPLY THE RULE / CHANGE ONE THING

Comment out the cast and attempt a.fetch().

Answer: Compilation fails because fetch is not declared on Animal.

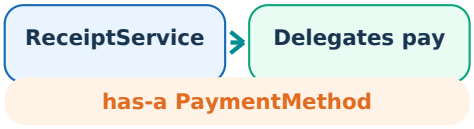
Composition versus inheritance: choosing wisely

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Use inheritance for a genuine substitutable is-a relationship. Use composition when one object should use another object's behavior.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class ReceiptService
{
    private final PaymentMethod payment;
    ReceiptService(PaymentMethod payment)
    {
        this.payment = payment;
    }
    String process(int cents)
    {
        return payment.pay(cents);
    }
}
```

EXPECTED OUTPUT

```
No console output (delegating class).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. ReceiptService stores the supplied PaymentMethod collaborator.
2. process delegates to payment.pay(cents).
3. The service does not need to extend PaymentMethod to use that behavior.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Composition helps avoid brittle base-class assumptions. ReceiptService can delegate to a payment interface and be tested with a fake implementation; it does not need to extend the payment processor. Inheritance remains appropriate for well-defined subtype families, but it is harder to revise because base behavior and constructor rules propagate into every child. Inheritance should be reserved for a true substitutable is-a relationship. Composition connects independent responsibilities and makes collaborators replaceable, testable, and easier to evolve. Prefer depending on an interface through constructor injection, as in this example, so replacing Card with Cash does not require changing ReceiptService.

COMMON TRAP

"Prefer composition" is guidance, not a ban on inheritance; abstract hierarchies still have legitimate uses.

TRY THIS IN INTELLIJ

Swap CardPayment for BankTransfer without changing ReceiptService.

REMEMBER The service does not need to extend PaymentMethod to use that behavior.

APPLY THE RULE / CHANGE ONE THING

Swap Card for a test PaymentMethod returning a fixed value.

Answer: ReceiptService remains unchanged; the dependency supplies different behavior.

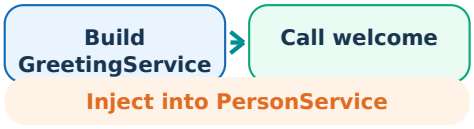
Dependency injection before Spring

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Constructor injection is ordinary Java: a class receives an abstraction from the outside instead of constructing a concrete dependency internally. An interface-typed field and constructor makes a dependency explicit.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
GreetingService greeter =
    new FriendlyGreetingService();
PersonService people =
    new PersonService(greeter);
System.out.println(people.welcome(person));
```

EXPECTED OUTPUT

```
Hello, Asha Rao! (project demo)
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. An implementation of GreetingService is constructed.
2. The PersonService constructor receives that interface reference.
3. welcome delegates to the injected service and prints the supplied greeting.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Constructor injection means the caller chooses the collaborator and supplies it as a dependency. Spring later builds on the same idea by managing bean lifecycles and wiring dependencies. The plain Java example already teaches the important structure: interface-typed field, explicit constructor contract and interchangeable implementations. No framework magic is required to understand it. Dependency injection is a design pattern before it is a framework feature. The caller or composition root decides which implementation to construct; the consumer declares its needs through a constructor. Spring will later automate much of the assembly, but the principles of abstraction, ownership, and testability are ordinary Java.

COMMON TRAP

Creating new FriendlyGreetingService() inside PersonService would hide the collaborator and make substitution harder.

TRY THIS IN INTELLIJ

Replace FriendlyGreetingService with a second implementation.

REMEMBER welcome delegates to the injected service and prints the supplied greeting.

APPLY THE RULE / CHANGE ONE THING

Pass a FakeGreetingService with a predictable output instead.

Answer: PersonService can be tested without changing its implementation.

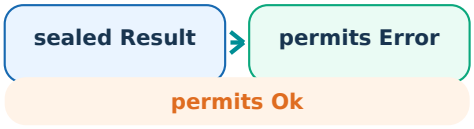
sealed hierarchies in Java 17

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A sealed class or interface restricts which directly declared types may extend or implement it. Java 17 supports sealed, permits, non-sealed, and final.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
sealed interface Result
    permits Ok, Error
{
}
final class Ok implements Result
{
}
final class Error implements Result
{
}
```

EXPECTED OUTPUT

```
No console output (closed hierarchy).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Result declares the only permitted direct implementations.
2. Ok and Error are final concrete implementations.
3. No unlisted class may directly implement Result in an otherwise valid context.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Sealed classes and interfaces are a Java 17 language feature. Direct permitted subtypes must obey the sealed/final/non-sealed classification, so the type family's expansion is controlled. This helps when reasoning about a limited set of outcomes. Sealed types do not eliminate the need to design meaningful contracts or handle unknown states when integrating external input. Sealed hierarchies make allowed subtype sets explicit. Permitted direct subclasses must themselves be final, sealed, or non-sealed as required, and permitted types must satisfy language locality rules. In Java 17, sealed types are useful when modeling finite result families, though ordinary interfaces remain better for open extension points.

COMMON TRAP

A sealed interface is not automatically an enum and does not force all implementations into one source file.

TRY THIS IN INTELLIJ

Try adding class Pending implements Result without changing permits.

REMEMBER No unlisted class may directly implement Result in an otherwise valid context.

APPLY THE RULE / CHANGE ONE THING

Add class Pending implements Result without permitting it.

Answer: The new direct implementation is rejected by the compiler.

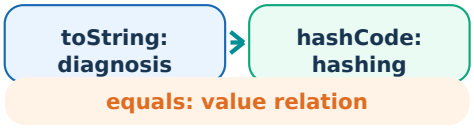
Object methods: toString, equals, hashCode

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Every class inherits core Object behavior. toString supports a readable diagnostic representation; equals/hashCode define equality and hashing semantics. Override Object methods together where relevant and keep their contracts consistent.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person p = new Person(101, "Asha");
System.out.println(p.toString());
System.out.println(p.equals(p));
```

EXPECTED OUTPUT

```
Person[id=101, name=Asha] (illustrative format)
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The created Person has a readable toString() in this illustration.
2. p.equals(p) is true by reflexivity.
3. If logical equality is defined, hashCode must be consistent with it.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

toString is primarily for debugging and diagnostics; do not rely on its default format for serialization. equals and hashCode should describe stable logical identity where needed. getClass reports the actual runtime class; casts do not alter it. Sensitive values like passwords or tokens should not leak into automatically generated toString methods. Object supplies foundational methods, but their default implementations may not match your domain’s identity requirements. If you override equals(), also override hashCode(): equal objects must have equal hashes. A custom toString() should support diagnosis without exposing secrets. Records synthesize value-oriented implementations for their components.

COMMON TRAP

A stable toString format is not guaranteed by default Object.toString and should not be parsed as business data.

TRY THIS IN INTELLIJ

Compare p.toString() with the overridden output in the companion project.

REMEMBER If logical equality is defined, hashCode must be consistent with it.

APPLY THE RULE / CHANGE ONE THING

Insert two equal value objects into a HashSet without a consistent hashCode().

Answer: Hash-based collections can behave incorrectly when equals/hashCode contracts are broken.

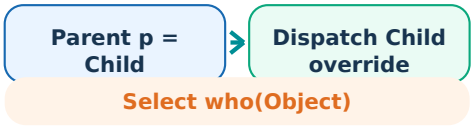
Trace declared types, overloads, and overrides

Understand the rule • trace the code • read the diagram • test it

CORE RULE

When a complex expression combines inheritance with overloaded methods, resolve signatures first, then find runtime implementation. Compiler sees Parent and chooses who(Object).

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Parent
{
    String who(Object o)
    {
        return "P:Object";
    }
}
class Child extends Parent
{
    @Override
    String who(Object o)
    {
        return "C:Object";
    }
    String who(String s)
    {
        return "C:String";
    }
}
Parent p = new Child();
System.out.println(p.who("hi"));
```

EXPECTED OUTPUT

```
C:Object
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The variable p is declared Parent though it holds a Child.
2. At compile time, only who(Object) is visible on Parent.
3. At runtime, Child overrides who(Object), so it prints C:Object.

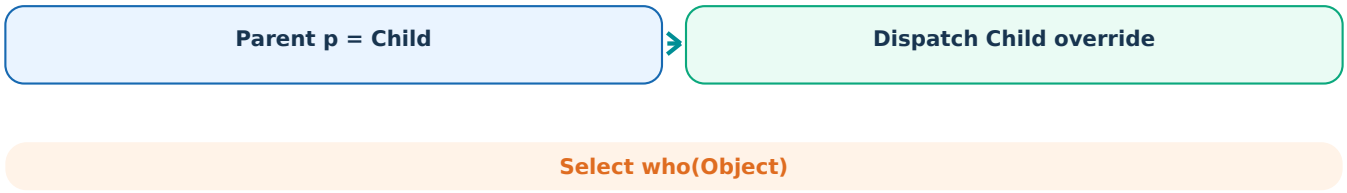
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Trace declared types, overloads, and overrides

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

The variable p is declared Parent though it holds a Child. At compile time, only who(Object) is visible on Parent. At runtime, Child overrides who(Object), so it prints C:Object.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The reliable algorithm is: (1) list candidates visible through the declared receiver type, (2) resolve overload using compile-time argument types, (3) choose the runtime override for that selected signature. Child.who(String) is not considered through a Parent variable because the Parent API does not declare it.

COMMON TRAP

Do not skip the compile-time resolution step just because the runtime object is a subclass.

TRY THIS IN INTELLIJ

Change the declared type of p to Child and predict the result.

REMEMBER At runtime, Child overrides who(Object), so it prints C:Object.

APPLY THE RULE / CHANGE ONE THING

Change Parent p to Child p without changing new Child().

Answer: Now who(String) can be selected, producing C:String.

CHAPTER 3 · PART C · VISUAL MASTER MAP

Core rules • mental models • common pitfalls • executable examples



01 INHERITANCE

- extends means substitutable is-a.
- Superclass construction happens first.
- Do not inherit only to reuse code.

02 OVERRIDING

- Same compatible instance signature.
- Runtime receiver chooses implementation.
- Use @Override to catch mistakes.

03 INTERFACES

- Contracts separate callers from implementations.
- One class implements many interfaces.
- Default methods may provide shared behavior.

04 OVERLOADING

- Different parameter lists.
- Chosen with compile-time argument types.
- Not itself runtime polymorphism.

05 CASTING & TYPE TESTS

- Upcasting is normally implicit.
- Downcast can fail at runtime.
- instanceof patterns narrow safely.

06 COMPOSITION & DI

- Has-a instead of fragile is-a.
- Constructor-inject interfaces.
- Spring later automates this wiring.

EXAMPLE — SOURCE AND OUTPUT

CODE / RULE

```
Animal a = new Dog("Rex");
System.out.println(a.speak());
// Dog.speak() executes at runtime.
```

EXPECTED RESULT

```
woof
```

PART C / PRACTICE

Predict before running

9 thinking questions. Answer from the Java rule, not from memorized output.

C1 Why does Animal a = new Dog() allow a.speak() but not a.fetch()?

C2 When a call is overloaded and overridden, which decision happens first?

C3 Is a same-named static method in a subclass an override?

C4 Does a field access dynamically dispatch like a method call?

C5 Why is a Cat-to-Dog downcast unsafe?

C6 What does instanceof Dog dog introduce?

C7 When are sealed hierarchies helpful?

C8 Why prefer composition for ReceiptService and PaymentMethod?

C9 What is dependency injection without Spring?

PART C / ANSWER KEY

Reasoning and answers

Explain each rule in your own words before moving on.

C1 Member lookup uses declared Animal type; overriding of speak dispatches using the Dog runtime receiver.

C2 Compiler selects a method signature from declared types; runtime dispatch chooses an override of that signature.

C3 No. Static methods are hidden and selected via compile-time type.

C4 No. Fields are selected using compile-time reference type.

C5 A cast does not transform the object; incompatible casts throw ClassCastException (or fail compilation).

C6 A pattern variable of Dog type available where a successful type test is known.

C7 When the direct subtype family should be explicitly constrained and understood.

C8 ReceiptService uses payment behavior; it is not a kind of payment implementation.

C9 Construct a class with collaborators supplied from outside, often as interface-typed constructor parameters.

Chapter 4
Essential Java Types for Business Applications

WHY THIS CHAPTER MATTERS

Most production bugs involving values are not syntax bugs. They come from comparing the wrong things, unexpected nulls, binary floating-point rounding, or accidentally mixing a local date-time with an absolute instant.

What you will be able to do

- Choose the right type for text, numbers, statuses, money and time.
- Explain equality, null, immutability and formatting rules.
- Use BigDecimal without introducing silent decimal errors.
- Model calendar dates, wall-clock times and UTC instants correctly.

HOW TO STUDY

Read the explanation; trace the code before looking at output; test the variation in IntelliJ. Code panels are focused teaching snippets; the companion project contains complete compilable examples.

23 practical lessons • illustrated reference poster • executable companion project

CHAPTER 4 • ESSENTIAL JAVA TYPES

Text • wrappers • enums • precision • dates • utility APIs

WHY THIS MATTERS

Core Java building blocks for reliable business applications.

MASTER THESE

- String is immutable; compare content with equals().
- Use BigDecimal(String) for decimal money.
- java.time distinguishes dates, instants and zones.

KEY TAKEAWAY

CHOOSE A TYPE THAT MATCHES YOUR DATA

1 String & Equality

Immutable text and comparison

- String stores text; operations create new String values.
- Use equals() for content; == checks object identity.
- length() counts UTF-16 code units, not always characters.

QUICK MODEL

Same content
Different objects
Use equals()

JAVA EXAMPLE • illustrative snippet

```
String a = "Java";
String b = new String("Java");
System.out.println(a.equals(b));
System.out.println(a == b);
```

OUTPUT true | false

2 StringBuilder

Efficient mutable text

- StringBuilder changes its own character sequence.
- Prefer it for repeated concatenation in a loop.
- StringBuffer is synchronized; rarely needed for new code.

QUICK MODEL

String
immutable
Builder: mutable

JAVA EXAMPLE • illustrative snippet

```
StringBuilder sb = new StringBuilder("Hi");
sb.append(" Java");
System.out.println(sb);
```

OUTPUT Hi Java

3 Wrapper Types

Autoboxing, null and identity

- Integer, Double and Boolean wrap primitives.
- Boxing/unboxing is automatic in many expressions.
- Unboxing a null wrapper throws NullPointerException.

QUICK MODEL

Integer ↔ int
Nullable wrapper
Avoid == for boxed values

JAVA EXAMPLE • illustrative snippet

```
Integer n = Integer.valueOf(42);
int value = n;
System.out.println(value);
```

OUTPUT 42

4 Enums

Named, type-safe values

- Enums model a fixed set of legitimate values.
- They can declare fields, constructors and methods.
- Enum constants are compared safely with ==.

QUICK MODEL

Status.NEW
Status.PAID
Type-safe

JAVA EXAMPLE • illustrative snippet

```
enum Status
{
    NEW, PAID
}
System.out.println(Status.PAID.name());
```

OUTPUT PAID

5 BigDecimal

Money and exact decimal rules

- Construct decimal values from strings, not double.
- Use add/subtract/multiply and controlled division.
- equals checks scale too; compareTo compares numeric value.

QUICK MODEL

0.1 + 0.2
double ≠ exact
BigDecimal exact

JAVA EXAMPLE • illustrative snippet

```
BigDecimal a = new BigDecimal("0.10");
BigDecimal b = new BigDecimal("0.20");
System.out.println(a.add(b));
```

OUTPUT 0.30

6 java.time

Date, time and time zones

- LocalDate is a calendar date with no time or zone.
- Instant represents a UTC timeline point.
- ZonedDateTime accounts for zone rules and DST.

QUICK MODEL

LocalDate: date
Instant: point
Zone: local rules

JAVA EXAMPLE • illustrative snippet

```
LocalDate day = LocalDate.of(2026, 10, 11);
System.out.println(day.plusDays(7));
```

OUTPUT 2026-10-18

WATCH OUT • COMMON MISTAKES

- Using == to compare String content
- Using double for decimal money
- Unboxing null wrappers
- Assuming LocalDate has a timezone

QUICK RECALL • BEST PRACTICES

Utility helpers: Objects.requireNonNull, Arrays.toString, Math, UUID.

Chapter 4 reminder: immutable value types simplify business logic.

CHAPTER 4 / LESSON 01 OF 23

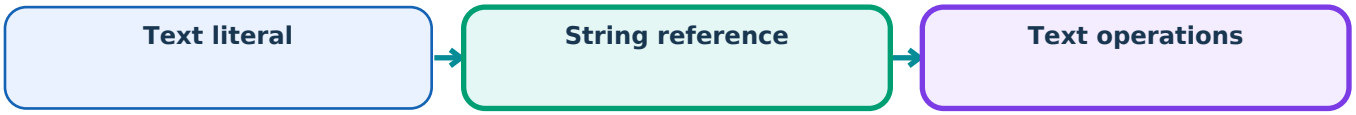
String is a reference type, not a primitive

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

String stores text and is a reference type. Its variable can refer to an object or be null; a char stores one UTF-16 code unit.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String city = "Chicago";
char first = city.charAt(0);
System.out.println(city.length());
System.out.println(first);
```

EXPECTED OUTPUT / RESULT

```
7
C
```

EXECUTION TRACE / READ THE STEPS

1. The literal creates or retrieves a String instance and assigns a reference value.
2. length() reports seven UTF-16 code units in this ASCII example.
3. charAt(0) reads the first UTF-16 code unit, C.

DEEPER UNDERSTANDING / WHY IT MATTERS

Unlike primitive numeric variables, a String reference can be null. Calling a method on null throws NullPointerException. String itself is final and immutable: its contents cannot be changed in place. Java automatically provides convenient syntax with double quotes and +, but that convenience does not turn String into a primitive.

REAL-WORLD DECISION / EDGE CASE

Choose String for names, identifiers, messages, and other textual values. Do not use char for a general Unicode character when supplementary characters are possible.

COMMON PITFALL

Treating String as a numeric type for quantities leads to sorting and arithmetic surprises.

PRACTICE / CHANGE ONE THING

What happens after city = null; city.length()?

CHECK YOUR ANSWER

The method call throws NullPointerException.

String content versus reference identity

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use equals() to compare text content. == compares whether two references identify the same object.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b);
System.out.println(a.equals(b));
```

EXPECTED OUTPUT / RESULT

```
false
true
```

EXECUTION TRACE / READ THE STEPS

1. Each new String call creates a distinct object.
2. == compares object identity and returns false.
3. equals() checks characters and returns true.

DEEPER UNDERSTANDING / WHY IT MATTERS

String literals may reuse interned objects, which makes == appear to work in some examples. That is an implementation-backed optimization, not the right semantic test for business text. When a value might be null, use a known constant as the receiver: "ACTIVE".equals(status). For nullable variables use Objects.equals(a,b).

REAL-WORLD DECISION / EDGE CASE

equalsIgnoreCase() is useful in limited contexts, but business identifiers may require a clearly defined case-normalization policy.

COMMON PITFALL

Using == for user input or database text may pass tests with literals and fail later.

PRACTICE / CHANGE ONE THING

Will "Java" == "Java" often be true? Should you rely on it?

CHECK YOUR ANSWER

Both literal expressions refer to the interned literal; yes here, but do not use == for text content.

CHAPTER 4 / LESSON 03 OF 23

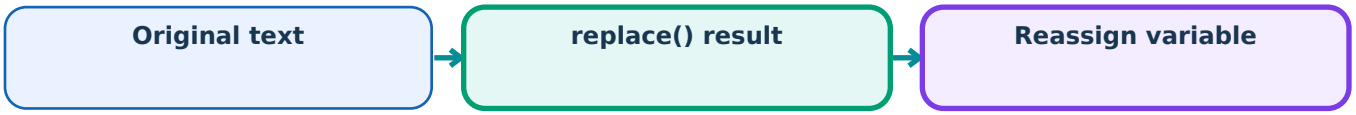
Immutability and reassignment

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

String methods never edit the existing String instance; they produce a new String result when a change is needed.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String text = "red";
text.replace("r", "b");
System.out.println(text);
text = text.replace("r", "b");
System.out.println(text);
```

EXPECTED OUTPUT / RESULT

```
red
bed
```

EXECUTION TRACE / READ THE STEPS

1. The first replace() computes a new text value and it is ignored.
2. The original immutable String remains red.
3. The second result is assigned back, so the variable refers to bed.

DEEPER UNDERSTANDING / WHY IT MATTERS

Immutability makes strings safe to share: a helper cannot unexpectedly edit the same String instance that another component holds. The variable itself is not immutable unless the variable is final. Rebinding a variable is different from mutating an object. For repeated string assembly use StringBuilder because repeatedly concatenating in a loop can create many temporary objects.

REAL-WORLD DECISION / EDGE CASE

String is an ideal value-type-like reference: avoid exposing internal mutable arrays when implementing classes that hold textual data.

COMMON PITFALL

Assuming .toUpperCase(), trim(), or replace() mutates the receiver.

PRACTICE / CHANGE ONE THING

What prints after String s=" a "; s.trim(); System.out.print(s.length()); ?

CHECK YOUR ANSWER

3: the result of trim() was discarded.

CHAPTER 4 / LESSON 04 OF 23

Indexing, length and Unicode text

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

String.length() counts UTF-16 code units, not necessarily human-visible characters or Unicode code points.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String emoji = "\uD83D\uDE80";
System.out.println(emoji.length());
System.out.println(emoji.codePointCount(0, emoji.length()));
```

EXPECTED OUTPUT / RESULT

```
2
1
```

EXECUTION TRACE / READ THE STEPS

1. The rocket emoji is represented by a surrogate pair.
2. length() returns the two UTF-16 code units.
3. codePointCount() reports one Unicode code point.

DEEPER UNDERSTANDING / WHY IT MATTERS

For ASCII text the three notions often coincide: code units, code points, and perceived characters. For emoji, accented sequences, and many languages they diverge. Even codePointCount does not count user-perceived grapheme clusters such as some emoji sequences. This matters when validating names or trimming text to UI limits. Java charAt() works with code units; codePoints() can be more appropriate for full Unicode code points.

REAL-WORLD DECISION / EDGE CASE

Do not reject all non-ASCII characters just to simplify validation. Use code-point-aware logic when the requirement is about Unicode characters.

COMMON PITFALL

A maximum length of 10 is ambiguous until you define UTF-16 units versus code points versus displayed characters.

PRACTICE / CHANGE ONE THING

What is the length of the single rocket emoji in Java String?

CHECK YOUR ANSWER

2 UTF-16 code units.

EXPANSION 01 / AFTER ORIGINAL LESSON 04

Unicode: characters, code units and code points

WHY THE DISTINCTION MATTERS

Java String uses UTF-16 code units. A visible character is not necessarily one char, and some user-perceived characters consist of multiple Unicode code points.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  String s = "A\uD83D\uDE00";
2  System.out.println(s.length());
3  System.out.println(s.codePointCount(0, s.length()));
4  System.out.println(s.substring(1).codePointAt(0));
```

EXPECTED OUTPUT

```
3
2
128512
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The letter A uses one UTF-16 code unit; the emoji is represented by a surrogate pair, so length() returns 3 rather than 2. codePointCount() returns two Unicode code points. charAt(1) would return only the high surrogate, not the entire emoji. Code-point iteration is safer when an algorithm processes Unicode characters rather than UTF-16 units.

PRODUCTION RULE OR FAILURE MODE

Even code points are not identical to user-perceived grapheme clusters (for example, letters with combining accents or joined emoji). For cursor movement and localized text selection, use character-break algorithms rather than assuming one code point per visual character.

TRY THIS, THEN REVEAL THE REASON

Challenge: If a string contains a letter followed by one emoji, how many results would chars().count() and codePoints().count() return?
Reasoned answer: 3 and 2, respectively: chars() iterates UTF-16 code units, whereas codePoints() recognizes the surrogate pair.

CHAPTER 4 / LESSON 05 OF 23

substring boundaries and off-by-one rules

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

substring(start,end) includes start but excludes end. Index positions start at zero.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String code = "ABCD1234";
System.out.println(code.substring(0, 4));
System.out.println(code.substring(4));
```

EXPECTED OUTPUT / RESULT

```
ABCD
1234
```

EXECUTION TRACE / READ THE STEPS

1. Positions 0 through 3 form the first result.
2. The end index 4 is excluded.
3. substring(4) takes text from position 4 to the end.

DEEPER UNDERSTANDING / WHY IT MATTERS

The exclusive end boundary aligns with array/list slice ideas and makes length equal to end-start. Invalid indexes throw StringIndexOutOfBoundsException. Never assume external IDs have a fixed length unless the contract guarantees it. For variable-length fields prefer delimiter parsing with explicitly checked indexes. Note that substring does not mutate its source.

REAL-WORLD DECISION / EDGE CASE

When extracting data from a URL, filename or account ID, validate boundaries before slicing.

COMMON PITFALL

Treating end as inclusive drops or adds a character in production mapping code.

PRACTICE / CHANGE ONE THING

What does "123456".substring(2,5) produce?

CHECK YOUR ANSWER

345.

Text cleanup: strip, trim and replace

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Prefer strip() to remove Unicode whitespace, and replace() for literal text replacement; know when regex is involved.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String raw = "  Chicago  ";
System.out.println(raw.strip());
System.out.println(raw.replace("Chicago", "IL"));
```

EXPECTED OUTPUT / RESULT

```
Chicago
  IL
```

EXECUTION TRACE / READ THE STEPS

1. strip() returns a new String with surrounding Unicode whitespace removed.
2. replace(CharSequence,CharSequence) replaces literal content.
3. raw remains unchanged for both operations.

DEEPER UNDERSTANDING / WHY IT MATTERS

trim() is older and removes only characters up to U+0020 at both ends; strip() uses Character.isWhitespace and handles more Unicode whitespace, though not every visually blank character. replaceAll() accepts a regular expression, not a literal token. Validation should distinguish normalization from silently changing an identifier. Reject unexpected text rather than over-correcting business keys.

REAL-WORLD DECISION / EDGE CASE

Use a documented normalization policy for inbound names and codes; keep original values when auditing requires exact input.

COMMON PITFALL

replaceAll(".", "") removes every character because dot is a regex wildcard.

PRACTICE / CHANGE ONE THING

What happens when raw.replace("Chicago","IL") is called without assignment?

CHECK YOUR ANSWER

A new value is produced but raw still contains the original string.

Splitting, joining and regex hazards

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

String.split() takes a regular expression; Pattern.quote protects literal delimiters that contain regex metacharacters.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String input = "A|B|C";
String[] parts = input.split("\\|");
System.out.println(parts.length);
System.out.println(String.join("-", parts));
```

EXPECTED OUTPUT / RESULT

```
3
A-B-C
```

EXECUTION TRACE / READ THE STEPS

1. A pipe has special meaning in regular expressions.
2. The regex escaped pipe matches the literal | delimiter.
3. join() combines the resulting values using a hyphen.

DEEPER UNDERSTANDING / WHY IT MATTERS

In Java source, a backslash is also escaped, so the regex literal pipe needs two backslashes in the Java source string: "\\|". A simpler general solution is input.split(Pattern.quote(delimiter)). split() normally drops trailing empty fields: "A,".split(",") produces one element. Use split(",", -1) if the trailing empty value is significant.

REAL-WORLD DECISION / EDGE CASE

For CSV with quoted commas, use a CSV parser rather than a naive split.

COMMON PITFALL

Using split(".") does not split on a literal period.

PRACTICE / CHANGE ONE THING

How do you preserve trailing empty fields when splitting A,?

CHECK YOUR ANSWER

Call split(",", -1).

EXPANSION 02 / AFTER ORIGINAL LESSON 07

Text at boundaries: normalize, split and validate

WHY THE DISTINCTION MATTERS

Input often arrives with extra whitespace, arbitrary capitalization, and separators that are regex metacharacters. Treat parsing as a boundary concern rather than scattering string fixes across business logic.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  String raw = "  a|b|c  ";
2  String normalized = raw.strip();
3  String[] parts = normalized.split("\\|");
4  System.out.println(parts.length);
5  System.out.println(parts[1]);
```

EXPECTED OUTPUT

```
3
b
```

UNDER THE HOOD / FOLLOW THE EXECUTION

strip() removes Unicode whitespace according to its definition, whereas trim() only handles characters at or below U+0020. split() takes a regex, not an ordinary literal separator; the vertical bar means alternation in regex and therefore must be escaped. The Java string literal itself also processes backslashes, creating two levels of escaping. In production, Pattern.quote(delimiter) makes a dynamic delimiter safe.

PRODUCTION RULE OR FAILURE MODE

String.split(regex) discards trailing empty fields by default. For a fixed-column import, use split(regex, -1) to retain trailing empty fields and validate the expected field count before converting values.

TRY THIS, THEN REVEAL THE REASON

Challenge: What is the difference between "A,B,".split(",").length and "A,B,".split(",", -1).length?
Reasoned answer: 2 versus 3. The default split drops trailing empty strings; negative limit keeps them.

StringBuilder and efficient assembly

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use StringBuilder for repeated changes to a text buffer; StringBuilder is mutable and not thread-safe.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
StringBuilder builder = new StringBuilder();
builder.append("Order-");
builder.append(42);
System.out.println(builder.toString());
```

EXPECTED OUTPUT / RESULT

```
Order-42
```

EXECUTION TRACE / READ THE STEPS

1. A mutable builder begins empty.
2. append() adds text and the numeric value to the same builder.
3. toString() provides an immutable String snapshot.

DEEPER UNDERSTANDING / WHY IT MATTERS

Ordinary + concatenation is clear and efficient for a few pieces; the Java compiler may optimize it. In a repeated loop or complex assembly, StringBuilder makes the mutation intentional and usually avoids unnecessary intermediate strings. StringBuffer is synchronized and is rarely needed in new ordinary application code. Never share one StringBuilder across threads without coordination.

REAL-WORLD DECISION / EDGE CASE

Do not use StringBuilder as a substitute for parameterized SQL queries or safe HTML escaping.

COMMON PITFALL

Repeated String concat inside a very large loop can allocate many intermediate values.

PRACTICE / CHANGE ONE THING

Does builder.append("!") change a previously returned String snapshot?

CHECK YOUR ANSWER

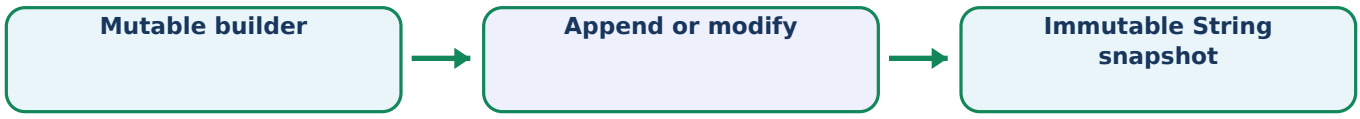
No. The String snapshot remains unchanged.

EXPANSION 03 / AFTER ORIGINAL LESSON 08

StringBuilder: mutation, snapshot and iteration cost

WHY THE DISTINCTION MATTERS

Text concatenation with + is clear for a few values. A builder is useful when assembling a variable amount of text in a loop or producing a structured message incrementally.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  StringBuilder sb = new StringBuilder("A");
2  String snapshot = sb.toString();
3  sb.append("B");
4  System.out.println(snapshot);
5  System.out.println(sb);
6  sb.deleteCharAt(0);
7  System.out.println(sb);
```

EXPECTED OUTPUT

```
A
AB
B
```

UNDER THE HOOD / FOLLOW THE EXECUTION

A StringBuilder is mutable: append, insert, replace and delete alter its contents. The String returned by toString() does not change after the builder is subsequently mutated. Avoid repeated result = result + item within large loops when a builder communicates intent and can avoid repeated copying. The compiler can optimize simple concatenation expressions, so do not replace straightforward + usage everywhere.

PRODUCTION RULE OR FAILURE MODE

StringBuilder is not designed for synchronized shared mutation. Prefer one builder owned by one task; share the immutable String once construction is complete. StringBuffer is synchronized but is rarely the right default for modern code.

TRY THIS, THEN REVEAL THE REASON

Challenge: After saving a String snapshot, can a later append() modify that original String?
Reasoned answer: No. The builder changes, but an already-created String remains immutable.

CHAPTER 4 / LESSON 09 OF 23

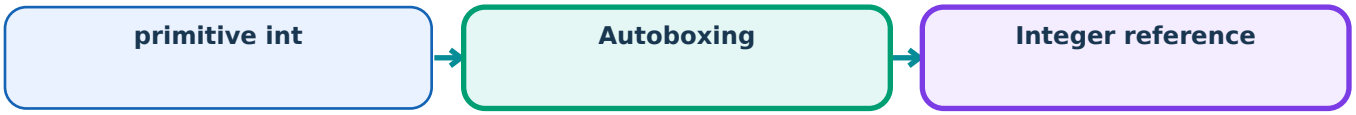
Wrapper types and autoboxing

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Wrapper classes such as Integer and Double allow primitive-like values in generic containers, but they remain references.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
int count = 4;
Integer boxed = count;
int again = boxed;
System.out.println(again + 1);
```

EXPECTED OUTPUT / RESULT

```
5
```

EXECUTION TRACE / READ THE STEPS

1. Java automatically boxes int 4 to an Integer value.
2. Unboxing retrieves the primitive int.
3. Arithmetic uses the primitive and prints 5.

DEEPER UNDERSTANDING / WHY IT MATTERS

Generics operate on reference types, so List<Integer> is allowed and List<int> is not. Boxing and unboxing are convenient but can add allocations, null risk, and overhead in tight numeric loops. BigDecimal is not a wrapper for double; it is a decimal arithmetic type with its own scale and rounding rules.

REAL-WORLD DECISION / EDGE CASE

Choose primitives for required numeric fields with no missing state; wrappers are useful for nullable fields and generic APIs.

COMMON PITFALL

Accidentally treating an Integer as guaranteed non-null.

PRACTICE / CHANGE ONE THING

Can List<int> compile?

CHECK YOUR ANSWER

No; List<Integer> is the correct generic form.

CHAPTER 4 / LESSON 10 OF 23

Null unboxing and safe defaults

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Unboxing a null wrapper throws NullPointerException even when the target variable is primitive.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Integer count = null;
System.out.println(count == null);
int safe = count == null ? 0 : count;
System.out.println(safe);
```

EXPECTED OUTPUT / RESULT

```
true
0
```

EXECUTION TRACE / READ THE STEPS

1. The wrapper variable contains null.
2. The ternary checks null before selecting a branch.
3. Only a non-null wrapper would be unboxed in the second branch.

DEEPER UNDERSTANDING / WHY IT MATTERS

The dangerous pattern is int n = count; with a nullable Integer. A null check on the reference must happen before arithmetic, relational operators, or primitive assignment that triggers unboxing. When zero is not a valid stand-in for missing data, keep the missing state explicit rather than silently defaulting. APIs should make their nullability contract clear.

REAL-WORLD DECISION / EDGE CASE

Prefer database and JSON mapping contracts that distinguish missing, empty, and zero when business meaning differs.

COMMON PITFALL

An innocent-looking count > 0 can throw NPE if count is Integer and null.

PRACTICE / CHANGE ONE THING

What happens with Integer x=null; System.out.println(x+1);?

CHECK YOUR ANSWER

NullPointerException during unboxing.

CHAPTER 4 / LESSON 11 OF 23

Wrapper equality and cached instances

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use equals() for wrapper value equality after handling null; == tests object identity.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Integer a = Integer.valueOf(200);
Integer b = Integer.valueOf(200);
System.out.println(a.equals(b));
System.out.println(a.intValue() == b.intValue());
```

EXPECTED OUTPUT / RESULT

```
true
true
```

EXECUTION TRACE / READ THE STEPS

1. Integer.valueOf creates or returns an Integer with value 200.
2. equals() compares contained integer values.
3. Unboxing compares primitive ints, both 200.

DEEPER UNDERSTANDING / WHY IT MATTERS

Integer caching guarantees reuse for at least -128 through 127; outside that range identity is not guaranteed. Depending on caching and JVM configuration, == between wrapper objects may seem consistent in one environment and differ in another. Comparison operators such as > trigger unboxing, which has null risks. Explicit equals and primitive comparisons express intent better.

REAL-WORLD DECISION / EDGE CASE

For numeric business values, decide whether equality is numeric, textual, or decimal-scaled; they are not always interchangeable.

COMMON PITFALL

Expecting Integer a=100; Integer b=100; a==b behavior to generalize to 1000.

PRACTICE / CHANGE ONE THING

Should two Integer objects with value 200 be compared by ==?

CHECK YOUR ANSWER

No; use equals() or compare their primitive values.

EXPANSION 04 / AFTER ORIGINAL LESSON 11

Wrapper equality, unboxing and null-safe comparisons

WHY THE DISTINCTION MATTERS

Wrapper objects introduce an object identity distinction and a null state that primitives do not have. Small cached integers sometimes make a buggy == comparison appear to work.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  Integer a = Integer.valueOf(200);
2  Integer b = Integer.valueOf(200);
3  System.out.println(a.equals(b));
4  System.out.println(a.intValue() == b.intValue());
5  Integer missing = null;
6  System.out.println(java.util.Objects.equals(missing, b));
```

EXPECTED OUTPUT

```
true
true
false
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The two Integer.valueOf(200) calls produce references representing the same numeric value; a portable demonstration compares numeric wrappers by equals() or unboxed values, never by reference identity. The default Integer cache covers -128 through 127, but relying on identity behavior for any value is incorrect. Objects.equals(a,b) handles null references safely. Automatic unboxing to an int can throw NullPointerException when the wrapper contains null.

PRODUCTION RULE OR FAILURE MODE

For nullable input, make the policy explicit: reject null, preserve it, or assign a documented default. Calling map.get(key) + 1 can fail if the map returns a null Integer; getOrDefault also does not replace an explicit null mapping.

TRY THIS, THEN REVEAL THE REASON

Challenge: Why can Integer x = null; int y = x; compile and then fail at runtime?
Reasoned answer: The compiler inserts unboxing. Java attempts to call intValue() on null, throwing NullPointerException.

CHAPTER 4 / LESSON 12 OF 23

Parsing numbers at input boundaries

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Parsing turns text into typed values; invalid input throws exceptions that should be handled at the boundary.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
String input = "42";
int amount = Integer.parseInt(input);
System.out.println(amount + 8);
```

EXPECTED OUTPUT / RESULT

```
50
```

EXECUTION TRACE / READ THE STEPS

1. The input is a String containing digits.
2. parseInt validates and converts to int.
3. Normal integer addition produces 50.

DEEPER UNDERSTANDING / WHY IT MATTERS

Integer.parseInt throws NumberFormatException for non-numeric or out-of-range input. You should decide whether leading and trailing whitespace are allowed: trim or strip explicitly if so. Integer.parseInt accepts a leading sign but not decimal points. For monetary values, parse BigDecimal directly from the decimal string to avoid double conversion.

REAL-WORLD DECISION / EDGE CASE

Boundary code should produce useful validation messages rather than exposing raw parsing exceptions to end users.

COMMON PITFALL

Applying + to two strings concatenates; it does not parse and add their values.

PRACTICE / CHANGE ONE THING

What does Integer.parseInt("12.0") do?

CHECK YOUR ANSWER

It throws NumberFormatException.

EXPANSION 05 / AFTER ORIGINAL LESSON 12

Parsing numeric input: validation and overflow

WHY THE DISTINCTION MATTERS

User input is text. Treat invalid format, out-of-range values and null input as separate decisions instead of assuming parseInt will always succeed.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  for (String text : new String[]{"42", "4x"})
2  {
3      try
4      {
5          System.out.println(Integer.parseInt(text));
6      }
7      catch (NumberFormatException ex)
8      {
9          System.out.println("Invalid integer");
10     }
11 }
```

EXPECTED OUTPUT

```
42
Invalid integer
```

UNDER THE HOOD / FOLLOW THE EXECUTION

Integer.parseInt() rejects non-integer characters and values outside the int range with NumberFormatException. Long.parseLong() permits a wider range but is not unlimited. When domain constraints are narrower than the Java type, validate them after parsing. Monetary input should generally be parsed straight into BigDecimal from decimal text, not first converted to double.

PRODUCTION RULE OR FAILURE MODE

Validation belongs at input boundaries. Use a specific error message for the user but retain enough context for diagnosis; avoid logging sensitive raw input indiscriminately.

TRY THIS, THEN REVEAL THE REASON

Challenge: Would Integer.parseInt("2147483648") succeed?
Reasoned answer: No. That is one more than Integer.MAX_VALUE (2,147,483,647), so it throws NumberFormatException.

CHAPTER 4 / LESSON 13 OF 23

Enums model closed sets of states

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

An enum provides named singleton constants for a known set of valid statuses; it is safer than free-form strings.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
enum Status
{
    NEW, ACTIVE, CLOSED
}
Status state = Status.ACTIVE;
System.out.println(state == Status.ACTIVE);
```

EXPECTED OUTPUT / RESULT

```
true
```

EXECUTION TRACE / READ THE STEPS

1. Java declares three enum constants.
2. state refers to the unique ACTIVE enum instance.
3. == is appropriate for enum identity equality.

DEEPER UNDERSTANDING / WHY IT MATTERS

Enums can contain fields, constructors, and methods, and implement interfaces. They can work in switch expressions to keep all known statuses explicit. Persisting enum.name() directly creates a coupling to source names; if changing a name would break external data, map stable external codes instead. Avoid ordinal() as a persisted value because reordering constants changes the number.

REAL-WORLD DECISION / EDGE CASE

Use enums for bounded states such as NEW, RUNNING, FAILED, not for user-extensible lists of categories.

COMMON PITFALL

Persisting ordinal numbers for durable business status values.

PRACTICE / CHANGE ONE THING

Can enum variables be null?

CHECK YOUR ANSWER

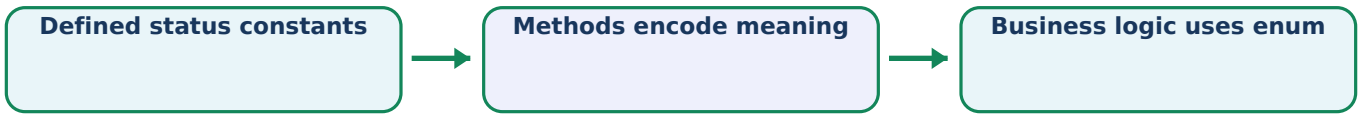
Yes. Enum type does not prevent a reference from being null.

EXPANSION 06 / AFTER ORIGINAL LESSON 13

Enums with fields, methods and controlled transitions

WHY THE DISTINCTION MATTERS

Enums are not merely numbered constants. They are real Java classes with a fixed set of instances, constructors, fields and methods, suitable for validated business statuses.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  enum State
2  {
3      NEW("Open"), SHIPPED("In transit");
4      private final String label;
5      State(String label) { this.label = label; }
6      String label() { return label; }
7  }
8  System.out.println(State.SHIPPED.label());
```

EXPECTED OUTPUT

```
In transit
```

UNDER THE HOOD / FOLLOW THE EXECUTION

An enum constructor is called when its fixed constants are initialized. It cannot be called with new State(...) from outside. Enum fields should usually be final when representing fixed metadata. Compared with free-form strings, enums make invalid statuses harder to introduce and allow exhaustive handling in switch expressions for well-defined cases.

PRODUCTION RULE OR FAILURE MODE

The declaration above is shown as a class-level enum followed by a line that runs in main(). In a real program, separate the enum declaration from the caller. Persisting enum ordinal() is fragile because reordering constants changes numbers; prefer stable string codes for external storage.

TRY THIS, THEN REVEAL THE REASON

Challenge: What happens if you rename an enum constant used as its serialized database value?
Reasoned answer: Existing stored values may no longer deserialize. Use explicit, stable external codes when a persistent contract matters.

CHAPTER 4 / LESSON 14 OF 23

Switch expressions over enums

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Switch expressions return a value and can enforce exhaustive handling of known enum constants.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
enum State
{
    OPEN, CLOSED
}
State s = State.OPEN;
String label = switch (s)
{
    case OPEN -> "Ready";
    case CLOSED -> "Done";
};
System.out.println(label);
```

EXPECTED OUTPUT / RESULT

```
Ready
```

EXECUTION TRACE / READ THE STEPS

1. The selector is the OPEN enum constant.
2. The OPEN arrow arm is selected.
3. The entire switch expression evaluates to Ready.

DEEPER UNDERSTANDING / WHY IT MATTERS

Modern switch expressions use -> to avoid accidental fall-through. A switch expression over all enum constants can be exhaustive without default; adding a new enum constant then forces a compile-time review when the code is recompiled. Avoid a default branch if you want the compiler to remind you about new states; add one when external compatibility requires it.

REAL-WORLD DECISION / EDGE CASE

Switch expressions are especially useful for readable mapping from status to label, priority, or allowed operation.

COMMON PITFALL

Overusing default can hide new enum constants that should receive explicit business handling.

PRACTICE / CHANGE ONE THING

What happens after a new enum constant is added but not handled in an otherwise exhaustive switch? Answer: The switch expression must be updated to compile against the expanded enum.

BigDecimal: decimal money versus floating-point

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use BigDecimal created from a decimal string or valueOf(double) for predictable decimal values; double is binary floating point.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
BigDecimal x = new BigDecimal("0.1");
BigDecimal y = new BigDecimal("0.2");
System.out.println(x.add(y));
```

EXPECTED OUTPUT / RESULT

```
0.3
```

EXECUTION TRACE / READ THE STEPS

1. Each BigDecimal is constructed from an exact decimal string.
2. add() performs arbitrary-precision decimal arithmetic.
3. The result is the exact decimal 0.3.

DEEPER UNDERSTANDING / WHY IT MATTERS

The expression 0.1 + 0.2 using double is not exactly 0.3. new BigDecimal(0.1) captures the exact binary floating-point approximation, producing a long decimal representation. BigDecimal.valueOf(0.1) uses Double.toString and is often suitable when a double has already entered the system, but direct decimal text remains preferable at input boundaries.

REAL-WORLD DECISION / EDGE CASE

Store monetary amounts with an agreed scale and currency. BigDecimal does not automatically know the currency, tax rule, or allowed precision.

COMMON PITFALL

new BigDecimal(0.1) instead of new BigDecimal("0.1").

PRACTICE / CHANGE ONE THING

Should you convert a price string to double before BigDecimal?

CHECK YOUR ANSWER

No. Construct BigDecimal directly from the decimal text.

CHAPTER 4 / LESSON 16 OF 23

BigDecimal scale and rounding contracts

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Division can be non-terminating; choose scale and RoundingMode explicitly when the result requires rounding.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
BigDecimal one = new BigDecimal("1");
BigDecimal three = new BigDecimal("3");
System.out.println(one.divide(three, 2, RoundingMode.HALF_UP));
```

EXPECTED OUTPUT / RESULT

```
0.33
```

EXECUTION TRACE / READ THE STEPS

1. The exact value 1/3 has an infinite decimal expansion.
2. The requested scale is two decimal places.
3. HALF_UP rounds the value to 0.33.

DEEPER UNDERSTANDING / WHY IT MATTERS

divide(divisor) without a rounding context throws ArithmeticException if the quotient cannot be represented exactly as a finite decimal. HALF_UP rounds ties away from zero in magnitude; HALF_EVEN reduces cumulative bias for certain repeated rounding calculations. Round at documented business boundaries rather than after every intermediate operation, otherwise small errors compound. Rounding to two places is not universally correct for every currency or rate.

REAL-WORLD DECISION / EDGE CASE

Make rounding a domain decision, not a hidden default in a utility method.

COMMON PITFALL

Calling one.divide(three) without specifying rounding produces ArithmeticException.

PRACTICE / CHANGE ONE THING

What does new BigDecimal("1.005").setScale(2, HALF_UP) produce?

CHECK YOUR ANSWER

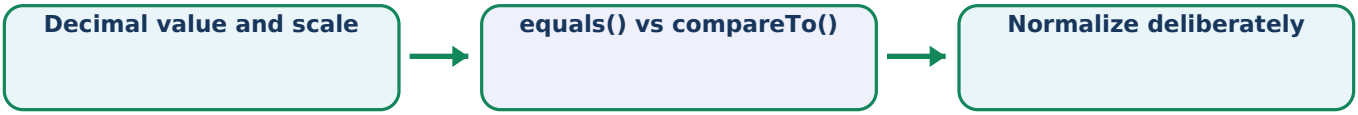
1.01.

EXPANSION 07 / AFTER ORIGINAL LESSON 16

BigDecimal: scale, equality and numeric comparison

WHY THE DISTINCTION MATTERS

Two decimal values can represent the same number but have different scales. That distinction affects equality, HashSet keys, display formatting and financial rules.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  BigDecimal a = new BigDecimal("2.0");
2  BigDecimal b = new BigDecimal("2.00");
3  System.out.println(a.equals(b));
4  System.out.println(a.compareTo(b) == 0);
5  System.out.println(a.scale() + "," + b.scale());
```

EXPECTED OUTPUT

```
false
true
1,2
```

UNDER THE HOOD / FOLLOW THE EXECUTION

equals() considers numeric value and scale, while compareTo() tests numeric ordering and returns zero for equal numerical values. HashSet follows equals/hashCode and therefore may retain 2.0 and 2.00 as separate keys; TreeSet commonly follows compareTo and may consider them duplicates. This can produce surprising differences between hashed and sorted collections.

PRODUCTION RULE OR FAILURE MODE

Decide whether scale is part of the business meaning. Normalize at boundaries when appropriate, but be careful: stripTrailingZeros() can produce a negative scale for values such as 1000, which changes string representation.

TRY THIS, THEN REVEAL THE REASON

Challenge: Should monetary equality checks automatically use equals()?
Reasoned answer: Not always. For numeric equivalence despite scale differences, compareTo()==0 may be the intended rule; domain-specific currency and scale constraints still matter.

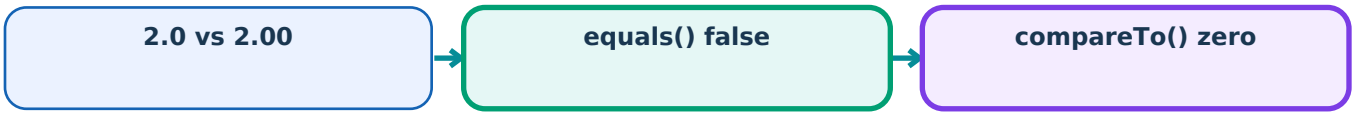
BigDecimal equals, compareTo and collection behavior

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

BigDecimal.equals checks value and scale; compareTo compares numeric value. They may disagree for 2.0 and 2.00.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
BigDecimal a = new BigDecimal("2.0");
BigDecimal b = new BigDecimal("2.00");
System.out.println(a.equals(b));
System.out.println(a.compareTo(b));
```

EXPECTED OUTPUT / RESULT

```
false
0
```

EXECUTION TRACE / READ THE STEPS

1. The first decimal has scale one.
2. The second has scale two.
3. equals() distinguishes scales; compareTo treats the amounts as numerically equal.

DEEPER UNDERSTANDING / WHY IT MATTERS

This difference affects collections: HashSet uses equals()/hashCode(), while TreeSet uses its comparator or natural ordering (compareTo). Therefore a HashSet could retain both values while a naturally ordered TreeSet sees them as equivalent. For monetary comparisons, compareTo()==0 often expresses numeric equality, but choose a consistent scale policy if values are map keys or persisted in equality-sensitive structures.

REAL-WORLD DECISION / EDGE CASE

Do not use BigDecimal as a hash key without understanding whether scale is part of identity.

COMMON PITFALL

Using equals() when the business requirement is numeric equality independent of scale.

PRACTICE / CHANGE ONE THING

Would HashSet and TreeSet necessarily treat 2.0 and 2.00 identically?

CHECK YOUR ANSWER

No; they can differ because equals and compareTo differ.

EXPANSION 08 / AFTER ORIGINAL LESSON 17

Money allocation: rounding once and distributing leftovers

WHY THE DISTINCTION MATTERS

Division of a monetary total among several recipients cannot always be exact at the currency scale. Avoid creating or losing cents through repeated independent rounding.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```java
int totalCents = 100;
int people = 3;
int base = totalCents / people;
int extra = totalCents % people;
for (int i = 0; i < people; i++)
{
    System.out.print((base + (i < extra ? 1 : 0)) + " ");
}
```

EXPECTED OUTPUT

```
34 33 33
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The allocation computes a common integer number of cents per recipient and distributes the remainder one cent at a time. The three shares add back to 100, preserving the invariant that allocated money must equal the original amount. A BigDecimal representation is still appropriate for many financial calculations, but integer minor units can simplify allocation for a known currency scale.

PRODUCTION RULE OR FAILURE MODE

Different currencies use different minor-unit conventions; taxes and interest calculations may require greater precision before rounding. Store the rounding and allocation policies with the business rules, not hidden inside an arithmetic helper.

TRY THIS, THEN REVEAL THE REASON

Challenge: If 101 cents are split among three recipients, what are the allocations using this policy?
Reasoned answer: 34, 34 and 33 cents. The allocations add to 101 without loss.

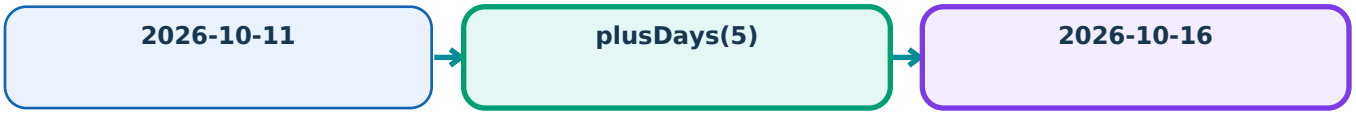
LocalDate for calendar dates

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

LocalDate represents a calendar date with no clock time and no time zone.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
LocalDate date = LocalDate.of(2026, 10, 11);
System.out.println(date.plusDays(5));
```

EXPECTED OUTPUT / RESULT

```
2026-10-16
```

EXECUTION TRACE / READ THE STEPS

1. The date stores year, month, and day.
2. plusDays(5) computes a new calendar date.
3. The original LocalDate remains unchanged.

DEEPER UNDERSTANDING / WHY IT MATTERS

Use LocalDate for birthdays, due dates, and business dates that are calendar-based rather than instants. A LocalDate does not answer what absolute moment midnight represents because no zone is present. java.time classes such as LocalDate are immutable, making them easier to share across layers. Do not represent date-only business information as a midnight Instant unless a specific time zone conversion is intended.

REAL-WORLD DECISION / EDGE CASE

DB DATE and Java LocalDate often correspond, but confirm database drivers and application mapping rules.

COMMON PITFALL

Adding 24 hours to an Instant is not always the same as moving to the next local calendar date across DST.

PRACTICE / CHANGE ONE THING

What does LocalDate.of(2024,2,28).plusDays(1) return?

CHECK YOUR ANSWER

2024-02-29, because 2024 is a leap year.

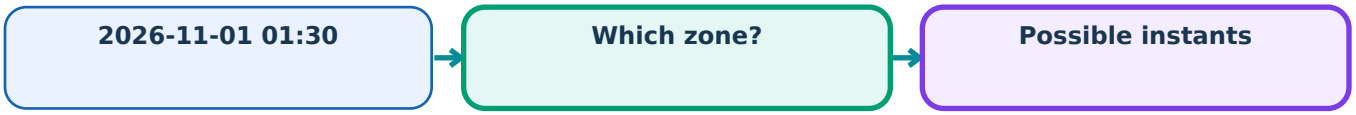
LocalDateTime is not an absolute instant

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

LocalDateTime is a wall-clock date and time without a time zone or offset; it cannot unambiguously identify an instant.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
LocalDateTime slot = LocalDateTime.of(2026, 11, 1, 1, 30);
System.out.println(slot.getHour());
System.out.println(slot.toLocalDate());
```

EXPECTED OUTPUT / RESULT

```
1
2026-11-01
```

EXECUTION TRACE / READ THE STEPS

1. The wall-clock fields are created directly.
2. getHour() reads the hour field.
3. toLocalDate() extracts just the calendar date.

DEEPER UNDERSTANDING / WHY IT MATTERS

This is useful for schedules described by local clocks, but some wall-clock times repeat or disappear during daylight-saving transitions. Converting to Instant requires explicit zone or offset rules. Avoid mixing UTC instants, local date-times, and strings in a domain model without stating what each field means. Storing a LocalDateTime without a zone for an audit event can leave an ambiguous timestamp.

REAL-WORLD DECISION / EDGE CASE

A branch office appointment may need LocalDateTime plus ZoneId; an audit creation time should usually be an Instant.

COMMON PITFALL

Assuming LocalDateTime.now() and Instant.now() are interchangeable values.

PRACTICE / CHANGE ONE THING

Can LocalDateTime identify an unambiguous absolute moment on its own?

CHECK YOUR ANSWER

No.

CHAPTER 4 / LESSON 20 OF 23

Instant and UTC timeline values

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Instant represents an absolute point on the timeline; it is ideal for event timestamps and ordering.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Instant event = Instant.parse("2026-10-11T12:00:00Z");
System.out.println(event);
```

EXPECTED OUTPUT / RESULT

```
2026-10-11T12:00:00Z
```

EXECUTION TRACE / READ THE STEPS

1. The ISO-8601 string includes Z, indicating UTC offset zero.
2. Instant.parse() creates a timeline value.
3. Printing retains the UTC timestamp representation.

DEEPER UNDERSTANDING / WHY IT MATTERS

Different users can display the same Instant in different local zones while referring to one exact event moment. java.time Instant stores epoch-based seconds and nanoseconds. It is not a date-only value, and it should not be formatted as local wall-clock time without an explicit ZoneId. For testable current time, inject a Clock rather than calling now() throughout business logic.

REAL-WORLD DECISION / EDGE CASE

Log creation timestamps and persisted event times are good Instant candidates.

COMMON PITFALL

Treating 2026-10-11T12:00 in Chicago as if it meant 12:00 UTC.

PRACTICE / CHANGE ONE THING

Does changing the viewer’s time zone change the Instant?

CHECK YOUR ANSWER

No. Only its local representation changes.

EXPANSION 09 / AFTER ORIGINAL LESSON 20

LocalDateTime is not an instant: converting explicitly

WHY THE DISTINCTION MATTERS

A local clock value is ambiguous without a time zone. Translating it into a timestamp requires a zone or offset; choosing the wrong zone changes the moment recorded.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  LocalDateTime local = LocalDateTime.of(2026, 1, 1, 12, 0);
2  ZoneId chicago = ZoneId.of("America/Chicago");
3  Instant instant = local.atZone(chicago).toInstant();
4  System.out.println(instant);
```

EXPECTED OUTPUT

```
2026-01-01T18:00:00Z
```

UNDER THE HOOD / FOLLOW THE EXECUTION

Chicago is on standard time in January, UTC-6, so noon locally becomes 18:00 UTC. An Instant uniquely identifies a point on the timeline, while LocalDate and LocalDateTime express human calendar concepts without a zone. Use the type that matches the meaning: birth date as LocalDate, an audit event as Instant, and an appointment rule tied to a location as a local time plus ZoneId.

PRODUCTION RULE OR FAILURE MODE

Never assume server-default time zone is the user or event time zone. Accept an explicit ZoneId and document the behavior for ambiguous or nonexistent local times.

TRY THIS, THEN REVEAL THE REASON

Challenge: Would the exact same LocalDateTime represent the same instant in Tokyo and Chicago?
Reasoned answer: No. The wall-clock components are equal, but the zone offsets differ.

Time zones and daylight-saving behavior

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

ZonedDateTime pairs date-time fields with a zone’s historical and future rules; some local times are skipped or repeated.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
ZoneId zone = ZoneId.of("America/Chicago");
Instant instant = Instant.parse("2026-01-01T12:00:00Z");
System.out.println(instant.atZone(zone).getHour());
```

EXPECTED OUTPUT / RESULT

6

EXECUTION TRACE / READ THE STEPS

1. A January UTC Instant is selected.
2. America/Chicago uses UTC-06:00 on that date.
3. The local hour is 6 AM.

DEEPER UNDERSTANDING / WHY IT MATTERS

Use named region zones such as America/Chicago rather than a hard-coded offset like -06:00 when scheduling across seasons. Offset is a numerical displacement; ZoneId includes daylight-saving and historical rules. During spring transitions some wall-clock times do not exist; during fall transitions some occur twice. APIs resolving overlaps/gaps have defined policies, which must match the application’s business intent.

REAL-WORLD DECISION / EDGE CASE

Financial cutoffs and recurring meetings should use explicit business-zone rules, not the server default zone.

COMMON PITFALL

Converting using a fixed -06:00 offset in July when Chicago is on daylight time (-05:00).

PRACTICE / CHANGE ONE THING

Is ZoneOffset.ofHours(-6) equivalent to America/Chicago year-round?

CHECK YOUR ANSWER

No.

EXPANSION 10 / AFTER ORIGINAL LESSON 21

DST gaps and overlaps: local clocks are not continuous

WHY THE DISTINCTION MATTERS

Some zones skip an hour in spring and repeat an hour in autumn. A local date-time may therefore map to zero, one, or two valid offsets.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  ZoneId zone = ZoneId.of("America/New_York");
2  LocalDateTime gap = LocalDateTime.of(2026, 3, 8, 2, 30);
3  LocalDateTime overlap = LocalDateTime.of(2026, 11, 1, 1, 30);
4  System.out.println(zone.getRules().getValidOffsets(gap).size());
5  System.out.println(zone.getRules().getValidOffsets(overlap).size());
```

EXPECTED OUTPUT

```
0
2
```

UNDER THE HOOD / FOLLOW THE EXECUTION

At the spring-forward transition the 02:30 local time does not exist, so the list of valid offsets is empty. During the fall-back transition, 01:30 occurs twice and has two valid offsets. atZone() applies default resolution rules; when scheduling high-stakes events, the application may need to ask the user to resolve ambiguity or define a consistent business policy.

PRODUCTION RULE OR FAILURE MODE

A calendar recurrence such as “9 a.m. every day” should normally be modeled by LocalTime and ZoneId, not by adding a fixed 24-hour Duration to yesterday’s Instant. Check whether an operation is calendar-based or duration-based.

TRY THIS, THEN REVEAL THE REASON

Challenge: What should happen to a requested appointment at 02:30 during a spring DST gap?
Reasoned answer: There is no universal answer. The application must define whether to reject, move forward, or choose another time.

CHAPTER 4 / LESSON 22 OF 23

Period, Duration and temporal arithmetic

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Period models calendar units (years, months, days); Duration models elapsed time-based units.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Instant start = Instant.parse("2026-01-01T00:00:00Z");
Instant end = start.plusSeconds(90);
System.out.println(Duration.between(start, end).getSeconds());
```

EXPECTED OUTPUT / RESULT

```
90
```

EXECUTION TRACE / READ THE STEPS

1. Two fixed instants identify a 90-second interval.
2. Duration.between calculates timeline elapsed time.
3. getSeconds() prints 90.

DEEPER UNDERSTANDING / WHY IT MATTERS

A period of one day on the local calendar and a duration of 24 hours are not always interchangeable near daylight saving. Period is useful for age, billing-cycle dates, and calendar arithmetic. Duration is useful for timeouts, processing latency and elapsed intervals. LocalDate.plusMonths applies calendar rules, including end-of-month adjustment.

REAL-WORLD DECISION / EDGE CASE

Choose elapsed time for SLA timers and calendar periods for anniversaries or business cycle boundaries.

COMMON PITFALL

Treating plusDays(1) on a zoned date as exactly 24 hours across DST.

PRACTICE / CHANGE ONE THING

Should a timeout of 30 seconds use Period or Duration?

CHECK YOUR ANSWER

Duration.

CHAPTER 4 / LESSON 23 OF 23

DateTimeFormatter, locale and stable output

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Parse and format with explicit patterns and locale/zone when required; do not depend on a machine’s default settings.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
LocalDate d = LocalDate.of(2026, 10, 11);
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("MM/dd/uuuu");
System.out.println(d.format(fmt));
```

EXPECTED OUTPUT / RESULT

```
10/11/2026
```

EXECUTION TRACE / READ THE STEPS

1. The LocalDate is independent of display format.
2. The formatter applies a month/day/year pattern.
3. The result is a String for presentation or interchange.

DEEPER UNDERSTANDING / WHY IT MATTERS

Use predefined ISO formatters for machine-to-machine contracts where possible. In custom patterns, lower-case mm means minutes while upper-case MM means month; yyyy and uuuu also have nuanced era behavior. Formatter instances are immutable and thread-safe. When formatting an Instant as local time, supply a ZoneId; parsing a text without an offset cannot magically recover the original time zone.

REAL-WORLD DECISION / EDGE CASE

Prefer ISO-8601 strings at service boundaries and localized formatting in presentation code.

COMMON PITFALL

Using "mm/dd/yyyy" expecting month but actually printing minutes.

PRACTICE / CHANGE ONE THING

Is DateTimeFormatter thread-safe for reuse?

CHECK YOUR ANSWER

Yes, java.time formatters are immutable and thread-safe.

EXPANSION 11 / AFTER ORIGINAL LESSON 23

Parsing and formatting: machine contracts versus local display

WHY THE DISTINCTION MATTERS

Stable storage and API formats are different from how people expect dates, decimal numbers and labels to appear on-screen.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  LocalDate day = LocalDate.parse("2026-10-11");
2  DateTimeFormatter f = DateTimeFormatter.ofPattern(
3      "dd MMM yyyy", Locale.US);
4  System.out.println(day.format(f));
5  System.out.println(day);
```

EXPECTED OUTPUT

```
11 Oct 2026
2026-10-11
```

UNDER THE HOOD / FOLLOW THE EXECUTION

LocalDate.parse() defaults to an ISO-8601 date. The explicit DateTimeFormatter turns it into a human-friendly representation. Unlike SimpleDateFormat, modern java.time formatters are immutable and safe to share. In pattern syntax, uppercase M means month, lowercase m means minute, and lowercase y denotes year-of-era; pattern letters matter.

PRODUCTION RULE OR FAILURE MODE

Avoid serializing a localized display string as a machine API contract. Prefer ISO-8601 or a documented structured format at service boundaries and apply locale rules at the presentation edge.

TRY THIS, THEN REVEAL THE REASON

Challenge: What would the pattern "dd mm yyyy" accidentally show in a LocalDate formatting operation?
Reasoned answer: It asks for minutes, which a LocalDate does not contain; formatting can fail rather than providing a month.

Chapter 4 — Master Recall Map

A fast scan of the most important invariants and practical decisions. Each section links back to the lessons you just studied.

JAVA CHAPTER 4 | MASTER INFOGRAPHIC

CORE RULES • EXAMPLES • REAL-WORLD DECISIONS

01 STRINGS

- ✓ Immutable text; equals() checks content.
- ✓ Substring end index is exclusive.
- ✓ Unicode code points are not always char values.

02 WRAPPERS

- ✓ Integer/Double are objects, not primitives.
- ✓ Unboxing null throws NullPointerException.
- ✓ Do not rely on wrapper == caching.

03 ENUMS

- ✓ Fixed named set of domain values.
- ✓ Use == on enum constants.
- ✓ Switch on explicit states.

04 MONEY

- ✓ Use new BigDecimal("0.1"), not new BigDecimal(0.1).
- ✓ State rounding mode explicitly.
- ✓ compareTo is numeric; equals includes scale.

05 DATES & TIME

- ✓ LocalDate = date; LocalDateTime = wall-clock.
- ✓ Instant = UTC timeline point.
- ✓ ZonedDateTime = local time plus zone rules.

06 PARSING & FORMATTING

- ✓ Parse strings at the boundary.
- ✓ Specify Locale and ZoneId when needed.
- ✓ Prefer explicit formatters for external data.

Real-world checklist before you move on

REVIEW POINT

☐ Can I state which text equality is intended?

REVIEW POINT

☐ Can a nullable wrapper unbox here?

REVIEW POINT

☐ Is a status best represented as enum or String?

REVIEW POINT

☐ Have I converted monetary text directly to BigDecimal?

REVIEW POINT

☐ Did I specify rounding only where business rules require it?

REVIEW POINT

☐ Does this value represent a date, local time, or absolute Instant?

REVIEW POINT

☐ Have I specified ZoneId and Locale at the proper boundary?

NEXT STEP

Run the matching *Demo.java file in the companion IntelliJ/Maven project. Compare actual console output to EXPECTED_OUTPUT.txt; then change inputs and predict behavior before rerunning.

Chapter 5
Collections and Generics

WHY THIS CHAPTER MATTERS

Collections are used everywhere: API results, validation errors, caches, repository data, indexing, grouping, and in-memory transformations. Selecting the right abstraction and respecting its contracts prevents subtle correctness and performance bugs.

What you will be able to do

- Select List, Set, or Map based on semantics and required ordering.
- Maintain correct equals/hashCode behavior for keys and set elements.
- Write safe iteration, filtering, sorting, and map updates.
- Understand generics, invariance, wildcards and type erasure at a practical depth.

HOW TO STUDY

Read the explanation; trace the code before looking at output; test the variation in IntelliJ. Code panels are focused teaching snippets; the companion project contains complete compilable examples.

23 practical lessons • illustrated reference poster • executable companion project

CHAPTER 5 • COLLECTIONS & GENERICS

List • Set • Map • iteration • ordering • generic type safety

WHY THIS MATTERS

Core Java building blocks for reliable business applications.

MASTER THESE

- List preserves positions and allows duplicates.
- Set enforces uniqueness; Map associates keys with values.
- Generic types prevent common casting errors.

KEY TAKEAWAY

CHOOSE BY MEANING, THEN BY PERFORMANCE

1 List

Ordered and allows duplicates

- ArrayList is a strong everyday default.
- Use get(index), add, set and remove deliberately.
- List.of creates an unmodifiable list.

QUICK MODEL

ordered
duplicates OK
indexed lookup

JAVA EXAMPLE • illustrative snippet

```
List<String> names = List.of("A", "B");
System.out.println(names.get(1));
```

OUTPUT B

2 Set

Unique values

- HashSet: no guaranteed iteration order.
- LinkedHashSet: insertion order; TreeSet: sorted.
- equals() and hashCode() drive hash-based uniqueness.

QUICK MODEL

no duplicates
set membership
order varies by type

JAVA EXAMPLE • illustrative snippet

```
Set<String> s = new LinkedHashSet<>();
s.add("A"); s.add("A");
System.out.println(s);
```

OUTPUT [A]

3 Map

Keys → values

- HashMap is a general-purpose key/value store.
- Keys are unique; putting an existing key replaces value.
- getOrDefault, merge and computeIfAbsent are useful.

QUICK MODEL

key → value
unique keys
fast lookup

JAVA EXAMPLE • illustrative snippet

```
Map<String, Integer> m = new HashMap<>();
m.put("a", 1);
m.merge("a", 2, Integer::sum);
System.out.println(m.get("a"));
```

OUTPUT 3

4 Safe Iteration

Avoid structural-modification traps

- Enhanced for is easy when not removing entries.
- Use removeIf or Iterator.remove for safe removal.
- ConcurrentModificationException is not a thread-safety guarantee.

QUICK MODEL

for-each: read
removeIf: edit
iterator.remove

JAVA EXAMPLE • illustrative snippet

```
List<String> a = new ArrayList<>(
    List.of("A", "B", "C"));
a.removeIf(s -> s.equals("B"));
System.out.println(a);
```

OUTPUT [A, C]

5 Generics

Reusable compile-time type safety

- List<String> restricts elements to String.
- ? extends T is a producer; ? super T is a consumer.
- Generic type arguments are erased at runtime.

QUICK MODEL

extends: read
super: write
PECS

JAVA EXAMPLE • illustrative snippet

```
List<? extends Number> nums = List.of(7);
Number n = nums.get(0);
System.out.println(n);
```

OUTPUT 7

6 Ordering & Choice

Comparator, snapshots, tradeoffs

- Comparator defines custom sorting; sorted() copies logically.
- Collections.sort mutates a modifiable List.
- Map views are live; List.copyOf makes a snapshot.

QUICK MODEL

List: sequence
Set: unique
Map: lookup

JAVA EXAMPLE • illustrative snippet

```
List<Integer> a = new ArrayList<>(
    List.of(3, 1, 2));
a.sort(Comparator.naturalOrder());
System.out.println(a);
```

OUTPUT [1, 2, 3]

WATCH OUT • COMMON MISTAKES

- Depending on HashMap/HashSet iteration order
- Mutating map keys after insertion
- Calling add on List.of results
- Modifying a List inside enhanced for

QUICK RECALL • BEST PRACTICES

Performance: ArrayList has fast indexed access; HashMap/HashSet average fast lookup.

Streams and Optional are covered in the later dedicated chapter.

CHAPTER 5 / LESSON 01 OF 23

Choose List, Set or Map by meaning

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use List for ordered sequences with duplicates, Set for membership uniqueness, and Map for key-to-value lookup.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("A");
System.out.println(list.size());
```

EXPECTED OUTPUT / RESULT

2

EXECUTION TRACE / READ THE STEPS

1. ArrayList preserves the sequence of additions.
2. List semantics allow duplicates.
3. size() reports two entries.

DEEPER UNDERSTANDING / WHY IT MATTERS

A good collection choice begins with the business invariant rather than performance folklore. Use a Set when duplicates are logically invalid, not merely when you hope removal is faster. Use a Map when lookup is naturally by key, such as customer ID. Program to interfaces (List, Set, Map) in method signatures and choose implementations deliberately.

REAL-WORLD DECISION / EDGE CASE

Do not expose a mutable List if callers should not modify internal state.

COMMON PITFALL

Using a List for repeated contains() lookups on large sets when Set membership would better model the problem.

PRACTICE / CHANGE ONE THING

Which abstraction best maps customer ID to Customer?

CHECK YOUR ANSWER

Map<String,Customer>.

CHAPTER 5 / LESSON 02 OF 23

ArrayList behavior and indexed access

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

ArrayList is usually the default general-purpose List: insertion order, duplicates, fast indexed reads and amortized append.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<String> items = new ArrayList<>();
items.add("alpha");
items.add("beta");
System.out.println(items.get(1));
```

EXPECTED OUTPUT / RESULT

```
beta
```

EXECUTION TRACE / READ THE STEPS

1. The first append occupies index 0.
2. The second append occupies index 1.
3. get(1) returns beta.

DEEPER UNDERSTANDING / WHY IT MATTERS

ArrayList is backed by a resizable array. get(index) is O(1), while inserting or removing near the front generally shifts elements (O(n)). Adding to the end is usually amortized O(1). For ordinary web application code, ArrayList is typically a better starting choice than LinkedList. Index boundaries run 0 to size()-1, and invalid access throws IndexOutOfBoundsException.

REAL-WORLD DECISION / EDGE CASE

When changing the list during iteration, use a safe method rather than indexed writes that miss entries.

COMMON PITFALL

Expecting list.get(list.size()) to access the final element.

PRACTICE / CHANGE ONE THING

What index holds the final item in a three-element List?

CHECK YOUR ANSWER

2.

EXPANSION 01 / AFTER ORIGINAL LESSON 02

ArrayList vs LinkedList: choose by access patterns

WHY THE DISTINCTION MATTERS

Both implement List, but their storage and constant factors differ. In practice, ArrayList is usually the default; LinkedList is rarely a performance improvement for normal application code.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  List<String> tasks = new ArrayList<>();
2  tasks.add("A");
3  tasks.add("C");
4  tasks.add(1, "B");
5  System.out.println(tasks);
6  System.out.println(tasks.get(2));
```

EXPECTED OUTPUT

```
[A, B, C]
C
```

UNDER THE HOOD / FOLLOW THE EXECUTION

ArrayList offers amortized constant-time append and fast indexed access, with insertion/removal in the middle typically moving references. LinkedList stores each element in a node with neighbor links; locating an index generally requires traversal. Although linking a known node is cheap, locating that node is often the costly part. Memory allocation, cache locality and iteration patterns tend to favor ArrayList in typical server applications.

PRODUCTION RULE OR FAILURE MODE

If the real requirement is FIFO or a double-ended queue, prefer ArrayDeque for single-threaded use rather than assuming LinkedList is automatically better. Choose by measured workload and the required abstraction, not just theoretical Big-O tables.

TRY THIS, THEN REVEAL THE REASON

Challenge: Does LinkedList.get(100000) have constant-time indexed access?
Reasoned answer: No. It traverses from one end, taking linear time in the worst case.

CHAPTER 5 / LESSON 03 OF 23

List.of() and unmodifiable collections

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

List.of creates an unmodifiable list and rejects null elements; its members may still be mutable objects.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<String> states = List.of("NEW", "DONE");
System.out.println(states.get(0));
System.out.println(states.size());
```

EXPECTED OUTPUT / RESULT

```
NEW
2
```

EXECUTION TRACE / READ THE STEPS

1. The factory returns an unmodifiable two-item List.
2. The first item is NEW.
3. size() remains two.

DEEPER UNDERSTANDING / WHY IT MATTERS

Unmodifiable does not imply deep immutability. A List<Person> can be structurally unmodifiable while Person objects inside it remain mutable. List.of rejects null; Arrays.asList allows set() but not add()/remove() and is backed by the original array. Collections.unmodifiableList provides a view over a backing list; changes to the backing list can still appear. List.copyOf creates an unmodifiable copy of elements (still shallow).

REAL-WORLD DECISION / EDGE CASE

Know whether an API promises a snapshot or merely a read-only view.

COMMON PITFALL

Calling states.add("X") would throw UnsupportedOperationException.

PRACTICE / CHANGE ONE THING

Does List.of(person) make person itself immutable?

CHECK YOUR ANSWER

No.

EXPANSION 02 / AFTER ORIGINAL LESSON 03

Unmodifiable, immutable and copied collections

WHY THE DISTINCTION MATTERS

Preventing callers from changing the contents through one reference does not necessarily freeze the objects stored inside a collection.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  List<String> base = new ArrayList<>(List.of("A"));
2  List<String> view = Collections.unmodifiableList(base);
3  List<String> copy = List.copyOf(base);
4  base.add("B");
5  System.out.println(view);
6  System.out.println(copy);
```

EXPECTED OUTPUT

```
[A, B]
[A]
```

UNDER THE HOOD / FOLLOW THE EXECUTION

An unmodifiable view forbids mutation through the view reference but reflects changes to its backing collection. List.copyOf creates an unmodifiable snapshot of the element references (it may reuse an already-unmodifiable collection). Neither necessarily deep-copies mutable elements. The words “unmodifiable collection,” “immutable collection” and “deeply immutable object graph” should not be used interchangeably.

PRODUCTION RULE OR FAILURE MODE

The collection factory methods such as List.of reject null elements; List.copyOf also rejects nulls. A collection that stores mutable DTOs may need defensive element copies to prevent callers from altering business state.

TRY THIS, THEN REVEAL THE REASON

Challenge: If each list contains a mutable Person, does List.copyOf(base) freeze Person objects?
Reasoned answer: No. It prevents structural changes through that list but retains references to the same mutable elements.

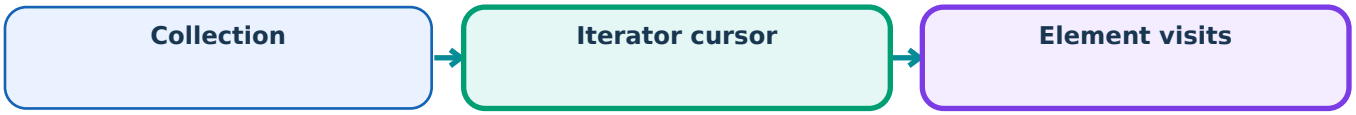
Iteration styles: for-each, index and Iterator

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Prefer enhanced for for ordinary traversal, index loops when position matters, and Iterator when removing during traversal.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<String> items = List.of("A", "B");
for (String item : items)
{
    System.out.print(item);
}
```

EXPECTED OUTPUT / RESULT

```
AB
```

EXECUTION TRACE / READ THE STEPS

1. The enhanced-for loop obtains an Iterator conceptually.
2. Each item is visited in list order.
3. print() writes A followed by B without spaces.

DEEPER UNDERSTANDING / WHY IT MATTERS

A for-each loop is clearer when no index is needed. Iterators expose traversal without assuming a backing array. ListIterator adds bidirectional operations for Lists, but is not typically needed for simple collection processing. When modifying elements or structure during iteration, study the collection contract; structural modification outside the iterator may cause ConcurrentModificationException.

REAL-WORLD DECISION / EDGE CASE

Avoid expensive side effects inside collection traversals where retries or parallel calls are possible.

COMMON PITFALL

Assuming for-each automatically supports safe list.remove(item) during iteration.

PRACTICE / CHANGE ONE THING

When do you prefer an index loop over for-each?

CHECK YOUR ANSWER

When you need the numeric position or index-based mutation.

Safe removal during iteration

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use removeIf or Iterator.remove rather than structurally modifying an ArrayList inside a for-each loop.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4));
nums.removeIf(n -> n % 2 == 0);
System.out.println(nums);
```

EXPECTED OUTPUT / RESULT

```
[1, 3]
```

EXECUTION TRACE / READ THE STEPS

1. A mutable ArrayList is created.
2. removeIf checks the predicate for every element.
3. Even numbers are removed safely, leaving 1 and 3.

DEEPER UNDERSTANDING / WHY IT MATTERS

removeIf is concise and designed for bulk conditional removal. An Iterator permits iterator.remove() once after next() as specified. Structural modification while using the enhanced-for iterator often triggers fail-fast detection, but that is a best-effort bug-detection feature, not a concurrency safety guarantee. For bulk transformations, consider constructing a new collection when preserving the original matters.

REAL-WORLD DECISION / EDGE CASE

If your method receives an unmodifiable list, make a mutable copy before removeIf.

COMMON PITFALL

Calling nums.remove(n) inside for (Integer n : nums) may lead to ConcurrentModificationException.

PRACTICE / CHANGE ONE THING

Does removeIf work on List.of(1,2,3)?

CHECK YOUR ANSWER

No; List.of returns an unmodifiable list.

EXPANSION 03 / AFTER ORIGINAL LESSON 05

Queue and Deque: processing work without index arithmetic

WHY THE DISTINCTION MATTERS

A queue expresses “process next” more clearly than removing element zero from an ArrayList. A Deque supports both FIFO and stack-like LIFO operations.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  Deque<String> queue = new ArrayDeque<>();
2  queue.addLast("first");
3  queue.addLast("second");
4  System.out.println(queue.removeFirst());
5  queue.addFirst("urgent");
6  System.out.println(queue.removeFirst());
```

EXPECTED OUTPUT

```
first
urgent
```

UNDER THE HOOD / FOLLOW THE EXECUTION

ArrayDeque is a resizable array-backed double-ended queue. addLast/removeFirst is a natural FIFO pattern, while push/pop is a natural LIFO stack pattern. Queue methods come in paired forms: add/remove/element throw on certain failures, while offer/poll/peek use special return values. Use the operations whose empty-queue behavior is appropriate.

PRODUCTION RULE OR FAILURE MODE

ArrayDeque does not allow null elements. It is not thread-safe; a producer-consumer system with multiple threads needs a suitable concurrent or blocking queue instead.

TRY THIS, THEN REVEAL THE REASON

Challenge: What does poll() return on an empty ArrayDeque?
Reasoned answer: null, instead of throwing. remove() throws NoSuchElementException.

EXPANSION 08 / AFTER ORIGINAL LESSON 05

Safe mutation during traversal: iterator and removeIf

WHY THE DISTINCTION MATTERS

Removing elements through the wrong path while iterating often leads to ConcurrentModificationException or silently incorrect logic.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  List<Integer> numbers = new ArrayList<>(
2      List.of(1, 2, 3, 4, 5));
3  numbers.removeIf(n -> n % 2 == 0);
4  System.out.println(numbers);
```

EXPECTED OUTPUT

```
[1, 3, 5]
```

UNDER THE HOOD / FOLLOW THE EXECUTION

removeIf expresses filtering-by-mutation cleanly. If you use an Iterator explicitly, its own remove method coordinates with traversal for supported collections. A typical enhanced for-loop uses an iterator internally, so structural changes through list.remove() during the loop can trigger fail-fast behavior. Fail-fast is for best-effort bug detection; it is not a concurrency safety guarantee.

PRODUCTION RULE OR FAILURE MODE

Do not assume removing from a collection while another thread iterates is safe. Design synchronization or use an appropriate concurrent collection and understand its iteration guarantees.

TRY THIS, THEN REVEAL THE REASON

Challenge: Is List.of(1,2,3).removeIf(x -> x > 1) valid?
Reasoned answer: No; List.of produces an unmodifiable list, so a mutating operation throws UnsupportedOperationException.

CHAPTER 5 / LESSON 06 OF 23

HashSet: membership and uniqueness

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

HashSet enforces uniqueness under equals()/hashCode() and does not guarantee iteration order.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Set<String> tags = new HashSet<>();
tags.add("blue");
tags.add("blue");
System.out.println(tags.size());
```

EXPECTED OUTPUT / RESULT

```
1
```

EXECUTION TRACE / READ THE STEPS

1. The first add inserts blue.
2. The second equal String is already present.
3. Set size remains one.

DEEPER UNDERSTANDING / WHY IT MATTERS

HashSet is typically fast for add, contains and remove, subject to normal hashing assumptions. Do not assume insertion order based on one printed example; choose LinkedHashSet to preserve insertion order or TreeSet for sorted ordering. Objects used in hash-based sets must have stable equality and hash codes while stored. A correctly implemented value object can be distinct in identity while equal in content.

REAL-WORLD DECISION / EDGE CASE

Use Set when uniqueness is an actual domain invariant, such as distinct permissions or IDs.

COMMON PITFALL

Relying on HashSet iteration order when serializing externally visible output.

PRACTICE / CHANGE ONE THING

What happens if you add "blue" again to the same HashSet?

CHECK YOUR ANSWER

add returns false, and the size does not increase.

equals() and hashCode() as one contract

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Equal objects must produce equal hash codes, or HashSet and HashMap lookups can fail.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
record Key(String id)
{
}
Set<Key> keys = new HashSet<>();
keys.add(new Key("X"));
System.out.println(keys.contains(new Key("X")));
```

EXPECTED OUTPUT / RESULT

```
true
```

EXECUTION TRACE / READ THE STEPS

1. Records automatically implement value-based equals/hashCode from components.
2. The first Key is inserted into the HashSet.
3. The newly created equal Key can be found.

DEEPER UNDERSTANDING / WHY IT MATTERS

When overriding equals(), you must also override hashCode() consistently. Different objects may share a hash code; equal hash codes do not mean objects are equal. Hash collisions are handled by the collection. An equality relation should also be reflexive, symmetric, transitive and consistent under unchanged state. Records are a convenient immutable-data carrier when their components have suitable equality semantics.

REAL-WORLD DECISION / EDGE CASE

If you have mutable fields participating in equals/hashCode, consider immutable keys or stable IDs instead.

COMMON PITFALL

Overriding equals only and relying on Object.hashCode breaks the required contract.

PRACTICE / CHANGE ONE THING

Does matching hashCode guarantee equals returns true?

CHECK YOUR ANSWER

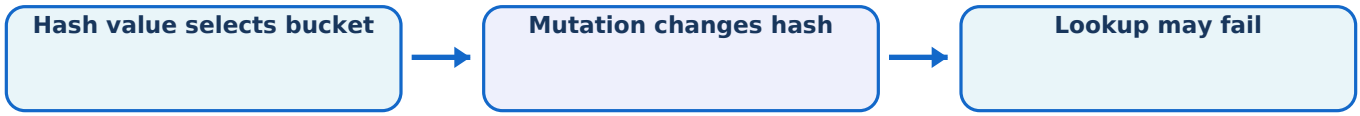
No.

EXPANSION 05 / AFTER ORIGINAL LESSON 07

Hash-based collection contracts: changing a key is dangerous

WHY THE DISTINCTION MATTERS

HashMap and HashSet rely on stable hashCode and equals behavior for keys already stored. Mutating a field participating in equality can make a key seemingly disappear.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  record Key(String id) {}
2  Map<Key, Integer> map = new HashMap<>();
3  map.put(new Key("A"), 10);
4  System.out.println(map.get(new Key("A")));
5  System.out.println(map.containsKey(new Key("B")));
```

EXPECTED OUTPUT

```
10
false
```

UNDER THE HOOD / FOLLOW THE EXECUTION

A record is immutable with respect to its component references and automatically provides value-based equals/hashCode based on those components. Using such a value object as a map key avoids a common mutable-key pitfall. If two objects are equal according to equals, their hash codes must match. A stable hashCode alone is insufficient: equals also determines whether a candidate is the same logical key.

PRODUCTION RULE OR FAILURE MODE

A record that contains a mutable component can still have a mutable equality-relevant object graph. For persistent identity, prefer stable keys such as immutable IDs or domain value objects, not an entity object whose properties change.

TRY THIS, THEN REVEAL THE REASON

Challenge: Why can a HashSet fail to find a mutable key after a field changes?
Reasoned answer: The new hash can point to a different bucket even though the key was stored under its old hash.

CHAPTER 5 / LESSON 08 OF 23

LinkedHashSet and TreeSet ordering

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Choose LinkedHashSet for insertion order and TreeSet for comparator-defined sorted order.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Set<Integer> values = new LinkedHashSet<>();
values.add(9);
values.add(2);
System.out.println(values);
```

EXPECTED OUTPUT / RESULT

```
[9, 2]
```

EXECUTION TRACE / READ THE STEPS

1. LinkedHashSet records insertion order.
2. Values 9 then 2 are inserted.
3. Iteration and toString reflect that insertion order.

DEEPER UNDERSTANDING / WHY IT MATTERS

TreeSet uses sorting comparison to determine element ordering and equivalence; a comparator returning 0 treats entries as equivalent for Set membership. This can disagree with equals, so design the comparator consistently when possible. TreeSet operations are normally O(log n), while linked hash sets preserve order without numeric sorting. Neither choice automatically provides concurrency safety.

REAL-WORLD DECISION / EDGE CASE

Order matters in user-facing results and reproducible exports; select it explicitly.

COMMON PITFALL

Treating TreeSet as if it used hashCode for its unique-element decision.

PRACTICE / CHANGE ONE THING

Which Set keeps insertion order without sorting?

CHECK YOUR ANSWER

LinkedHashSet.

EXPANSION 04 / AFTER ORIGINAL LESSON 08

PriorityQueue: highest priority is not sorted iteration

WHY THE DISTINCTION MATTERS

PriorityQueue efficiently retrieves the next element by comparator order, but iterating its contents does not promise globally sorted order.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```java
Queue<Integer> q = new PriorityQueue<>();
q.offer(30);
q.offer(10);
q.offer(20);
while (!q.isEmpty())
{
    System.out.print(q.poll() + " ");
}
```

EXPECTED OUTPUT

```
10 20 30
```

UNDER THE HOOD / FOLLOW THE EXECUTION

A Java PriorityQueue uses a heap and keeps its head as the least element under the natural or supplied comparator. offer and poll take logarithmic time in the typical bound; peek is constant time. Polling repeatedly produces priority order, while the iterator does not promise it. If priorities tie, the queue does not guarantee FIFO tie-breaking unless you include a sequence number in the comparator.

PRODUCTION RULE OR FAILURE MODE

PriorityQueue is not a thread-safe work scheduler. If multiple threads feed and consume prioritized work, look at PriorityBlockingQueue and explicitly consider capacity and fairness.

TRY THIS, THEN REVEAL THE REASON

Challenge: Can you sort a list merely by calling new PriorityQueue<>(list) and then iterating?
Reasoned answer: No. Poll repeatedly or copy to a list and sort; iterator order is unspecified.

CHAPTER 5 / LESSON 09 OF 23

Map as keyed lookup

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

A Map associates unique keys with values. put replaces an existing value for the same key.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Map<String, Integer> counts = new HashMap<>();
counts.put("A", 1);
counts.put("A", 3);
System.out.println(counts.get("A"));
```

EXPECTED OUTPUT / RESULT

```
3
```

EXECUTION TRACE / READ THE STEPS

1. First put associates A with 1.
2. Second put uses an equal key and replaces the value with 3.
3. get(A) returns 3.

DEEPER UNDERSTANDING / WHY IT MATTERS

A Map is not a collection of independent key/value pairs with duplicate keys: a key identifies at most one mapping. HashMap makes no ordering guarantee. LinkedHashMap preserves insertion order (or configurable access order), and TreeMap sorts keys. For external output, use explicit sorting or insertion order; do not rely on accidental HashMap traversal order.

REAL-WORLD DECISION / EDGE CASE

Keys should be stable business identifiers where possible.

COMMON PITFALL

Assuming put() throws on a duplicate key; it normally replaces the value.

PRACTICE / CHANGE ONE THING

What does put return when replacing a previous non-null value?

CHECK YOUR ANSWER

The previous value.

Map defaults, merge and computeIfAbsent

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use merge and computeIfAbsent for common update patterns, while keeping mapping functions side-effect-aware.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Map<String, Integer> tally = new HashMap<>();
tally.merge("A", 1, Integer::sum);
tally.merge("A", 2, Integer::sum);
System.out.println(tally.get("A"));
```

EXPECTED OUTPUT / RESULT

3

EXECUTION TRACE / READ THE STEPS

1. The first merge inserts the value 1 because the key is absent.
2. The second merge combines existing 1 and new 2 using Integer::sum.
3. The final mapping is 3.

DEEPER UNDERSTANDING / WHY IT MATTERS

Map.getOrDefault gives a fallback return value but does not insert it. computeIfAbsent computes and installs a value when missing or mapped to null. If its function returns null, no mapping is stored. merge is useful for counters and accumulators and handles missing/null differently from a manual get/put pattern. For concurrent maps, atomicity depends on the particular implementation and operation.

REAL-WORLD DECISION / EDGE CASE

Do not perform surprising modifications to the same Map inside compute functions; obey the method contract.

COMMON PITFALL

Using getOrDefault expecting it to populate the map.

PRACTICE / CHANGE ONE THING

What does tally.getOrDefault("B", 0) do to the map?

CHECK YOUR ANSWER

Returns 0 without inserting B.

EXPANSION 06 / AFTER ORIGINAL LESSON 10

Map.merge, computeIfAbsent and atomicity assumptions

WHY THE DISTINCTION MATTERS

Map convenience methods express update intent and reduce boilerplate, but ordinary HashMap operations are not automatically thread-safe transactions.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  Map<String, Integer> counts = new HashMap<>();
2  for (String s : List.of("A", "B", "A"))
3  {
4      counts.merge(s, 1, Integer::sum);
5  }
6  System.out.println(counts.get("A"));
7  System.out.println(counts.get("B"));
```

EXPECTED OUTPUT

```
2
1
```

UNDER THE HOOD / FOLLOW THE EXECUTION

merge inserts the supplied value when the key is absent or mapped to null, otherwise it applies the remapping function. Returning null from the remapping function removes the mapping. computeIfAbsent computes a value only when there is no non-null mapping. Do not use mapping functions for arbitrary side effects: they may be invoked under implementation-specific rules and are harder to reason about if they mutate the same map.

PRODUCTION RULE OR FAILURE MODE

In concurrent systems, use ConcurrentHashMap when its semantics fit, and still understand that a sequence of multiple separate map operations may not be an atomic business transaction.

TRY THIS, THEN REVEAL THE REASON

Challenge: What happens if a merge remapping function returns null?
Reasoned answer: The entry is removed. With an absent key, the initial non-null merge value is inserted.

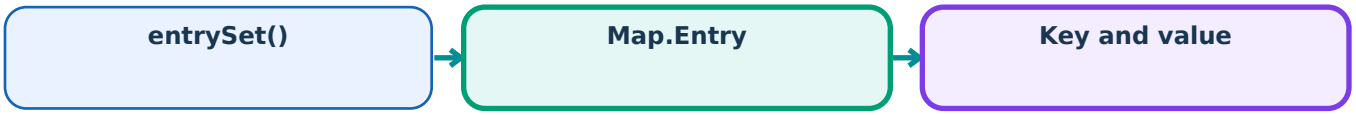
Map traversal: entrySet, keys and values

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use entrySet when both key and value are needed; keys and values are also live views for many map implementations.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Map<String, Integer> score = new LinkedHashMap<>();
score.put("A", 7);
for (Map.Entry<String, Integer> e : score.entrySet())
{
    System.out.println(e.getKey() + ":" + e.getValue());
}
```

EXPECTED OUTPUT / RESULT

```
A:7
```

EXECUTION TRACE / READ THE STEPS

1. LinkedHashMap remembers insertion order.
2. entrySet provides entries, not independent pair copies.
3. The loop prints key A and value 7.

DEEPER UNDERSTANDING / WHY IT MATTERS

Iterating entrySet avoids a separate get(key) for every key. keySet() is useful when only keys are needed, and values() when only values matter. These views may reflect later map changes and often support removal according to implementation, so do not mistake them for immutable snapshots. To persist a snapshot, copy the content into a new appropriate map.

REAL-WORLD DECISION / EDGE CASE

When exposing maps through APIs, decide whether the caller receives a live view, mutable copy or unmodifiable snapshot.

COMMON PITFALL

Editing a map while traversing its entries outside supported iterator methods.

PRACTICE / CHANGE ONE THING

Which map view is most useful when you need both key and value?

CHECK YOUR ANSWER

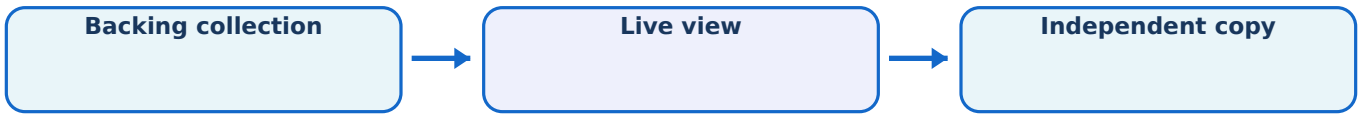
entrySet().

EXPANSION 07 / AFTER ORIGINAL LESSON 11

Views versus snapshots: map and subList surprises

WHY THE DISTINCTION MATTERS

Collection views are often live windows into underlying storage. A change through the backing collection can appear in a view, and structural changes may invalidate a subList.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  Map<String, Integer> map = new LinkedHashMap<>();
2  map.put("A", 1);
3  Set<String> liveKeys = map.keySet();
4  Set<String> snapshot = new LinkedHashSet<>(liveKeys);
5  map.put("B", 2);
6  System.out.println(liveKeys);
7  System.out.println(snapshot);
```

EXPECTED OUTPUT

```
[A, B]
[A]
```

UNDER THE HOOD / FOLLOW THE EXECUTION

keySet(), values() and entrySet() return views backed by the map. Here the live key set reflects addition of B, while the newly created LinkedHashSet is an independent structural snapshot. subList() is also a backed view, and unexpected structural modifications to the backing list can invalidate its behavior. When using a view, understand which modifications are supported and whether they flow in both directions.

PRODUCTION RULE OR FAILURE MODE

A snapshot of references is still shallow. If the values are mutable objects, changes to those objects remain visible through both original and copied collections.

TRY THIS, THEN REVEAL THE REASON

Challenge: Does new ArrayList<>(original) clone every contained object?
Reasoned answer: No. It copies the references, not the objects.

Absent keys versus mapped null

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Map.get() returning null cannot distinguish missing keys from a key intentionally mapped to null.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Map<String, String> data = new HashMap<>();
data.put("A", null);
System.out.println(data.get("A"));
System.out.println(data.containsKey("A"));
```

EXPECTED OUTPUT / RESULT

```
null
true
```

EXECUTION TRACE / READ THE STEPS

1. HashMap permits a null value.
2. get(A) returns null.
3. containsKey(A) proves that A is a present key.

DEEPER UNDERSTANDING / WHY IT MATTERS

Different Map implementations have different null restrictions; Map.of rejects null keys and values, while ConcurrentHashMap forbids them. When null means missing or unknown, model that state carefully rather than conflating it with absence. If the business logic needs a tri-state, choose an explicit representation and document it.

REAL-WORLD DECISION / EDGE CASE

Database null semantics and Java map absence are not interchangeable.

COMMON PITFALL

Using get(key)==null to conclude a key is absent in HashMap.

PRACTICE / CHANGE ONE THING

Which method confirms the key exists even if its value is null?

CHECK YOUR ANSWER

containsKey(key).

CHAPTER 5 / LESSON 13 OF 23

Mutable keys break hash lookups

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Do not mutate fields that participate in equals()/hashCode() while an object is stored as a hash key.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
record Key(String id)
{
}
Map<Key, String> map = new HashMap<>();
map.put(new Key("A"), "found");
System.out.println(map.get(new Key("A")));
```

EXPECTED OUTPUT / RESULT

```
found
```

EXECUTION TRACE / READ THE STEPS

1. A value-based immutable record key is inserted.
2. A second equal key has the same equality/hash semantics.
3. The HashMap lookup finds the value.

DEEPER UNDERSTANDING / WHY IT MATTERS

A mutable key can drift into a different conceptual hash bucket after insertion; subsequent lookup, removal or duplicate handling may become inconsistent. The easiest design is an immutable key: String, UUID, an immutable record, or a well-designed value object. A mutable Person object can safely be the Map value, while the Map key remains a stable person ID.

REAL-WORLD DECISION / EDGE CASE

Do not derive a Map key’s hashCode from fields that the application modifies regularly.

COMMON PITFALL

Changing id after insertion and expecting HashMap to rehash automatically.

PRACTICE / CHANGE ONE THING

Will HashMap automatically relocate entries after a key mutates?

CHECK YOUR ANSWER

No.

Comparable establishes natural ordering

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Comparable.compareTo supplies a natural sort order; it must be consistent and transitive.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<Integer> nums = new ArrayList<>(List.of(9, 2, 7));
Collections.sort(nums);
System.out.println(nums);
```

EXPECTED OUTPUT / RESULT

```
[2, 7, 9]
```

EXECUTION TRACE / READ THE STEPS

1. The mutable List contains 9,2,7.
2. Collections.sort invokes Integer natural ordering.
3. The List becomes sorted ascending.

DEEPER UNDERSTANDING / WHY IT MATTERS

Comparable is useful when a type has one obvious default order. compareTo returns negative/zero/positive, not necessarily -1/0/1; code must inspect the sign. Avoid subtraction comparisons like return a.id-b.id because overflow can produce wrong ordering. Sorting mutates a mutable List; List.of results are unmodifiable and cannot be sorted in place.

REAL-WORLD DECISION / EDGE CASE

Natural order should be documented and ideally compatible with equality for sorted-set behavior.

COMMON PITFALL

Implementing compareTo with integer subtraction and causing overflow.

PRACTICE / CHANGE ONE THING

Should comparator code check compare(a,b)==-1 to mean a<b?

CHECK YOUR ANSWER

No. Check whether the result is negative.

Comparator for multi-field sorting

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Comparator defines ordering separate from the class; chain comparing and thenComparing for deterministic tie-breaking.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<String> names = new ArrayList<>(List.of("Bob", "Ann", "Al"));
names.sort(Comparator.comparingInt(String::length)
        .thenComparing(Comparator.naturalOrder()));
System.out.println(names);
```

EXPECTED OUTPUT / RESULT

```
[Al, Ann, Bob]
```

EXECUTION TRACE / READ THE STEPS

1. The primary key is String length.
2. Equal-length names are compared alphabetically.
3. The final List is Al, Ann, Bob.

DEEPER UNDERSTANDING / WHY IT MATTERS

Comparator is best when you need multiple legitimate orderings, e.g. last name, salary, date, or status. thenComparing prevents unstable presentation when primary fields tie. nullsFirst and nullsLast can make null policies explicit. reversed() reverses the whole comparator it is called on, so placement matters. Keep sort criteria aligned with business meaning.

REAL-WORLD DECISION / EDGE CASE

Use Comparator for DTO sorting without forcing a domain object to have one permanent natural order.

COMMON PITFALL

Accidentally reversing every key when only one sort key should descend.

PRACTICE / CHANGE ONE THING

How would you sort longer names first?

CHECK YOUR ANSWER

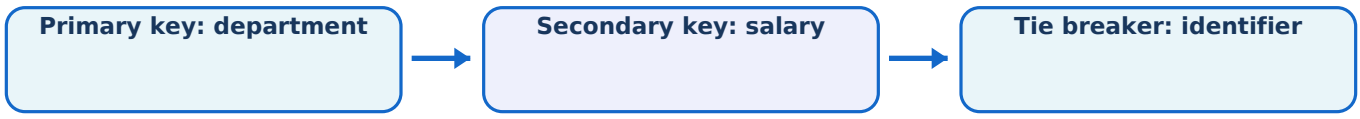
Use comparingInt(String::length).reversed(), then apply tie-breakers.

EXPANSION 09 / AFTER ORIGINAL LESSON 15

Comparator chains: deterministic multi-field ordering

WHY THE DISTINCTION MATTERS

Real records rarely sort by just one attribute. Comparator chains make priority rules explicit and avoid error-prone subtraction comparators.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  record Employee(String team, int salary) {}
2  List<Employee> people = new ArrayList<>(List.of(
3      new Employee("B", 90), new Employee("A", 80),
4      new Employee("A", 95)));
5  people.sort(Comparator.comparing(Employee::team)
6      .thenComparing(Comparator.comparingInt(
7          Employee::salary).reversed()));
8  System.out.println(people.get(0).salary());
```

EXPECTED OUTPUT

```
95
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The first comparator groups team A before team B. The secondary comparator sorts salary descending within each team. Using comparingInt avoids boxing a primitive int key. Comparators should obey consistent comparison contracts; avoid (a,b) -> a-b for general integer values because subtraction can overflow, reversing intended order.

PRODUCTION RULE OR FAILURE MODE

Add a stable final tie-breaker such as employee ID if downstream output must be deterministic across different input orders. Comparator.nullsFirst() and nullsLast() express explicit null ordering where needed.

TRY THIS, THEN REVEAL THE REASON

Challenge: What changes if reversed() is applied to the entire chained comparator rather than only the salary part? Reasoned answer: The whole order reverses, including team order. Parentheses and chaining location matter.

CHAPTER 5 / LESSON 16 OF 23

Factory collections, copies and views

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Collection factories and wrappers differ in mutability, null acceptance, and whether they snapshot or reflect a backing collection.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<String> base = new ArrayList<>();
base.add("A");
List<String> view = Collections.unmodifiableList(base);
base.add("B");
System.out.println(view.size());
```

EXPECTED OUTPUT / RESULT

2

EXECUTION TRACE / READ THE STEPS

1. The backing ArrayList initially has one item.
2. The unmodifiable view wraps that same list.
3. Later additions to base are visible through view.

DEEPER UNDERSTANDING / WHY IT MATTERS

Collections.unmodifiableList prevents mutation through the wrapper but not through the backing collection. List.copyOf makes an unmodifiable copy of elements and rejects null. Both are shallow: contained mutable objects can still change. Arrays.asList is fixed-size, backed by the array, and permits set(index,value). Know which contract you need before exposing a collection.

REAL-WORLD DECISION / EDGE CASE

Defensive snapshots are safer than live views for many boundary APIs.

COMMON PITFALL

Assuming unmodifiableList(base) is automatically a detached snapshot.

PRACTICE / CHANGE ONE THING

Will List.copyOf(base) reflect later structural additions to base?

CHECK YOUR ANSWER

No.

CHAPTER 5 / LESSON 17 OF 23

Generics enforce compile-time type safety

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Generic parameters express element types and catch incompatible inserts before runtime.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<String> names = new ArrayList<>();
names.add("Ada");
String first = names.get(0);
System.out.println(first);
```

EXPECTED OUTPUT / RESULT

Ada

EXECUTION TRACE / READ THE STEPS

1. The generic type is String.
2. The compiler permits String entries and rejects an Integer insertion.
3. get(0) returns a String without an explicit cast.

DEEPER UNDERSTANDING / WHY IT MATTERS

Before generics, callers often retrieved Object and cast manually, risking ClassCastException. Java generics are invariant by default, and runtime generic parameters are largely erased. Raw types discard type-checking and can create heap pollution. Favor generic APIs with meaningful bounds instead of Object collections and casts.

REAL-WORLD DECISION / EDGE CASE

Treat warnings about unchecked operations seriously, especially around legacy APIs.

COMMON PITFALL

Using raw List and ignoring unchecked warnings until a cast fails later.

PRACTICE / CHANGE ONE THING

Can names.add(42) compile for List<String>?

CHECK YOUR ANSWER

No.

CHAPTER 5 / LESSON 18 OF 23

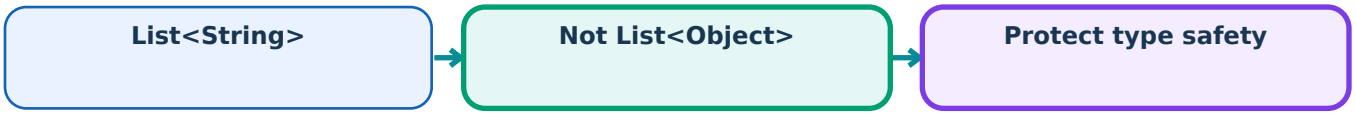
Generic invariance: List<String> is not List<Object>

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

A List<String> is not a subtype of List<Object>, even though String is a subtype of Object.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<String> strings = List.of("A");
List<? extends Object> objects = strings;
System.out.println(objects.get(0));
```

EXPECTED OUTPUT / RESULT

```
A
```

EXECUTION TRACE / READ THE STEPS

1. The source is a List<String>.
2. A wildcard producer view can refer to it safely.
3. Reading returns an Object, printed as A.

DEEPER UNDERSTANDING / WHY IT MATTERS

If List<String> were accepted as List<Object>, code could add an Integer through the second reference and corrupt the String list. The extends wildcard allows safe covariant reading but restricts inserting arbitrary objects. Arrays are covariant but perform runtime checks, which is one reason generics work differently. Learn the type relationship, not just the syntax.

REAL-WORLD DECISION / EDGE CASE

Use List<?> when you only need to inspect arbitrary elements without adding them.

COMMON PITFALL

Trying List<Object> objs = new ArrayList<String>();

PRACTICE / CHANGE ONE THING

Does List<? extends Object> permit objs.add(42)?

CHECK YOUR ANSWER

No, because the captured element type is unknown.

EXPANSION 10 / AFTER ORIGINAL LESSON 18

A practical generic class: keep element types aligned

WHY THE DISTINCTION MATTERS

The goal of generics is not merely to avoid casts on List. A generic class can preserve relationships between the types stored and returned by an API.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  class Box<T>
2  {
3      private final T value;
4      Box(T value) { this.value = value; }
5      T get() { return value; }
6  }
7  Box<String> box = new Box<>("Java");
8  System.out.println(box.get().toUpperCase());
```

EXPECTED OUTPUT

```
JAVA
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The type parameter T represents a type selected by the caller. For Box, get() has compile-time String type, so calling toUpperCase() is allowed without a manual cast. An attempted Box assignment to Box is rejected at compile time. Generic type parameters use reference types; Box is invalid, but Box works. Type erasure means these parameter types are not usually stored in distinct runtime classes.

PRODUCTION RULE OR FAILURE MODE

These snippets show a local helper class declaration plus caller statements; place the declaration at class or suitable local-class scope before the statements. Prefer meaningful bounds only when operations actually require them.

TRY THIS, THEN REVEAL THE REASON

Challenge: Can a Box be assigned to Box because String extends Object?
Reasoned answer: No. Generic classes are invariant unless a wildcard view expresses a safe relationship.

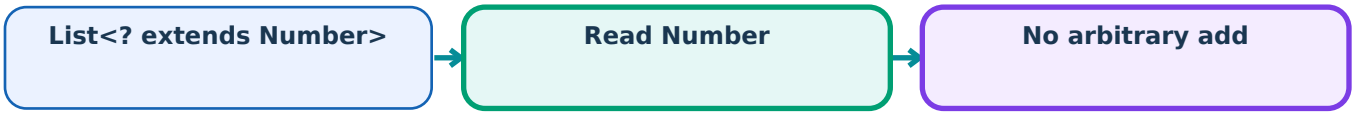
Upper-bounded wildcard: producer extends

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use ? extends T when a generic source produces T values for reading; adding non-null values is usually not type-safe.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<Integer> ints = List.of(2, 4);
List<? extends Number> values = ints;
Number first = values.get(0);
System.out.println(first);
```

EXPECTED OUTPUT / RESULT

2

EXECUTION TRACE / READ THE STEPS

1. Integer extends Number.
2. The upper-bounded wildcard accepts a List<Integer>.
3. get returns a value usable as Number.

DEEPER UNDERSTANDING / WHY IT MATTERS

PECS is a useful mnemonic: Producer Extends, Consumer Super. With ? extends Number, the actual element type might be Integer, Double, or another subtype. Adding a Number could violate that hidden type, so the compiler rejects such additions (except null). This is about type safety rather than whether the backing collection is structurally modifiable.

REAL-WORLD DECISION / EDGE CASE

Method parameter List<? extends Number> is flexible when summing values without modifying the source.

COMMON PITFALL

Assuming extends means the collection itself is mutable or extendable.

PRACTICE / CHANGE ONE THING

Can values.add(3.14) compile for List<? extends Number>?

CHECK YOUR ANSWER

No.

CHAPTER 5 / LESSON 20 OF 23

Lower-bounded wildcard: consumer super

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use ? super T when a collection consumes T values; reads have only Object guarantees.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<Number> destination = new ArrayList<>();
List<? super Integer> sink = destination;
sink.add(7);
System.out.println(destination.get(0));
```

EXPECTED OUTPUT / RESULT

```
7
```

EXECUTION TRACE / READ THE STEPS

1. List<Number> is a valid consumer of Integer values.
2. sink.add(7) is accepted.
3. The destination list now contains integer 7.

DEEPER UNDERSTANDING / WHY IT MATTERS

With ? super Integer, the underlying list might have element type Integer, Number, or Object. Inserting an Integer is safe into any of those, but the generic read type is Object because the exact supertype is unknown. Together with ? extends, this explains the safe design of methods that copy elements from one list into another.

REAL-WORLD DECISION / EDGE CASE

Use wildcard bounds on method parameters where consumers/producers benefit, not indiscriminately on all variables.

COMMON PITFALL

Expecting a List<? super Integer> get() to return Integer without a cast.

PRACTICE / CHANGE ONE THING

What return type can you rely on when reading sink.get(0)?

CHECK YOUR ANSWER

Object.

EXPANSION 12 / AFTER ORIGINAL LESSON 20

PECS in real APIs: copy from a producer to a consumer

WHY THE DISTINCTION MATTERS

Producer Extends, Consumer Super is most useful when a method must accept a collection of subtypes as input and another collection of supertypes as output.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  static <T> void copyAll(
2      List<? extends T> src, List<? super T> dst)
3  {
4      dst.addAll(src);
5  }
6  List<Integer> src = List.of(2, 4);
7  List<Number> dst = new ArrayList<>();
8  copyAll(src, dst);
9  System.out.println(dst);
```

EXPECTED OUTPUT

```
[2, 4]
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The producer may be List while the consumer is List; the method safely reads values of type T from the source and writes them to a destination that can accept T. It would be unsafe to add an arbitrary Number into List, because the actual source list could be a List. When designing APIs, put the most permissive type-safe bounds on parameters to increase reuse.

PRODUCTION RULE OR FAILURE MODE

Using List is appropriate for read-only inspection when element type is irrelevant; it is not a replacement for expressive type parameters in methods that must relate source and destination types.

TRY THIS, THEN REVEAL THE REASON

Challenge: What can safely be added to List without knowing its captured type?
Reasoned answer: Only null (assuming the underlying collection permits null); the compiler rejects arbitrary Number values.

CHAPTER 5 / LESSON 21 OF 23

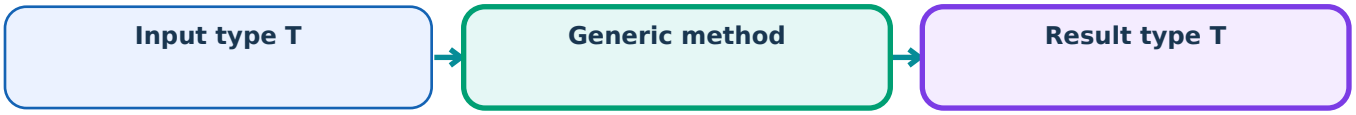
Generic methods and useful bounds

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

A method can declare its own type parameter, enabling reusable operations with compile-time consistency.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
static <T> T first(List<T> items)
{
    return items.get(0);
}
System.out.println(first(List.of("X", "Y")));
```

EXPECTED OUTPUT / RESULT

X

EXECUTION TRACE / READ THE STEPS

1. The method introduces its own type variable T.
2. The compiler infers T as String for this call.
3. It returns X as a String.

DEEPER UNDERSTANDING / WHY IT MATTERS

Type parameters are useful when a result must preserve its input type, not merely because Object would compile. A bound such as <T extends Comparable<? super T>> expresses an operation supported by all accepted types. Use simple signatures when possible; complicated wildcard combinations are a signal to check API design. A generic method's T is not the same as a class-level T unless deliberately connected.

REAL-WORLD DECISION / EDGE CASE

Avoid gratuitous generic abstraction for one-off local code.

COMMON PITFALL

Writing <T> methods when the result has no actual relation to T can mislead callers.

PRACTICE / CHANGE ONE THING

If first receives List<Integer>, what is its inferred return type?

CHECK YOUR ANSWER

Integer.

EXPANSION 11 / AFTER ORIGINAL LESSON 21

Generic methods and bounds: one algorithm, many types

WHY THE DISTINCTION MATTERS

A generic method introduces type parameters of its own. A bound communicates the capabilities the algorithm expects from the type.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  static <T extends Comparable<? super T>> T max(
2      T a, T b)
3  {
4      return a.compareTo(b) >= 0 ? a : b;
5  }
6  System.out.println(max(4, 9));
7  System.out.println(max("cat", "ant"));
```

EXPECTED OUTPUT

```
9
cat
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The bounded type parameter T ensures compareTo is available and returns a value of the input type. Comparable is more flexible than Comparable because a type may inherit comparison behavior defined for a superclass. Java infers T from arguments in these common calls. This is a genuine reusable algorithm, not merely a generic collection declaration.

PRODUCTION RULE OR FAILURE MODE

Avoid adding generic complexity just to be clever. If an API always returns a concrete domain type, use that type. Choose a generic method only when the relationship between inputs and outputs is central to its usefulness.

TRY THIS, THEN REVEAL THE REASON

Challenge: Why is List incompatible with List even though this generic max method accepts both Integer and String calls?
Reasoned answer: A method infers a different T for each call. Assigning one invariant parameterized collection type to another is a separate rule.

Type erasure and practical runtime limitations

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Java generics provide compile-time checks, but most generic type arguments are erased at runtime.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
List<String> strings = new ArrayList<>();
List<Integer> numbers = new ArrayList<>();
System.out.println(strings.getClass() == numbers.getClass());
```

EXPECTED OUTPUT / RESULT

```
true
```

EXECUTION TRACE / READ THE STEPS

1. Both variables reference ArrayList instances.
2. The runtime classes are the same ArrayList class.
3. String and Integer arguments do not create separate runtime classes.

DEEPER UNDERSTANDING / WHY IT MATTERS

You cannot write new T() or test instanceof List<String> because type arguments are not generally reified. Reflection can sometimes inspect declared generic signatures, but it does not turn every object into a distinct runtime generic type. Generic arrays are restricted because arrays and generics enforce type checking differently. Runtime serialization libraries may need a type token to preserve nested generic information.

REAL-WORLD DECISION / EDGE CASE

Do not suppress unchecked casts without a documented safety proof.

COMMON PITFALL

Assuming instanceof List<String> is a valid runtime type test.

PRACTICE / CHANGE ONE THING

Can you construct new T() inside an unconstrained generic method?

CHECK YOUR ANSWER

No.

CHAPTER 5 / LESSON 23 OF 23

Choosing implementations and understanding complexity

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Performance depends on access patterns: ArrayList indexed reads, hash lookup, or ordered tree operations.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Set<String> seen = new HashSet<>();
seen.add("X");
System.out.println(seen.contains("X"));
```

EXPECTED OUTPUT / RESULT

```
true
```

EXECUTION TRACE / READ THE STEPS

1. HashSet stores X.
2. contains performs hash-based membership lookup.
3. The result is true.

DEEPER UNDERSTANDING / WHY IT MATTERS

Big-O is a guide, not a substitute for profiling. ArrayList get is O(1); inserting near the front is O(n). HashMap/HashSet operations are often O(1) average under suitable hashing, while TreeMap/TreeSet are O(log n). LinkedHashMap/LinkedHashSet add predictable iteration order with some overhead. LinkedList is rarely the best default for standard business application Lists.

REAL-WORLD DECISION / EDGE CASE

Prefer clear semantics and measure actual workloads before complex micro-optimization.

COMMON PITFALL

Choosing LinkedList because "linked" sounds faster for generic list operations.

PRACTICE / CHANGE ONE THING

What is the usual default List implementation?

CHECK YOUR ANSWER

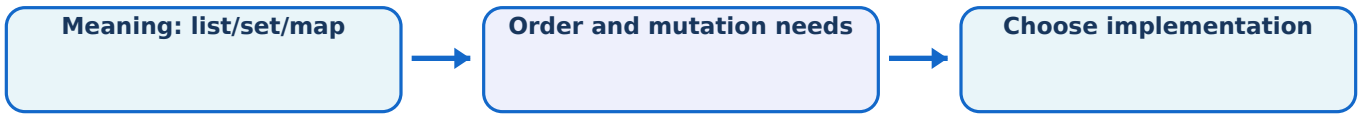
ArrayList.

EXPANSION 13 / AFTER ORIGINAL LESSON 23

Collection choice and costs: practical decision grid

WHY THE DISTINCTION MATTERS

Big-O notation is a starting point, not a substitute for thinking about order, uniqueness, lookup pattern, memory and concurrency.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  List<String> ordered = new ArrayList<>();
2  Set<String> unique = new LinkedHashSet<>();
3  Map<String, Integer> lookup = new HashMap<>();
4  ordered.add("A");
5  unique.add("A");
6  unique.add("A");
7  lookup.put("A", 1);
8  System.out.println(ordered.size() + "," + unique.size());
```

EXPECTED OUTPUT

```
1,1
```

UNDER THE HOOD / FOLLOW THE EXECUTION

ArrayList is an excellent default for ordered sequences. HashSet provides average constant-time membership when hashes distribute well, while LinkedHashSet additionally retains insertion order. HashMap provides average constant-time keyed lookup; LinkedHashMap is useful when reproducible iteration order matters. TreeMap and TreeSet keep a comparator-defined sorted order, generally with logarithmic operations. ArrayDeque supports efficient operations at both ends.

PRODUCTION RULE OR FAILURE MODE

If records are fetched from a database, consider whether filtering, sorting and grouping should occur in SQL rather than loading an enormous collection in memory. Use profiling when performance matters; asymptotic claims do not account for all constant factors.

TRY THIS, THEN REVEAL THE REASON

Challenge: Would you choose HashMap to preserve insertion order in a published response?
Reasoned answer: No. Its iteration order is unspecified; use LinkedHashMap or explicitly sort the output.

Chapter 5 — Master Recall Map

A fast scan of the most important invariants and practical decisions. Each section links back to the lessons you just studied.

JAVA CHAPTER 5 | MASTER INFOGRAPHIC

CORE RULES • EXAMPLES • REAL-WORLD DECISIONS

01 LIST

- ✓ Ordered, duplicates allowed, index-based reads.
- ✓ ArrayList is the useful default.
- ✓ List.of returns unmodifiable collections.

02 SET

- ✓ Uniqueness governed by equality semantics.
- ✓ HashSet no guaranteed order; LinkedHashSet insertion order.
- ✓ TreeSet sorted by comparator.

03 MAP

- ✓ key → value; unique keys.
- ✓ HashMap usually default; LinkedHashMap preserves insertion order.
- ✓ computeIfAbsent and merge simplify updates.

04 HASH CONTRACT

- ✓ Equal objects must have equal hashCode.
- ✓ Avoid mutating keys while inside hash collections.
- ✓ Ordering and equality may disagree.

05 GENERICS

- ✓ List<String> is not List<Object>.
- ✓ ? extends T for producers; ? super T for consumers.
- ✓ Types erased at runtime in common generic APIs.

06 ITERATION & VIEWS

- ✓ Avoid structural mutation in for-each loops.
- ✓ Iterator.remove or removeIf is safe.
- ✓ View != defensive copy.

Real-world checklist before you move on

REVIEW POINT

☐ Does the business requirement imply List, Set, or Map?

REVIEW POINT

☐ Do keys and members satisfy stable equals/hashCode contracts?

REVIEW POINT

☐ Am I relying on an unspecified iteration order?

REVIEW POINT

☐ Did I mutate a collection during for-each traversal?

REVIEW POINT

☐ Is this API returning a snapshot or a live view?

REVIEW POINT

☐ Can a nullable value be mistaken for an absent key?

REVIEW POINT

☐ Do my wildcard bounds reflect producer versus consumer?

NEXT STEP

Run the matching *Demo.java file in the companion IntelliJ/Maven project. Compare actual console output to EXPECTED_OUTPUT.txt; then change inputs and predict behavior before rerunning.

Chapter 6
Exceptions and Resource Management

WHY THIS CHAPTER MATTERS

Production systems do not just follow success paths. APIs fail, parsing fails, files disappear and external resources need cleanup. Good Java code expresses those failures clearly without swallowing causes or leaking resources.

What you will be able to do

• Distinguish checked and unchecked exceptions and select appropriate handling boundaries.

• Throw, wrap and propagate exceptions without losing useful causes.

• Use try-with-resources and understand close order and suppressed failures.

• Read stack traces and preserve meaningful application diagnostics.

HOW TO STUDY

Read the explanation; trace the code before looking at output; test the variation in IntelliJ. Code panels are focused teaching snippets; the companion project contains complete compilable examples.

20 practical lessons • illustrated reference poster • executable companion project

CHAPTER 6 • EXCEPTIONS & RESOURCE MANAGEMENT

Failure paths • error types • causes • cleanup • safe propagation

WHY THIS MATTERS

Core Java building blocks for reliable business applications.

MASTER THESE

- Catch failures you can genuinely handle.
- Preserve the original cause when wrapping exceptions.
- Use try-with-resources for AutoCloseable resources.

KEY TAKEAWAY

FAIL CLEARLY, PRESERVE CAUSES, CLEAN UP

1 Exception Hierarchy

Checked versus unchecked

- Throwable divides into Error and Exception.
- Checked exceptions require catch or throws.
- RuntimeException subclasses are unchecked.

QUICK MODEL

IOException: checked

IllegalArgument: unchecked

Error ≠ routine failure

JAVA EXAMPLE • Illustrative snippet

```
try
{
    throw new IllegalArgumentException("Bad");
}
catch (IllegalArgumentException ex)
{
    System.out.println("Handled");
}
```

OUTPUT Handled

2 try / catch / finally

Handle and cleanup predictably

- Java searches for a compatible catch handler.
- More specific catch types precede broader ones.
- finally normally runs even when exceptions occur.

QUICK MODEL

try: risky code

catch: response

finally: cleanup

JAVA EXAMPLE • Illustrative snippet

```
try
{
    System.out.println(10 / 0);
}
catch (ArithmeticException ex)
{
    System.out.println("Cannot divide");
}
```

OUTPUT Cannot divide

3 throw and throws

Signal a failure versus declare it

- throw creates or propagates a specific exception.
- throws declares a possible checked exception.
- Use meaningful, specific exception types.

QUICK MODEL

throw: action

throws: declaration

No silent failure

JAVA EXAMPLE • Illustrative snippet

```
static void check(int age)
{
    if (age < 0)
    {
        throw new IllegalArgumentException();
    }
}
```

BEHAVIOR Throws if age is negative

4 Cause & Propagation

Keep diagnostic context

- Unchecked failures can propagate through callers.
- Preserve the cause when translating boundaries.
- Catch at a layer that can add useful context.

QUICK MODEL

Stack trace

Cause chain

Good context

JAVA EXAMPLE • Illustrative snippet

```
try
{
    Integer.parseInt("not-a-number");
}
catch (NumberFormatException ex)
{
    System.out.println("Invalid input");
}
```

OUTPUT Invalid input

5 try-with-resources

Automatic close in reverse order

- Resources must implement AutoCloseable.
- Resources close in reverse declaration order.
- Suppressed exceptions retain close-time failures.

QUICK MODEL

Open resources

Run body

Close automatically

JAVA EXAMPLE • Illustrative snippet

```
try (StringReader r = new StringReader("X"))
{
    System.out.println((char) r.read());
}
```

OUTPUT X

6 Handling Policy

Logging, cleanup and boundaries

- Do not catch Exception merely to ignore it.
- Log once at a useful boundary; include context.
- Do not swallow interrupts or mask failures in finally.

QUICK MODEL

Catch specific

Preserve cause

Close resources

JAVA EXAMPLE • Illustrative snippet

```
try
{
    validateInput();
}
catch (IllegalArgumentException ex)
{
    System.out.println("Invalid request");
}
```

BEHAVIOR Invalid request if validation fails

WATCH OUT • COMMON MISTAKES

- Empty catch blocks that hide failures
- Not preserving the cause when wrapping
- Leaking files, sockets or JDBC connections
- Over-catching Exception/Error

QUICK RECALL • BEST PRACTICES

Suppressed exceptions: a close failure is attached when the body already failed.

Try-with-resources should be the default for files, streams and JDBC resources.

CHAPTER 6 / LESSON 01 OF 20

The exception hierarchy in practical Java

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Throwable includes Error and Exception. Most application failures are RuntimeException or checked Exception subclasses.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try
{
    Integer.parseInt("bad");
}
catch (NumberFormatException ex)
{
    System.out.println("Invalid integer");
}
```

EXPECTED OUTPUT / RESULT

```
Invalid integer
```

EXECUTION TRACE / READ THE STEPS

1. parseInt sees invalid digits.
2. It throws a NumberFormatException, a RuntimeException subclass.
3. The matching catch branch prints a clear message.

DEEPER UNDERSTANDING / WHY IT MATTERS

Unchecked exceptions generally represent programming bugs, invariant violations, or errors such as invalid user input that layers may translate. Checked exceptions represent failures the API requires callers to acknowledge. Errors often indicate severe runtime conditions and are not ordinarily caught by application business logic. There is no universal rule to catch every RuntimeException; handle exceptions where meaningful recovery or translation is possible.

REAL-WORLD DECISION / EDGE CASE

Exception handling controls failure flow, not the normal validation logic of every value in the application.

COMMON PITFALL

Writing catch(Throwable) everywhere and hiding severe Errors.

PRACTICE / CHANGE ONE THING

Is NumberFormatException checked? Answer: No; it extends RuntimeException.

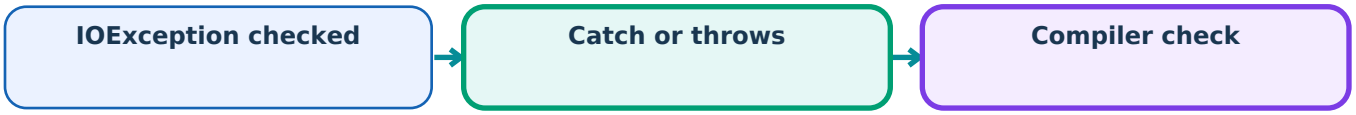
Checked versus unchecked exceptions

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

A checked exception must be caught or declared with throws; an unchecked exception need not be declared.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
static String readText(Path path) throws IOException
{
    return Files.readString(path);
}
```

EXPECTED OUTPUT / RESULT

```
No console output (method definition).
```

EXECUTION TRACE / READ THE STEPS

1. Files.readString can throw IOException.
2. The method declares throws IOException rather than catching it locally.
3. Its callers must handle or propagate the checked exception.

DEEPER UNDERSTANDING / WHY IT MATTERS

Checked exceptions commonly arise in file and some database APIs. Unchecked exceptions such as IllegalArgumentException and NullPointerException can still be caught, but the compiler does not require it. Whether to use checked or unchecked for a custom exception depends on API intent and established conventions. Do not convert every checked exception into an unchecked exception blindly; retain useful information and document the contract.

REAL-WORLD DECISION / EDGE CASE

At service boundaries, decide who owns recovery, retries, user feedback and logging.

COMMON PITFALL

Adding throws Exception to every method hides the actual recoverable failure types.

PRACTICE / CHANGE ONE THING

Must a method declare throws NumberFormatException?

CHECK YOUR ANSWER

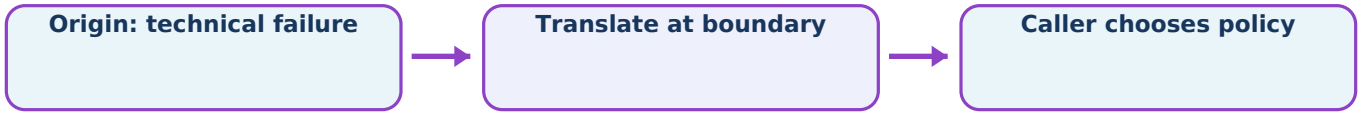
No.

EXPANSION 01 / AFTER ORIGINAL LESSON 02

Checked vs unchecked: decide who can recover

WHY THE DISTINCTION MATTERS

The key design question is not whether an exception is checked but whether the caller has a meaningful, documented recovery choice at its layer.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```java
static int requirePositive(int value)
{
    if (value <= 0)
    {
        throw new IllegalArgumentException("positive only");
    }
    return value;
}
System.out.println(requirePositive(5));
```

EXPECTED OUTPUT

```
5
```

UNDER THE HOOD / FOLLOW THE EXECUTION

IllegalArgumentException is an unchecked exception and is suitable for a violated method precondition. An IOException indicates a failure from an external I/O operation, which Java treats as checked. Checked exceptions force callers to catch or declare; unchecked exceptions are still part of the API contract when they represent expected validation or domain failures. Blindly changing every checked exception into RuntimeException may erase useful context rather than simplifying design.

PRODUCTION RULE OR FAILURE MODE

Document recoverable outcomes: missing input, rejected business rule, transient remote failure and programming defect should not all map to one generic exception. The type, message, cause and layer of handling must communicate what the caller can actually do.

TRY THIS, THEN REVEAL THE REASON

Challenge: Is every unchecked exception a programmer bug?
Reasoned answer: No. An application may deliberately use unchecked domain exceptions for business rule violations or nonrecoverable infrastructure problems.

CHAPTER 6 / LESSON 03 OF 20

try and catch: handle only what you can own

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Catch an exception when the current layer can recover, provide context, or translate it into a meaningful failure.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try
{
    int n = Integer.parseInt("12x");
    System.out.println(n);
}
catch (NumberFormatException ex)
{
    System.out.println("Enter digits only");
}
```

EXPECTED OUTPUT / RESULT

```
Enter digits only
```

EXECUTION TRACE / READ THE STEPS

1. The parse operation throws before printing n.
2. Control jumps to the matching catch.
3. The user-facing message explains the invalid input.

DEEPER UNDERSTANDING / WHY IT MATTERS

The statements after the throwing operation in the try block are skipped. A catch clause should be as narrow as necessary: broad catching can hide unexpected bugs. Handling might mean asking the user to correct input, returning a meaningful HTTP error, retrying a transient operation under policy, or adding context before rethrowing. Logging then rethrowing at every layer can create duplicate logs and noise.

REAL-WORLD DECISION / EDGE CASE

Avoid performing irreversible side effects before validation unless failure compensation is defined.

COMMON PITFALL

Catching Exception then returning an empty successful result.

PRACTICE / CHANGE ONE THING

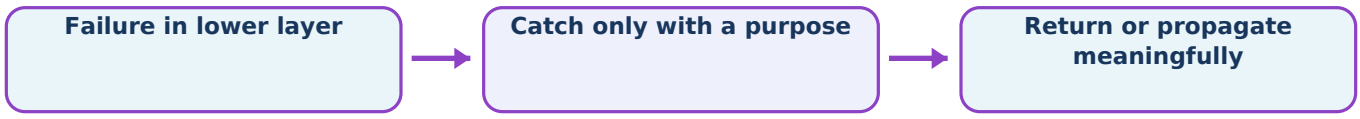
Does System.out.println(n) run after parseInt throws? Answer: No.

EXPANSION 02 / AFTER ORIGINAL LESSON 03

Catch at the right layer: recover, translate or propagate

WHY THE DISTINCTION MATTERS

Catching exceptions too early often causes duplicate logs, invented fallback values and hidden errors. A good catch block has a concrete responsibility.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  static int parsePort(String text)
2  {
3      try
4      {
5          return Integer.parseInt(text);
6      }
7      catch (NumberFormatException ex)
8      {
9          throw new IllegalArgumentException("Invalid port: " + text, ex);
10     }
11 }
12 System.out.println(parsePort("8080"));
```

EXPECTED OUTPUT

```
8080
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The lower-level NumberFormatException is transformed into an input-domain IllegalArgumentException with the original cause preserved. The caller can now describe invalid configuration without losing diagnostic evidence. If the caller is not able to recover, allowing the exception to propagate is often better than catching it solely to log and rethrow. The HTTP entry boundary may convert a known validation exception into a client error while preserving internal details in logs.

PRODUCTION RULE OR FAILURE MODE

Do not expose sensitive raw values in error messages or logs. The example is appropriate for a nonsecret port setting; authentication tokens and personal information require redaction.

TRY THIS, THEN REVEAL THE REASON

Challenge: What is lost by writing throw new IllegalArgumentException("Invalid port"); without including ex?
Reasoned answer: The original NumberFormatException and its stack trace are no longer linked as the cause.

CHAPTER 6 / LESSON 04 OF 20

Catch ordering and multi-catch

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Catch more specific exception types before broader ones; multi-catch handles unrelated alternatives together.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try
{
    Integer.parseInt("x");
}
catch (NumberFormatException ex)
{
    System.out.println("number");
}
catch (RuntimeException ex)
{
    System.out.println("runtime");
}
```

EXPECTED OUTPUT / RESULT

```
number
```

EXECUTION TRACE / READ THE STEPS

1. NumberFormatException is thrown.
2. The first catch matches exactly.
3. Later catch clauses are bypassed.

DEEPER UNDERSTANDING / WHY IT MATTERS

A catch block for RuntimeException before NumberFormatException would make the narrower catch unreachable and fail compilation. Multi-catch uses catch (IOException | ParseException ex) when different exception types share a handling strategy, but alternatives cannot be related by subclass/superclass. Keep catch blocks small enough to reveal what is really recoverable.

REAL-WORLD DECISION / EDGE CASE

Specific-to-general catch ordering makes the failure contract visible to maintainers.

COMMON PITFALL

Putting catch(Exception) before catch(IOException).

PRACTICE / CHANGE ONE THING

Can catch(NumberFormatException | RuntimeException ex) compile? Answer: No; one alternative is a subclass of the other.

CHAPTER 6 / LESSON 05 OF 20

throw creates a failure; throws declares propagation

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

throw raises a particular exception instance; throws in a method signature documents exceptions that may propagate.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
static void requirePositive(int n)
{
    if (n <= 0)
    {
        throw new IllegalArgumentException("must be positive");
    }
}
```

EXPECTED OUTPUT / RESULT

```
No console output (method definition).
```

EXECUTION TRACE / READ THE STEPS

1. The method tests the precondition.
2. For nonpositive inputs it creates an IllegalArgumentException.
3. throw stops ordinary execution of the method.

DEEPER UNDERSTANDING / WHY IT MATTERS

Use exceptions for violations of a method contract that cannot be represented as an ordinary result. Error messages should include safe, actionable context; avoid secrets, passwords, personal data or entire raw request bodies. Validate close to boundaries and centralize response translation in the appropriate layer. Do not use exceptions as a replacement for common branch logic like an absent search result if absence is expected.

REAL-WORLD DECISION / EDGE CASE

IllegalArgumentException is unchecked, so a throws declaration is optional.

COMMON PITFALL

Confusing the throw statement with the throws declaration.

PRACTICE / CHANGE ONE THING

Does execution continue after throw in the same path? Answer: No.

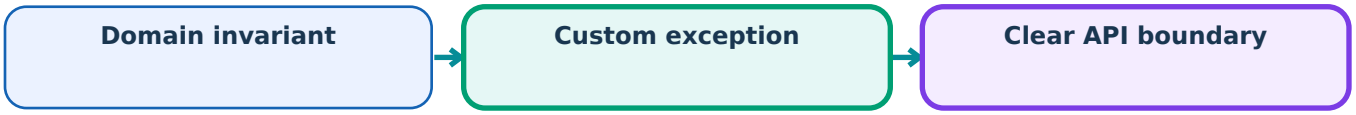
Custom exceptions that communicate intent

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Create a domain-specific exception when its type helps callers recognize or handle a meaningful failure class.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
class InvalidOrderException extends RuntimeException
{
    InvalidOrderException(String message)
    {
        super(message);
    }
}
throw new InvalidOrderException("order already closed");
```

EXPECTED OUTPUT / RESULT

```
Throws InvalidOrderException with message order already closed.
```

EXECUTION TRACE / READ THE STEPS

1. A custom RuntimeException subclass is defined.
2. The constructor passes the message to the superclass.
3. Throwing the exception stops normal flow.

DEEPER UNDERSTANDING / WHY IT MATTERS

Custom exception classes should carry context needed by handlers without exposing sensitive internals. Consider a meaningful error code or typed fields when callers need structured behavior. Avoid introducing dozens of nearly identical exception types that nobody handles differently. An exception name should express a stable business concept rather than implementation detail such as DaoMethodFooException.

REAL-WORLD DECISION / EDGE CASE

Keep custom checked exceptions for APIs that truly require explicit recovery choices.

COMMON PITFALL

Catching a custom exception only to return success and discard the failure.

PRACTICE / CHANGE ONE THING

Does a RuntimeException subclass require callers to declare throws? Answer: No.

CHAPTER 6 / LESSON 07 OF 20

Preserve the cause when translating exceptions

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

When wrapping a lower-level failure, pass the original exception as the cause instead of discarding it.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try
{
    throw new IOException("disk unavailable");
}
catch (IOException ex)
{
    throw new IllegalStateException("could not load config", ex);
}
```

EXPECTED OUTPUT / RESULT

```
Throws IllegalStateException; cause is IOException.
```

EXECUTION TRACE / READ THE STEPS

1. The IOException represents the original low-level failure.
2. The catch adds higher-level context.
3. The new exception retains ex as getCause().

DEEPER UNDERSTANDING / WHY IT MATTERS

Cause chaining preserves the original stack trace so operators can diagnose the underlying failure without losing the business-layer context. Wrapping is useful at abstraction boundaries (file parser to configuration service; SQL driver to repository exception). Avoid wrapping every layer mechanically, producing ten vague nested messages. Ensure the outer exception message adds actual context and not just the original ex.toString().

REAL-WORLD DECISION / EDGE CASE

Never include credentials or unredacted sensitive payloads in exception messages.

COMMON PITFALL

new IllegalStateException("failed") loses the IOException cause.

PRACTICE / CHANGE ONE THING

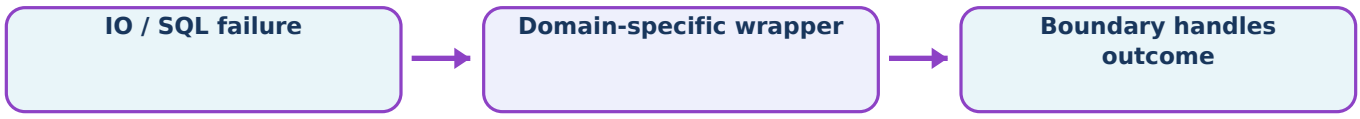
How can a handler access the original IOException?
Answer: By following getCause().

EXPANSION 03 / AFTER ORIGINAL LESSON 07

Cause chains across layers: preserve meaning and diagnosis

WHY THE DISTINCTION MATTERS

A service error may originate in an I/O or database call but needs a domain meaning at the application boundary. Wrapping should preserve the original cause.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  RuntimeException root = new RuntimeException("socket closed");
2  IllegalStateException service = new IllegalStateException(
3      "Could not load customer", root);
4  System.out.println(service.getMessage());
5  System.out.println(service.getCause().getMessage());
```

EXPECTED OUTPUT

```
Could not load customer
socket closed
```

UNDER THE HOOD / FOLLOW THE EXECUTION

An exception cause chain preserves the lower-level explanation while providing context relevant to a higher layer. The outer exception is what upper layers handle; the inner cause helps developers diagnose the source. Stack traces describe the call sites where each exception originated. Avoid wrapping a failure at every method boundary without adding meaningful information, because excessive layers make traces harder to read.

PRODUCTION RULE OR FAILURE MODE

At service boundaries, distinguish expected domain conditions (for example, a duplicate order) from infrastructure failures. Mapping both into one generic 500 response wastes useful information and often misleads callers.

TRY THIS, THEN REVEAL THE REASON

Challenge: Should you normally catch an exception just to log it and immediately rethrow it at every layer? Reasoned answer: No. Log once at a layer that owns the failure context or response, unless a specific lower layer adds otherwise unavailable context.

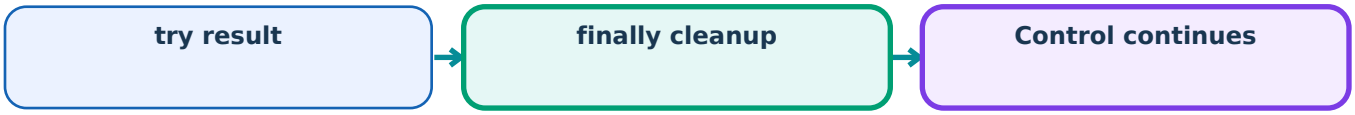
finally runs when leaving try/catch paths

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

finally normally executes when control leaves try/catch, whether by success, exception, or return.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try
{
    System.out.println("work");
}
finally
{
    System.out.println("cleanup");
}
```

EXPECTED OUTPUT / RESULT

```
work
cleanup
```

EXECUTION TRACE / READ THE STEPS

1. The try prints work.
2. Normal completion proceeds through finally.
3. finally prints cleanup.

DEEPER UNDERSTANDING / WHY IT MATTERS

finally is appropriate for some non-resource cleanup, but try-with-resources is preferable for AutoCloseable resources. A return from finally can replace the original return or hide a thrown exception, making debugging hard. Abrupt process termination and JVM failure are exceptions to the simplistic claim that finally always runs. Avoid returning or throwing from finally unless absolutely necessary.

REAL-WORLD DECISION / EDGE CASE

Use finally to restore thread-local context only with carefully specified behavior.

COMMON PITFALL

return in finally suppresses the original exception or return.

PRACTICE / CHANGE ONE THING

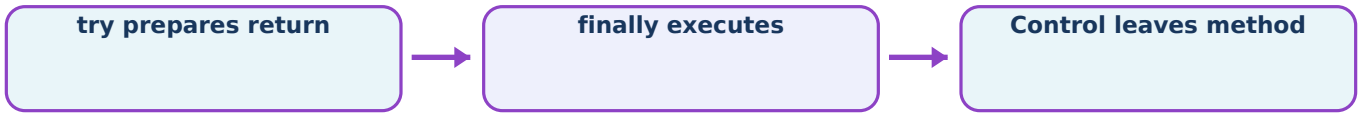
Does finally run after a normal return from try? Answer: Normally yes, before the method actually returns.

EXPANSION 04 / AFTER ORIGINAL LESSON 08

finally and return: cleanup executes, but can change the result

WHY THE DISTINCTION MATTERS

A finally block usually runs when control leaves a try/catch, including via return. Putting return or throw inside finally can replace an earlier result or exception.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  static int value()
2  {
3      try
4      {
5          return 7;
6      }
7      finally
8      {
9          System.out.println("cleanup");
10     }
11 }
12 System.out.println(value());
```

EXPECTED OUTPUT

```
cleanup
7
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The return value is computed before the method exits, but the finally block executes before control returns to the caller. A finally block should perform minimal cleanup and avoid a new return statement: if it returned another value or threw an exception, it could mask the original outcome. The more reliable alternative for AutoCloseable resources is try-with-resources, which provides defined exception suppression rules.

PRODUCTION RULE OR FAILURE MODE

finally is not a guarantee under abrupt VM termination or severe runtime failures. Avoid using it to promise durable external writes or to recover from application termination.

TRY THIS, THEN REVEAL THE REASON

Challenge: If finally itself throws after the try body throws, which exception commonly reaches the caller?
Reasoned answer: The exception from finally can replace/mask the original. Try-with-resources instead retains the try-body failure as primary and suppresses close failures.

CHAPTER 6 / LESSON 09 OF 20

Try-with-resources closes AutoCloseable values

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

try-with-resources automatically closes declared resources, including when an exception occurs.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try (StringReader reader = new StringReader("A"))
{
    System.out.println((char) reader.read());
}
```

EXPECTED OUTPUT / RESULT

```
A
```

EXECUTION TRACE / READ THE STEPS

1. A StringReader is created inside the resource header.
2. read() returns the code unit for A.
3. The resource is closed when the try block ends.

DEEPER UNDERSTANDING / WHY IT MATTERS

Any type implementing AutoCloseable can participate. The compiler generates cleanup behavior that is more reliable and concise than hand-written finally blocks. A resource declared in the try header is closed after the block, regardless of ordinary success or a thrown exception. Closing resource objects does not depend on garbage collection; you should still use try-with-resources even in code where GC eventually reclaims the Java object.

REAL-WORLD DECISION / EDGE CASE

Files, streams, channels, readers, writers and JDBC resources commonly use this pattern.

COMMON PITFALL

Opening a resource without closing it because it will "eventually be garbage collected".

PRACTICE / CHANGE ONE THING

Will the resource close if read() throws?

CHECK YOUR ANSWER

Yes, try-with-resources still attempts close().

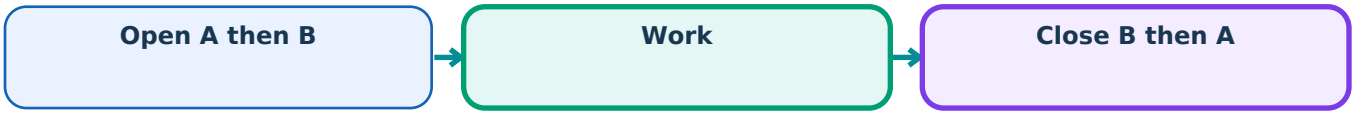
Multiple resources close in reverse order

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

When more than one resource is declared, Java closes them from the last declared to the first.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try (BufferedReader a = new BufferedReader(new StringReader("A"));
     BufferedReader b = new BufferedReader(new StringReader("B")))
{
    System.out.println(a.readLine() + b.readLine());
}
```

EXPECTED OUTPUT / RESULT

```
AB
```

EXECUTION TRACE / READ THE STEPS

1. Resource a is initialized, then b.
2. The body reads one line from each.
3. b is closed first, followed by a.

DEEPER UNDERSTANDING / WHY IT MATTERS

Reverse order mirrors stack-like resource dependencies: an outer wrapper should often close before the underlying resource. If creating the second resource fails, the first resource is still closed. A resource declared by an effectively-final variable can be used in a try-with-resources header (Java 9+). Keep dependencies straightforward so ownership of each resource is clear.

REAL-WORLD DECISION / EDGE CASE

Do not close a resource twice through competing owners when close has side effects or surprising behavior.

COMMON PITFALL

Assuming the first declared resource always closes first.

PRACTICE / CHANGE ONE THING

If resource b construction throws, is a closed?

CHECK YOUR ANSWER

Yes, Java attempts to close the already-created a.

EXPANSION 05 / AFTER ORIGINAL LESSON 10

Resource lifetime and reverse close order

WHY THE DISTINCTION MATTERS

Resources should be closed in the reverse order they were successfully opened, mirroring nested ownership and unwinding.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  class Resource implements AutoCloseable
2  {
3      final String name;
4      Resource(String n) { name = n; System.out.println("open " + n); }
5      public void close() { System.out.println("close " + name); }
6  }
7  try (Resource a = new Resource("A");
8       Resource b = new Resource("B"))
9  {
10     System.out.println("work");
11 }
```

EXPECTED OUTPUT

```
open A
open B
work
close B
close A
```

UNDER THE HOOD / FOLLOW THE EXECUTION

Resources are initialized in declaration order, then closed in reverse order at the end of the try. This matters when B depends on A remaining alive during its close operation. AutoCloseable.close() may throw Exception; narrower interfaces like Closeable declare IOException. A resource closed by try-with-resources must be owned by that scope; do not close a resource that a caller still intends to use.

PRODUCTION RULE OR FAILURE MODE

Try-with-resources also works when an exception occurs in the body. If a later resource constructor fails, already-opened earlier resources are closed automatically.

TRY THIS, THEN REVEAL THE REASON

Challenge: What is closed if Resource A opens but Resource B constructor fails?
Reasoned answer: A is still closed automatically. B was not fully initialized as a resource to close.

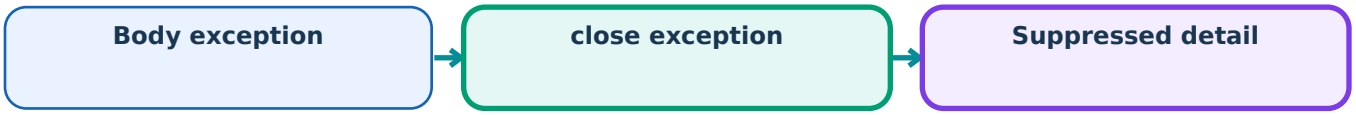
Suppressed exceptions preserve cleanup failures

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

If the try body throws and close also throws, the body exception remains primary and the close exception is suppressed.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try (AutoCloseable resource = () ->
{
    throw new Exception("close");
})
{
    throw new Exception("body");
}
catch (Exception ex)
{
    System.out.println(ex.getMessage());
    System.out.println(ex.getSuppressed().length);
}
```

EXPECTED OUTPUT / RESULT

```
body
1
```

EXECUTION TRACE / READ THE STEPS

1. The try block throws the Exception body.
2. Closing the AutoCloseable throws another Exception close.
3. The body exception stays primary; one suppressed failure is recorded.

REMEMBER

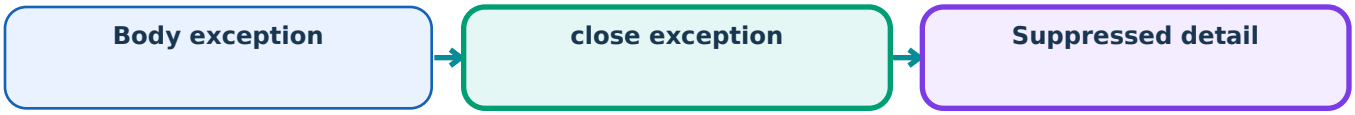
If the try body throws and close also throws, the body exception remains primary and the close exception is suppressed.

Suppressed exceptions preserve cleanup failures — Why it matters

CONCEPT TO RETAIN

If the try body throws and close also throws, the body exception remains primary and the close exception is suppressed.

VISUAL MODEL / HOW THE RULE WORKS



DEEPER UNDERSTANDING / WHY IT MATTERS

This behavior prevents cleanup failures from replacing the more important error in the operation itself. Inspect getSuppressed() when logs show a failing try-with-resources close. The suppressed exception retains its own stack trace. With hand-written finally, a new exception thrown during cleanup may mask the original; try-with-resources provides better default semantics.

REAL-WORLD DECISION / EDGE CASE

Suppressed exceptions matter for real JDBC/IO diagnostic work even if you rarely call getSuppressed manually.

COMMON PITFALL

Logging only the outer message and ignoring both cause and suppressed failures.

PRACTICE / CHANGE ONE THING

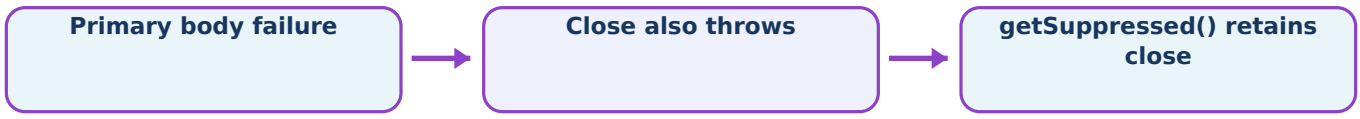
How many suppressed exceptions appear when exactly one close() call throws after the body failure? Answer: One.

EXPANSION 06 / AFTER ORIGINAL LESSON 11

Suppressed exceptions: retain body failure and close failure

WHY THE DISTINCTION MATTERS

A resource may fail during normal work and again during cleanup. The two failures are both important and should not silently overwrite each other.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  AutoCloseable r = () -> { throw new Exception("close"); };
2  try (r)
3  {
4      throw new Exception("body");
5  }
6  catch (Exception ex)
7  {
8      System.out.println(ex.getMessage());
9      System.out.println(ex.getSuppressed()[0].getMessage());
10 }
```

EXPECTED OUTPUT

```
body
close
```

UNDER THE HOOD / FOLLOW THE EXECUTION

Java retains the body exception as primary, and stores the exception thrown by close() in the primary exception's suppressed list. Calling getSuppressed() makes the cleanup failure inspectable, and normal stack-trace printing can include it. In contrast, a handwritten finally that throws a new exception may mask the original. This rule helps diagnose JDBC connection cleanup and file-stream close failures where a root error triggered additional cleanup errors.

PRODUCTION RULE OR FAILURE MODE

If multiple close() calls fail, there can be multiple suppressed exceptions; do not inspect only the first and assume the rest did not happen.

TRY THIS, THEN REVEAL THE REASON

Challenge: Does a try-with-resources statement always have exactly one suppressed exception on failure?
Reasoned answer: No. It may have none or several, depending on which resources fail during close.

CHAPTER 6 / LESSON 12 OF 20

Reading stack traces and cause chains

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

The stack trace gives the throw location and call path. Follow Caused by blocks to find deeper root conditions.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try
{
    Integer.parseInt("?");
}
catch (NumberFormatException ex)
{
    System.out.println(ex.getClass().getSimpleName());
}
```

EXPECTED OUTPUT / RESULT

```
NumberFormatException
```

EXECUTION TRACE / READ THE STEPS

1. Parsing fails with a NumberFormatException.
2. The catch inspects the exception type.
3. A full stack trace would include the throwing path and callers.

DEEPER UNDERSTANDING / WHY IT MATTERS

Read stack traces from the exception type/message to the first relevant application code frame. Library frames may be present, but your own call site often explains why invalid input reached the library. In nested failures, Caused by lines preserve lower-level reasons. In modern Java, suppressed failures may also appear. Logging frameworks normally record the full exception object, not only getMessage().

REAL-WORLD DECISION / EDGE CASE

Avoid displaying raw server stack traces to external users.

COMMON PITFALL

Only recording ex.getMessage() without a stack trace or request correlation information.

PRACTICE / CHANGE ONE THING

What does Caused by usually indicate in a stack trace?
Answer: An underlying wrapped exception.

EXPANSION 07 / AFTER ORIGINAL LESSON 12

Read a stack trace from the first meaningful cause

WHY THE DISTINCTION MATTERS

A stack trace records the failure type, message, and sequence of active calls. “Caused by” sections help identify the original failure rather than merely the highest-level wrapper.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  Exception root = new IllegalArgumentException("bad age");
2  Exception wrapped = new IllegalStateException("parse failed", root);
3  System.out.println(wrapped.getClass().getSimpleName());
4  System.out.println(wrapped.getCause().getMessage());
```

EXPECTED OUTPUT

```
IllegalStateException
bad age
```

UNDER THE HOOD / FOLLOW THE EXECUTION

The outer exception can be a meaningful application-level failure while the cause points to the underlying event. In logs, read the top exception for the impact and then trace the cause chain to find the earliest relevant application frame. A long sequence of framework and reflection frames can obscure the source; use package names and line numbers to locate your code. The top frame is not necessarily the original cause of the entire chain.

PRODUCTION RULE OR FAILURE MODE

Log the exception object, not only ex.getMessage(), when diagnostic stack traces are appropriate. Do not leak internal stack traces to public API clients.

TRY THIS, THEN REVEAL THE REASON

Challenge: Which is generally more useful in a service log: only exception message, or message plus throwable stack trace?
Reasoned answer: The stack trace and causes normally give much better diagnostic context.

Exception boundaries in layered applications

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Catch and translate exceptions where you can produce a meaningful response, not at every method in the call chain.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
static String label(int age)
{
    if (age < 0)
    {
        throw new IllegalArgumentException("age must be >= 0");
    }
    return "valid";
}
```

EXPECTED OUTPUT / RESULT

```
No console output (method definition).
```

EXECUTION TRACE / READ THE STEPS

1. A precondition checks for nonsensical negative age.
2. Invalid input raises a clear exception.
3. The higher-level entry point decides how to communicate the error.

DEEPER UNDERSTANDING / WHY IT MATTERS

A repository can propagate an IO/SQL failure; a service can add meaningful domain context if needed; a web or batch boundary can map known exceptions to a user-facing error or job status. Business methods should not know HTTP-specific codes if they are reused by batch jobs. This separation helps avoid duplicate logging and keeps the failure path understandable.

REAL-WORLD DECISION / EDGE CASE

A controller or batch coordinator should not convert unknown exceptions into successful outcomes.

COMMON PITFALL

catch(Exception) at every helper level returning null.

PRACTICE / CHANGE ONE THING

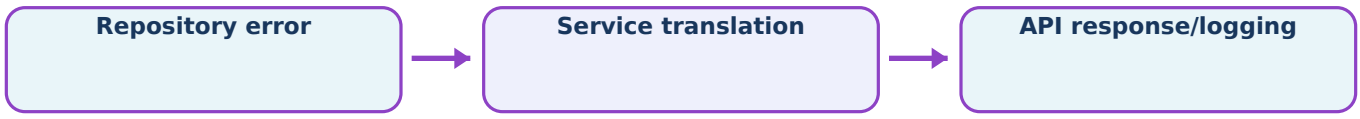
Should a repository return a fake Customer when the database call fails? Answer: Usually no; propagate or translate the failure.

EXPANSION 08 / AFTER ORIGINAL LESSON 13

Exception boundaries: API, service and repository

WHY THE DISTINCTION MATTERS

Failure handling should follow layers. A repository can know SQL and connectivity; a service knows business meaning; an API boundary knows the external response contract.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```java
static String lookup(boolean found)
{
    if (!found)
    {
        throw new NoSuchElementException("Customer missing");
    }
    return "Customer";
}
try
{
    System.out.println(lookup(false));
}
catch (NoSuchElementException ex)
{
    System.out.println("404-like: " + ex.getMessage());
}
```

EXPECTED OUTPUT

```
404-like: Customer missing
```

UNDER THE HOOD / FOLLOW THE EXECUTION

A missing customer is a domain not-found outcome rather than a generic server crash. The final API framework maps it to an HTTP status and safe response body. A database connectivity failure, in contrast, is often an internal failure where retry and telemetry policies matter. The source code above prints a simple “404-like” label for learning; it is not implementing HTTP by itself.

PRODUCTION RULE OR FAILURE MODE

Retries should be applied only to transient, idempotent operations with bounded attempts and backoff. Do not retry validation failures or endlessly repeat a non-idempotent payment write.

TRY THIS, THEN REVEAL THE REASON

Challenge: Where should an API-specific status code normally be chosen?
Reasoned answer: At the transport/API boundary, not in low-level SQL or data-access utilities.

CHAPTER 6 / LESSON 14 OF 20

Validate expected cases without using exceptions as loops

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use ordinary branches for expected alternatives, exceptions for contract violations or failures that block normal work.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Map<String, String> names = Map.of("A", "Ada");
String found = names.get("B");
System.out.println(found == null ? "missing" : found);
```

EXPECTED OUTPUT / RESULT

```
missing
```

EXECUTION TRACE / READ THE STEPS

1. The map contains only A.
2. get(B) returns null for this implementation.
3. The conditional prints missing as a normal expected case.

DEEPER UNDERSTANDING / WHY IT MATTERS

An absent record may be ordinary in a search operation but exceptional in a command that requires the record to exist. Define the API contract rather than enforcing a single pattern everywhere. Returning Optional or a status/result object is often suitable for expected absence, while invalid operation states can use specific exceptions. Exception creation and stack traces are costlier than ordinary branching, but correctness and API clarity matter more than micro-optimization.

REAL-WORLD DECISION / EDGE CASE

Do not catch NullPointerException as a routine null-checking technique.

COMMON PITFALL

Using exceptions for every "if not found" case when absence is routine.

PRACTICE / CHANGE ONE THING

Is a missing result always an exception?

CHECK YOUR ANSWER

No. It depends on the operation contract.

CHAPTER 6 / LESSON 15 OF 20

Logging exceptions once with useful context

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Log a failure at the boundary that owns its reporting; pass the exception object to retain stack and causes.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try
{
    Integer.parseInt("bad");
}
catch (NumberFormatException ex)
{
    System.out.println("Rejected invalid input");
}
```

EXPECTED OUTPUT / RESULT

```
Rejected invalid input
```

EXECUTION TRACE / READ THE STEPS

1. The parse fails.
2. The catch handles that known validation failure.
3. For a real service, the boundary would record structured context as needed.

DEEPER UNDERSTANDING / WHY IT MATTERS

Production logging frameworks accept an exception as a dedicated argument and preserve its stack trace. Include correlation ID, operation name and safe identifiers, not entire payloads or secrets. If you catch solely to log and rethrow at every layer, the same failure produces multiple redundant logs. Decide whether the exception is already expected/handled; expected validation errors may not need ERROR-level full stack traces.

REAL-WORLD DECISION / EDGE CASE

Protect logs from credential leakage and high-cardinality sensitive context.

COMMON PITFALL

logger.error(ex.getMessage()) discards the stack trace.

PRACTICE / CHANGE ONE THING

Should every helper method log before rethrowing? Answer: No. Usually log at the appropriate reporting boundary.

CHAPTER 6 / LESSON 16 OF 20

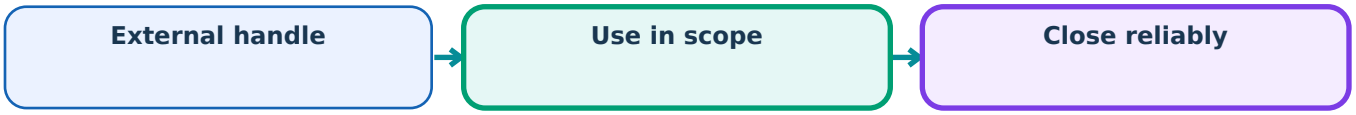
Resource safety beyond files

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Sockets, DB connections, streams and HTTP responses may hold resources that need deterministic cleanup.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try (StringReader reader = new StringReader("OK"))
{
    System.out.println((char) reader.read());
}
```

EXPECTED OUTPUT / RESULT

0

EXECUTION TRACE / READ THE STEPS

1. A resource is created in a bounded scope.
2. One code unit is read and printed.
3. The resource is closed automatically after the block.

DEEPER UNDERSTANDING / WHY IT MATTERS

Garbage collection only reclaims reachable Java objects under its own timing; it is not a deterministic resource release mechanism. Files, network sockets and database connections typically represent operating-system or remote resources. Resource ownership needs to be explicit, particularly with wrappers. Not every class with a close method is safe to reuse after close; follow its contract.

REAL-WORLD DECISION / EDGE CASE

Close JDBC Connection, Statement and ResultSet according to ownership; data-access frameworks may manage lifecycle for you.

COMMON PITFALL

Assuming GC promptly closes database connections during a busy request.

PRACTICE / CHANGE ONE THING

Why is try-with-resources preferred over waiting for GC?

CHECK YOUR ANSWER

It deterministically invokes close when leaving the scope.

CHAPTER 6 / LESSON 17 OF 20

IO exceptions, Path and character encodings

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Use Path and Files APIs with explicit Charset where the data contract requires it; handle IOException at an appropriate boundary.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
Path temp = Files.createTempFile("java-ch6-", ".txt");
try
{
    Files.writeString(temp, "Hi", StandardCharsets.UTF_8);
    System.out.println(Files.readString(temp, StandardCharsets.UTF_8));
}
finally
{
    Files.deleteIfExists(temp);
}
```

EXPECTED OUTPUT / RESULT

```
Hi
```

EXECUTION TRACE / READ THE STEPS

1. The program creates a temporary file.
2. It writes and reads UTF-8 text.
3. finally deletes the file after reading or on failure.

REMEMBER

Use Path and Files APIs with explicit Charset where the data contract requires it; handle IOException at an appropriate boundary.

CHAPTER 6 / LESSON 17 / DEEP DIVE

IO exceptions, Path and character encodings — Why it matters

CONCEPT TO RETAIN

Use Path and Files APIs with explicit Charset where the data contract requires it; handle IOException at an appropriate boundary.

VISUAL MODEL / HOW THE RULE WORKS



DEEPER UNDERSTANDING / WHY IT MATTERS

The Files API performs many small operations safely without manually opening streams, but it can still throw IOException. UTF-8 is generally the right explicit interchange choice. Platform default encodings and line endings can lead to surprises across environments. Even cleanup can fail; decide how a cleanup IOException should affect the operation. Paths also need validation against your security boundary to prevent unintended file access.

REAL-WORLD DECISION / EDGE CASE

For a complex stream of data, prefer try-with-resources around the actual Reader/Writer rather than hand-managed close calls.

COMMON PITFALL

Silently reading text with an undocumented platform-specific encoding.

PRACTICE / CHANGE ONE THING

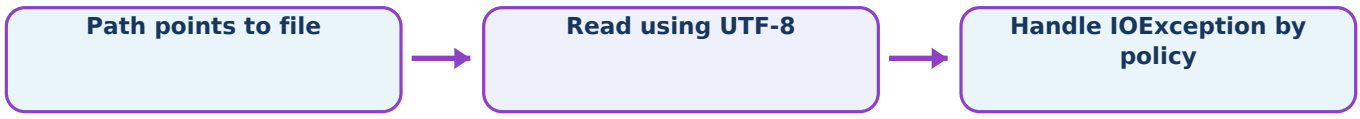
What should you specify when exchanging text between machines? Answer: An explicit charset such as StandardCharsets.UTF_8.

EXPANSION 09 / AFTER ORIGINAL LESSON 17

File boundaries: Path, UTF-8 and meaningful IO failures

WHY THE DISTINCTION MATTERS

Text I/O should use an explicit character encoding and deterministic resource ownership. Paths can be invalid, missing, inaccessible or replaced while the program runs.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  Path p = Path.of("notes.txt");
2  System.out.println(p.getFileName());
3  System.out.println(StandardCharsets.UTF_8.name());
```

EXPECTED OUTPUT

```
notes.txt
UTF-8
```

UNDER THE HOOD / FOLLOW THE EXECUTION

Path represents a filesystem path; it does not imply that the file exists. Files.readString(path, UTF_8) reads the entire file into memory and throws IOException for many failures. Use Files.newBufferedReader with try-with-resources for large or line-by-line input. UTF-8 should be chosen explicitly when the surrounding file format requires it. Distinguish a missing optional configuration file from a missing required business input.

PRODUCTION RULE OR FAILURE MODE

Do not assume that checking Files.exists and then opening the file is atomic; another actor may change the filesystem between those steps. Handle the actual file operation’s exception.

TRY THIS, THEN REVEAL THE REASON

Challenge: Would Files.readString be ideal for a multi-gigabyte log file?
Reasoned answer: No. Stream line by line with a buffered reader; loading the full file can exhaust memory.

When finally or close itself fails

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

A cleanup exception can obscure an earlier failure if coded carelessly; prefer try-with-resources where it applies.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try
{
    System.out.println("task");
}
finally
{
    System.out.println("finally");
}
```

EXPECTED OUTPUT / RESULT

```
task
finally
```

EXECUTION TRACE / READ THE STEPS

1. The task writes its message.
2. The finally block runs during normal completion.
3. No exception is replaced in this example.

DEEPER UNDERSTANDING / WHY IT MATTERS

If a finally block throws, it can replace an exception from the try block and obscure the failure you cared about. That is a major reason resource cleanup via try-with-resources is preferable. For non-AutoCloseable cleanup, handle failures carefully while preserving the original failure whenever appropriate. Suppressed exceptions or explicit cause chaining help retain diagnostic information.

REAL-WORLD DECISION / EDGE CASE

A cleanup action must not blindly transform every failure into success.

COMMON PITFALL

Returning from finally can silently override both return values and exceptions.

PRACTICE / CHANGE ONE THING

Why is throw inside finally risky? Answer: It can mask the original failure.

Testing error paths with assertions

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

Tests should verify exception type and useful message/behavior instead of checking only that an operation fails somehow.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
static int parseAge(String text)
{
    return Integer.parseInt(text);
}
System.out.println(parseAge("25"));
```

EXPECTED OUTPUT / RESULT

```
25
```

EXECUTION TRACE / READ THE STEPS

1. A normal input parses successfully.
2. The result is returned as int 25.
3. A unit test should additionally check bad input raises NumberFormatException.

DEEPER UNDERSTANDING / WHY IT MATTERS

In JUnit 5, assertThrows(NumberFormatException.class, () -> parseAge("bad")) verifies the contract, and the returned exception lets you inspect its message or cause. Avoid brittle assertions on full stack traces or incidental JDK error text. Test that resources close when an operation throws, ideally with a small recording AutoCloseable fake. For layered applications also test how known failures are translated at API boundaries.

REAL-WORLD DECISION / EDGE CASE

Error-path coverage is part of correctness; happy-path-only tests miss cleanup and translation bugs.

COMMON PITFALL

Writing tests that assert catch(Exception) succeeds, allowing unrelated bug types.

PRACTICE / CHANGE ONE THING

What JUnit assertion checks a specific thrown exception type?

CHECK YOUR ANSWER

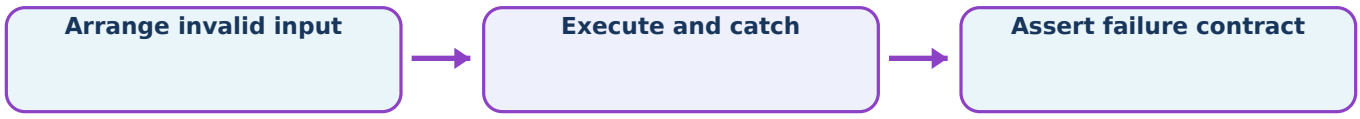
assertThrows(ExpectedType.class, () -> call()).

EXPANSION 10 / AFTER ORIGINAL LESSON 19

Testing failure paths without external frameworks

WHY THE DISTINCTION MATTERS

Success-only tests miss important guarantees. Validate exception type, cause preservation, state consistency, and cleanup effects after a failure.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  boolean caught = false;
2  try
3  {
4      Integer.parseInt("oops");
5  }
6  catch (NumberFormatException ex)
7  {
8      caught = true;
9  }
10 System.out.println(caught);
```

EXPECTED OUTPUT

```
true
```

UNDER THE HOOD / FOLLOW THE EXECUTION

This lightweight check verifies that invalid numeric text throws the expected exception. In production projects, use JUnit assertThrows to express this test clearly and inspect the returned exception’s message/cause where meaningful. When testing resources, use a fake AutoCloseable that records close calls; confirm it closes on both success and failure. For transactional systems, assert that partially completed operations do not leave invalid state.

PRODUCTION RULE OR FAILURE MODE

Do not write tests coupled to exact full stack-trace strings or unstable low-level error messages. Assert stable exception types, domain codes or documented substrings instead.

TRY THIS, THEN REVEAL THE REASON

Challenge: What else should a test check when an operation opens a resource and then throws?
Reasoned answer: That the resource was closed, the correct primary exception propagated, and any close failure was preserved as suppressed.

CHAPTER 6 / LESSON 20 OF 20

Interruption and cancellation must not be swallowed

RULE · VISUAL MODEL · CODE · OUTPUT · DEEP REASONING · PRACTICE

THE CORE RULE

InterruptedException signals a request to stop blocking work; restore interruption when you cannot propagate it.

VISUAL MODEL / HOW THE RULE WORKS



JAVA CODE / ALLMAN BRACES

```
try
{
    Thread.sleep(1);
}
catch (InterruptedException ex)
{
    Thread.currentThread().interrupt();
    System.out.println("interrupted");
}
```

EXPECTED OUTPUT / RESULT

```
Usually no output; prints interrupted only if interrupted.
```

EXECUTION TRACE / READ THE STEPS

1. sleep may complete normally after about one millisecond.
2. If interrupted, it throws InterruptedException and clears the flag.
3. The handler restores the flag so callers can observe cancellation.

DEEPER UNDERSTANDING / WHY IT MATTERS

Most web and batch developers meet interruption inside thread pools, task scheduling or blocking APIs. Catching InterruptedException and doing nothing can make shutdown and cancellation unreliable. If your method can declare throws InterruptedException, propagation is often the cleanest approach; otherwise restore Thread.currentThread().interrupt() and exit or respond appropriately. Full concurrency design belongs in the later concurrency chapter.

REAL-WORLD DECISION / EDGE CASE

Never retry interrupted blocking work indefinitely without respecting cancellation policy.

COMMON PITFALL

catch(InterruptedException ignored) {} loses the cancellation signal.

PRACTICE / CHANGE ONE THING

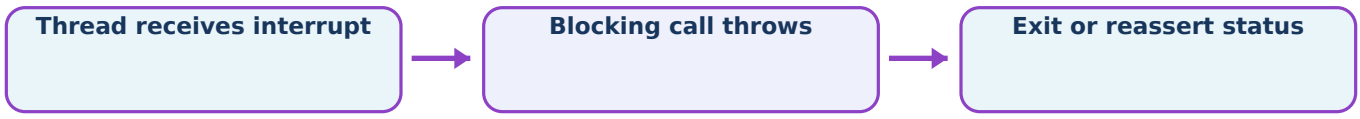
Why call Thread.currentThread().interrupt() in a catch that cannot rethrow? Answer: To restore the interruption status for higher-level code.

EXPANSION 11 / AFTER ORIGINAL LESSON 20

InterruptedException: preserve cancellation signals

WHY THE DISTINCTION MATTERS

Interruption is a cooperative thread-cancellation mechanism. Swallowing InterruptedException can keep a task running after its owner requested shutdown.



EXECUTABLE JAVA IDEA / PUT INSIDE main() UNLESS SHOWN OTHERWISE

```
1  Thread.currentThread().interrupt();
2  try
3  {
4      Thread.sleep(10);
5  }
6  catch (InterruptedException ex)
7  {
8      Thread.currentThread().interrupt();
9      System.out.println("cancelled");
10 }
11 System.out.println(Thread.currentThread().isInterrupted());
```

EXPECTED OUTPUT

```
cancelled
true
```

UNDER THE HOOD / FOLLOW THE EXECUTION

An interruptible blocking call often throws InterruptedException and clears the interrupt status. If you cannot propagate InterruptedException due to your API, restore the interrupt flag with Thread.currentThread().interrupt() and stop or return appropriately. Interruption does not forcibly terminate arbitrary code; it is a request that cooperating components honor. Treat cancellation as part of correct resource management and shutdown design.

PRODUCTION RULE OR FAILURE MODE

Avoid retrying a blocking call forever after interruption. That effectively ignores cancellation and can delay executor shutdown. Running this snippet on an application main thread changes its interrupt flag, so isolated tests should not reuse that thread without cleanup.

TRY THIS, THEN REVEAL THE REASON

Challenge: If you catch InterruptedException and merely print a message, what may be lost?
Reasoned answer: The interrupt signal and the caller’s intent to cancel work may be lost, leaving code running unexpectedly.

Chapter 6 — Master Recall Map

A fast scan of the most important invariants and practical decisions. Each section links back to the lessons you just studied.

JAVA CHAPTER 6 | MASTER INFOGRAPHIC

CORE RULES • EXAMPLES • REAL-WORLD DECISIONS

01 EXCEPTION TYPES

- ✓ Throwable → Error and Exception.
- ✓ RuntimeException is unchecked.
- ✓ Checked failures need catch or throws.

02 CATCH & PROPAGATE

- ✓ Catch only when you can handle or translate.
- ✓ Preserve the cause when wrapping.
- ✓ Order catches from specific to general.

03 TRY WITH RESOURCES

- ✓ AutoCloseable closes automatically.
- ✓ Resources close in reverse creation order.
- ✓ Suppressed close exceptions remain accessible.

04 API BOUNDARIES

- ✓ Validate requests; translate known failures.
- ✓ Do not expose sensitive internal details.
- ✓ Avoid catch-all success responses.

05 STACK TRACES

- ✓ Top frames usually point to throw site.
- ✓ Read Caused by chain for root context.
- ✓ Preserve interruption and cancellation.

06 CLEANUP

- ✓ GC does not close files/sockets reliably.
- ✓ Use timeouts and explicit cleanup.
- ✓ Test both success and failure paths.

Real-world checklist before you move on

REVIEW POINT

☐ Do I catch the specific type I can actually handle?

REVIEW POINT

☐ Did I preserve the original cause when translating?

REVIEW POINT

☐ Can cleanup throw and mask the first failure?

REVIEW POINT

☐ Are files, streams, sockets and DB handles closed deterministically?

REVIEW POINT

☐ Are exceptions logged once with useful safe context?

REVIEW POINT

☐ Do error paths have unit or integration tests?

REVIEW POINT

☐ Do I preserve interruption or cancellation signals?

NEXT STEP

Run the matching *Demo.java file in the companion IntelliJ/Maven project. Compare actual console output to EXPECTED_OUTPUT.txt; then change inputs and predict behavior before rerunning.